CPE333 Operating Systems

สรุปเนื้อหาสอบกลางภาค

ขอบเขต Lecture 1–6

หัวข้อที่ครอบคลุม

1	Introduction to Operating System
2	Process and Process APIs
3	Scheduling
4	Memory Concept and Contiguous Memory Allocation
5	Noncontiguous Memory Allocation
6	Virtual Memory

> **ภาพรวมเนื้อหาสอบกลางภาค**
>
> เริ่มจากบทบาทของ OS และ System Call แล้วต่อไปยัง process การสลับบริบท และการจัดสรร CPU ก่อนเชื่อมสู่การจัดการหน่วยความจำ ตั้งแต่ contiguous allocation, segmentation และ paging ไปจนถึง virtual memory และ page fault
> **ทักษะหลัก:** อธิบายเหตุผล ไล่สถานะหรือ algorithm ตามลำดับ และคำนวณเวลา/ที่อยู่จากเงื่อนไขของโจทย์

วิธีอ่าน: เริ่มจากแนวคิดและพฤติกรรมใน Lecture อ่านภาพประกอบควบคู่คำอธิบาย แล้วฝึกไล่ขั้นตอนหรือคำนวณจากตัวอย่าง ก่อนทบทวนจุดที่มักสับสนท้ายบท

ขอบเขตเนื้อหาและทักษะที่ควรได้

เนื้อหาเรียงตาม Lecture 1–6 และปิดท้ายด้วยบททบทวน เลขหัวข้อเริ่มใหม่ในแต่ละ Lecture เช่น 2.1 และ 3.1 เพื่อให้เห็นขอบเขตของแต่ละบทชัดเจน

บทเรียน	หัวข้อ	เมื่ออ่านจบ ควรทำอะไรได้
1	Introduction to Operating System	อธิบายบทบาทของ OS, virtualization, User/Kernel Mode, System Call, concurrency และ persistence
2	Process and Process APIs	อธิบาย process/PCB ไล่ state และ context switch รวมถึงใช้ fork, wait, exec และ pipes
3	Scheduling	คำนวณ turnaround/response และไล่ FIFO, SJF, STCF, RR, MLFQ และ Lottery จาก timeline
4	Memory Concept and Contiguous Memory Allocation	แปลง logical/physical address และวิเคราะห์ Base/Bounds, relocation และ fragmentation
5	Noncontiguous Memory Allocation	ไล่การแปล address ด้วย Segmentation/Paging และอธิบาย page table กับ TLB
6	Virtual Memory	ไล่ขั้นตอน page fault และวิเคราะห์ page replacement, EAT, thrashing และ working set
ทบทวน	เชื่อมโยงทั้งรายวิชา	เชื่อม hardware, OS, process, scheduler และ memory พร้อมทวนสูตรและจุดที่มักสับสน

วิธีอ่านภาพประกอบ

เริ่มจากดูว่าในภาพมีองค์ประกอบใดบ้าง แล้วไล่ตามทิศทางของลูกศรเพื่อดูว่าข้อมูลหรือสถานะเปลี่ยนอย่างไร จากนั้นอ่านคำอธิบายข้างภาพเพื่อเชื่อมแต่ละขั้นกับหลักการ หากโจทย์มีเงื่อนไขหรือข้อสมมติสำคัญ จะระบุไว้ใกล้กับตัวอย่างนั้น

Contents

ขอบเขตเนื้อหาและทักษะที่ควรได้ **1**

Lecture 1 — Introduction to Operating System **5**

1.1 Operating System อยู่ตรงไหนในระบบคอมพิวเตอร์ 5

1.2 System Call: ทำไมโปรแกรมทั่วไปเข้าถึง hardware โดยตรงไม่ได้ 6

1.3 Virtualization: ทำให้ physical resource ดูเหมือน resource ที่จัดการได้ง่ายขึ้น 8

1.4 OS as Resource Manager 9

1.5 Concurrency: เมื่อหลายสิ่งดำเนินไปพร้อมกัน 9

1.6 Persistence: ข้อมูลต้องอยู่รอดแม้ memory จะหาย 9

1.7 สิ่งที่ต้องคำนึงเมื่อออกแบบ OS 10

1.8 Historical Motivation 10

1.9 Chapter Summary 10

Lecture 2 — Process and Process APIs **12**

2.1 From Program to Process 12

2.2 Process = (P, D, C, S) 12

2.3 Process Address Space และ Process Descriptor 13

2.4 Process State: Ready, Running, Blocked 13

2.5 Typical Process APIs และ OS Data Structures 14

2.6 CPU Virtualization Challenge: Performance + Control 15

2.7 2) Switch Between Processes — Scheduler vs Context Switch 15

2.8 3) OS จะเอา CPU กลับมาได้อย่างไร 16

2.9 UNIX Process APIs 17

2.10 Chapter Summary 20

Lecture 3 — Scheduling **22**

3.1 Scheduling คือ Policy สำหรับแบ่งปันทรัพยากร 22

3.2 Scheduling Metrics 22

3.3 FIFO / FCFS 23

3.4 Shortest Job First (SJF) 24

3.5 เมื่อ Arrival Time ต่างกัน: STCF / Preemptive SJF 24

3.6 Round Robin (RR) 25

3.7 Trade-off ระหว่าง Turnaround กับ Response 26

3.8 นำ I/O มาพิจารณาในการจัดตาราง 26
3.9 เมื่อไม่รู้ Job Length ล่วงหน้า: Multi-Level Feedback Queue (MLFQ) 26
3.10 Proportional-Share และ Lottery Scheduling 28
3.11 Scheduling Comparison 29
3.12 Chapter Summary 29
Lecture 4 — Memory Concept and Contiguous Memory Allocation 30
4.1 Memory Hierarchy และหน้าที่ของ OS 30
4.2 Memory / Address Space Abstraction 30
4.3 Address Translation / Address Binding 31
4.4 Programming API Perspective: code, stack, heap 31
4.5 Memory Management and Organization 32
4.6 Single Partition Allocation และ Boundary Scheme 32
4.7 Multiple Partition Allocation 33
4.8 Static vs Dynamic Relocation 33
4.9 Fixed-Size Partitions: Internal Fragmentation 34
4.10 Flexible-Size Partitions: External Fragmentation 35
4.11 Free List และ lifecycle ของ memory 36
4.12 Trailing Lecture Material: C Storage Classes 36
4.13 Chapter Summary 36
Lecture 5 — Noncontiguous Memory Allocation 38
5.1 ทำไมต้อง Noncontiguous Allocation 38
5.2 Segmentation Scheme 38
5.3 Paging 40
5.4 Page Table และการแปล Address 41
5.5 Page Table Size Problem 42
5.6 Multi-Level Page Table 42
5.7 Inverted Page Table 43
5.8 Hashed Page Table 43
5.9 Translation Lookaside Buffer (TLB) 43
5.10 Page Size Trade-offs 44
5.11 เปรียบเทียบ Segmentation กับ Paging 45

5.12 Hybrid Segmentation + Paging 45

5.13 Chapter Summary 46

Lecture 6 — Virtual Memory 47

6.1 เหตุผลที่ใช้ Virtual Memory: Address Space ใหญ่กว่า Physical Memory 47

6.2 Process Switching ต่างจาก Process Swapping 48

6.3 Present Bit และ Page Fault 48

6.4 Page Selection Policies — เมื่อใดควรเอา page เข้า 50

6.5 Page Replacement — เมื่อ memory เต็มควรนำ page ใดออก 50

6.6 LRU Approximation และ Second Chance 57

6.7 Other Replacement Policies 58

6.8 Effective Access Time (EAT) 58

6.9 Thrashing 58

6.10 การจัดสรร Frame: Global กับ Local 59

6.11 Working Set 59

6.12 ประเด็นเชื่อมโยงท้ายบท: TLB Reach, Locality และ Page Size 60

6.13 Chapter Summary 61

CPE333 FINAL REVIEW 62

R.1 แผนภาพรวมแนวคิดของ OS 62

R.2 เปรียบเทียบ Mechanism กับ Policy 62

R.3 ลำดับการเปลี่ยน State และ Flow สำคัญ 63

R.4 Formula Sheet 63

R.5 ตารางเปรียบเทียบ Scheduling 64

R.6 ตารางเปรียบเทียบการจัดการ Memory 64

R.7 เปรียบเทียบโครงสร้าง Page Table 64

R.8 เปรียบเทียบ Page Replacement 65

R.9 จุดที่มักสับสน 65

R.10 คำถามทบทวน 30 ข้อ 66

R.11 Last-Minute Trace Drills 66

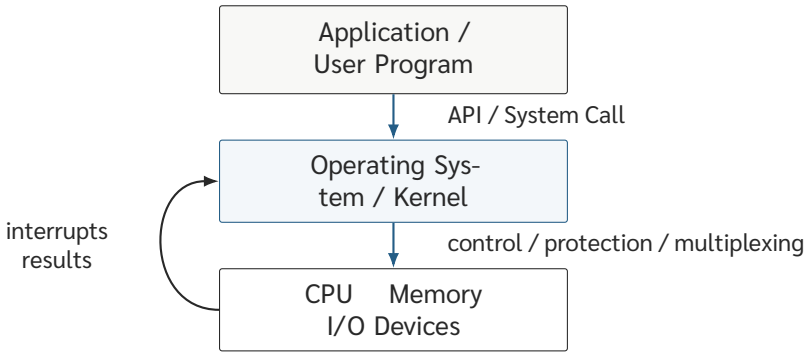
Lecture 1 — Introduction to Operating System

อ้างอิง: Lecture 1 หน้า 1–15 | บทบาทของ OS ระบบเรียกใช้บริการ การทำเสมือน และการจัดการทรัพยากร

1.1 Operating System อยู่ตรงไหนในระบบคอมพิวเตอร์

เนื้อหาจาก Lecture

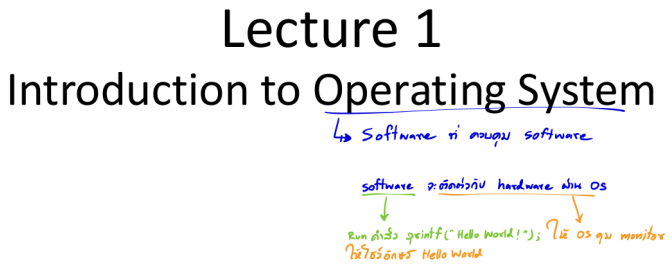
Operating System (OS) เป็นซอฟต์แวร์ที่ทำหน้าที่เป็นตัวกลางระหว่างโปรแกรมกับทรัพยากรฮาร์ดแวร์ โปรแกรมจึงไม่ต้องควบคุม CPU, memory, disk, display หรือ network device โดยตรงทุกครั้งที่ต้องการใช้งาน หมายเหตุในสไลด์หน้าแรกชี้ให้เห็นว่าโปรแกรมใช้ hardware ผ่าน OS ตัวอย่าง `printf("Hello World!")` แสดงให้เห็นว่าคำสั่งระดับโปรแกรมต้องอาศัยบริการของระบบก่อนข้อความจะแสดงผลบนอุปกรณ์



รายละเอียดสำคัญจากสไลด์

Lecture 1 หน้า 3 — ทบทวน Fetch / Decode / Execute สไลด์เชื่อมกับ Computer Architecture โดยอธิบายว่า processor ดึง instruction จาก memory (fetch) แปลความหมายของคำสั่ง (decode) แล้วจึง execute หรือเขียนผลลัพธ์กลับ จุดนี้ช่วยให้เห็นว่า OS ไม่ได้ execute instruction แทน CPU แต่จัด environment และควบคุมการทำงานรอบ instruction stream

ภาพประกอบ 1.1 Lecture 1 หน้า 1 — Software → OS → Hardware



จุดสังเกต

- ลูกศรเขียนมือชี้ว่า user software ไม่ได้ติดต่อ hardware โดยตรงอย่างไร้ข้อจำกัด
- ตัวอย่าง `printf()` สื่อเส้นทางจาก program ไปยัง service ด้านล่าง ไม่ได้หมายความว่า printf = system call ทุกกรณี
- ใช้ภาพนี้เป็นแบบจำลองตั้งต้นก่อนอ่านบทต่อไป

ที่มา: Lecture 1 หน้า 1 พร้อมหมายเหตุเขียนมือในสไลด์

OS ไม่ได้มีหน้าที่เพียงทำให้โปรแกรมเริ่มทำงาน แต่ยังต้องแบ่งทรัพยากรที่ใช้ร่วมกันให้โปรแกรมแต่ละตัวทำงานได้อย่างเป็นอิสระ พร้อมรักษาประสิทธิภาพ การควบคุม และการแยกขอบเขตของระบบ

1.2 System Call: ทำไมโปรแกรมทั่วไปเข้าถึง hardware โดยตรงไม่ได้

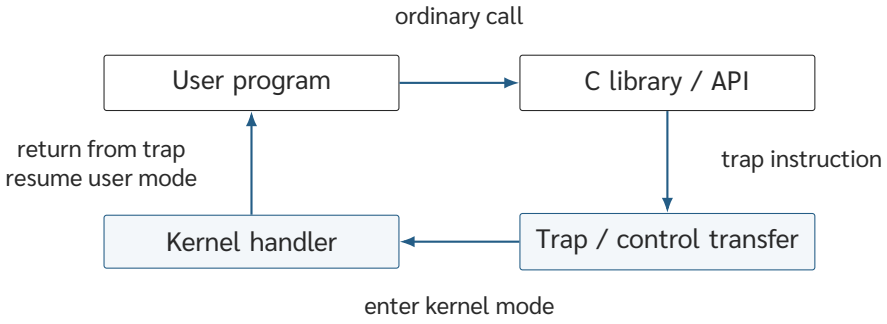
Lecture แนะนำ **User Mode**, **Kernel Mode** และ **System Call** ตั้งแต่ต้น เพื่ออธิบายว่า operation ที่มีสิทธิ์สูง เช่น การอ่านไฟล์หรือส่งข้อมูลผ่าน network ต้องผ่าน OS

	User Mode	Kernel Mode
ผู้ใช้งานหลัก	application / normal program	Operating System kernel
สิทธิ์	จำกัด; privileged operation ทำไม่ได้	เข้าถึง/ควบคุม hardware ได้
การขอ service	system call	kernel handler ดำเนินงาน
เป้าหมาย	isolation และความปลอดภัย	ทำงานที่ต้องใช้ privilege

1.2.1 System Call flow

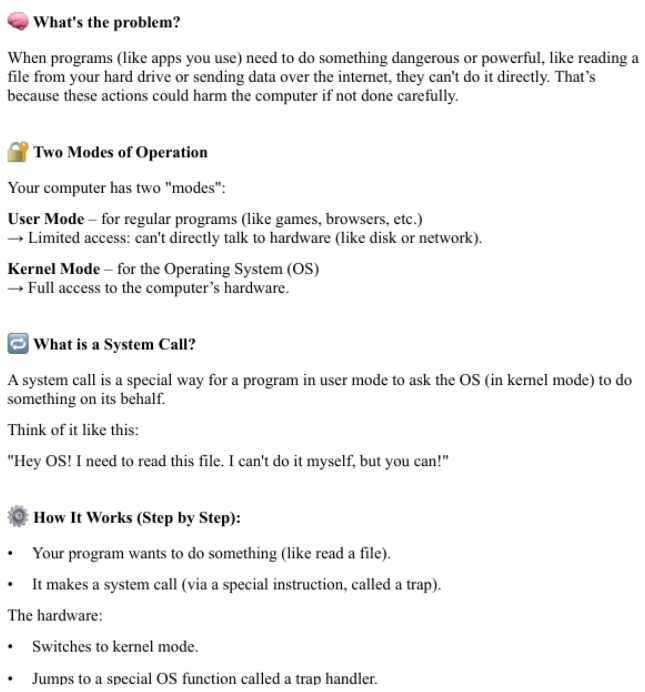
เมื่อ user program ต้องทำ operation ที่ถูกจำกัด ลำดับการทำงานเป็นดังนี้:

1. โปรแกรมเรียก API หรือ library function ที่เกี่ยวข้อง
2. library ใช้ special instruction หรือ convention เพื่อเรียก **trap**
3. hardware เปลี่ยนเป็น kernel mode และส่ง control ไปยัง **trap handler**
4. OS ทำ operation ที่ร้องขอด้วย privileged code
5. เมื่อทำงานเสร็จ OS ใช้ return mechanism เพื่อกลับไปยัง user program และลด privilege เป็น user mode



จุดที่มักสับสน

ภาพประกอบ 1.2 Lecture 1 หน้า 2 — User/Kernel Mode + System Call



What's the problem?

When programs (like apps you use) need to do something dangerous or powerful, like reading a file from your hard drive or sending data over the internet, they can't do it directly. That's because these actions could harm the computer if not done carefully.

Two Modes of Operation

Your computer has two "modes":

User Mode – for regular programs (like games, browsers, etc.)
→ Limited access: can't directly talk to hardware (like disk or network).

Kernel Mode – for the Operating System (OS)
→ Full access to the computer's hardware.

What is a System Call?

A system call is a special way for a program in user mode to ask the OS (in kernel mode) to do something on its behalf.

Think of it like this:

"Hey OS! I need to read this file. I can't do it myself, but you can!"

How It Works (Step by Step):

- Your program wants to do something (like read a file).
- It makes a system call (via a special instruction, called a trap).

The hardware:

- Switches to kernel mode.
- Jumps to a special OS function called a trap handler.

จุดสังเกต

- User Mode กับ Kernel Mode ต่างกันที่ระดับ privilege ไม่ใช่เพียงชนิดของโปรแกรม
- เมื่อต้องขอบริการ โปรแกรมเกิด trap เข้า kernel handler แล้ว return กลับสู่ user mode
- procedure call ปกติไม่ได้เปลี่ยน privilege boundary ด้วยตัวเอง

ที่มา: Lecture 1 หน้า 2 พร้อมคำอธิบายประกอบในสไลด์

Procedure Call ต่างจาก System Call procedure call เรียก function อื่นใน program/address space เดิมและไม่เปลี่ยน privilege level ด้วยตัวเอง ส่วน system call เปิดทางให้ user program ขอให้ kernel ทำ privileged operation

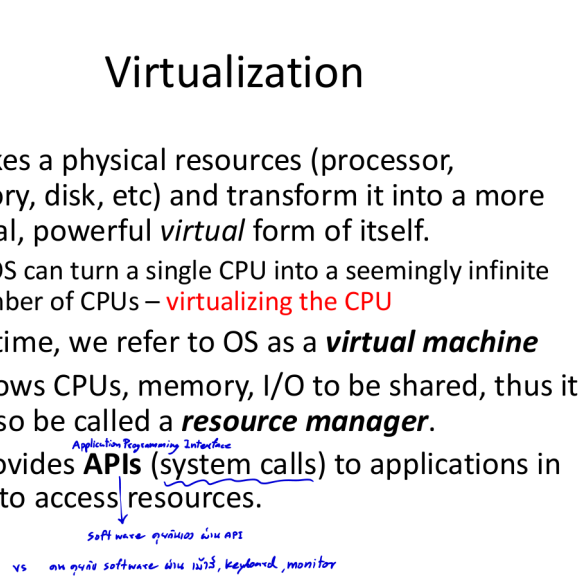
เชื่อมกับ Computer Architecture

เนื้อหา Computer Architecture เรื่อง I/O และ Processing Unit ช่วยอธิบายว่า OS ทำงานนี้ได้เพราะ hardware มีกลไกควบคุมสถานะของ processor ส่งต่อการทำงาน และจำกัด privileged operation ไม่ใช่เพราะซอฟต์แวร์สั่ง hardware ได้ทุกอย่างตามต้องการ ในบท Architecture ยังกล่าวถึง Privilege Exception และ return-from-interrupt ซึ่งเชื่อมโยงกับเส้นทาง trap/return ของ OS

1.3 Virtualization: ทำให้ physical resource ดูเหมือน resource ที่จัดการได้ง่ายขึ้น

Lecture นิยามภาพใหญ่ของวิชาผ่าน **Virtualization**

ภาพประกอบ 1.3 Lecture 1 หน้า 5 — Virtualization / Resource Manager



จุดสังเกต

- OS สร้าง abstraction จาก physical resources เพื่อให้โปรแกรมใช้งานได้สะดวกขึ้น
- OS ทำทั้ง **virtualization** และ **resource management**
- API และ System Call เป็น interface ที่ควบคุมได้ Virtualization จึงไม่ใช่เพียงการจำลองทรัพยากร

ที่มา: Lecture 1 หน้า 5.

OS รับ physical resource เช่น processor, memory, disk แล้วสร้าง abstraction ที่สะดวก ปลอดภัย และแชร์ได้

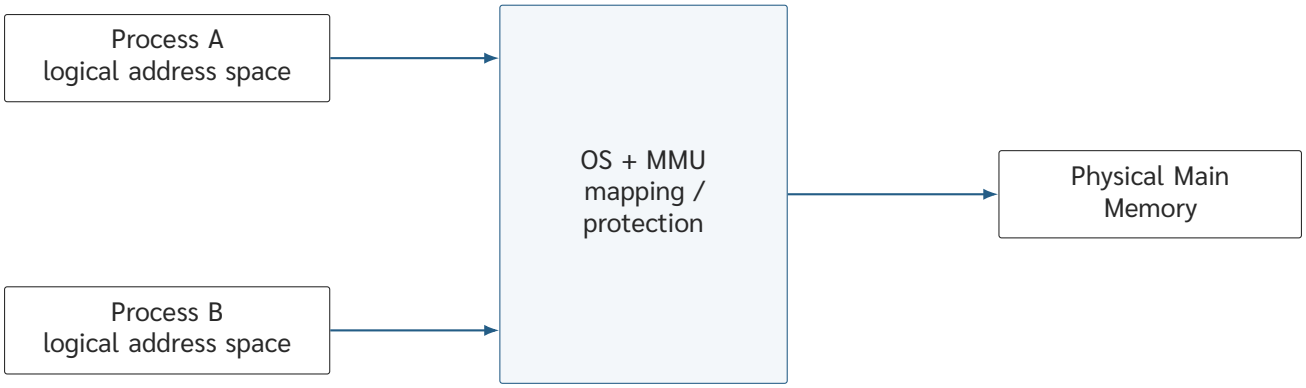
1.3.1 CPU Virtualization

หากมีหลายโปรแกรมต้องการทำงานบน processor core เดียว OS จะแบ่งเวลาและสลับการทำงานระหว่าง process เพื่อให้แต่ละ process ดูเสมือนว่ากำลังใช้ CPU อยู่ กลไกการสลับจะอธิบายใน Lecture 2 ส่วน policy ที่ใช้เลือก process ถัดไปอยู่ใน Lecture 3

> **อธิบายให้เห็นภาพ**
>
> แยกคำถามเรื่อง CPU virtualization ออกเป็นสองส่วน:
> - **Mechanism**: OS ดึง CPU กลับมาและสลับ process ได้อย่างไร?
> - **Policy**: เมื่อได้ CPU กลับมาแล้ว ควรเลือก process ใดให้ทำงานต่อ?
>
> การแยกสองคำนี้ตั้งแต่ Lecture 1 จะช่วยไม่ให้สับสนตอนเรียน Context Switch และ Scheduling

1.3.2 Memory Virtualization

ระหว่างที่โปรแกรมทำงาน ระบบจะอ่านและเขียนข้อมูลใน memory อยู่เสมอ OS จัดให้แต่ละ process มองเห็น address space ของตนเอง แม้ physical memory จะถูกใช้ร่วมกัน ใน Lecture 4–6 จะเริ่มจากความแตกต่างระหว่าง logical address กับ physical address แล้วต่อไปยัง Base/Bounds, Segmentation, Paging และ Virtual Memory



ภาพประกอบ 1.4 Process A/B ผ่าน MMU ไปยัง physical memory

ลูกศรสองเส้นแสดงว่า Process A และ B ส่ง logical address แยกกันเข้าสู่ MMU ระบบจะแปลง address และตรวจสิทธิ์ก่อนให้เข้าถึง physical memory ที่ใช้ร่วมกัน

1.4 OS as Resource Manager

นอกจากสร้าง abstraction แล้ว OS ยังเป็น **resource manager** โดยตัดสินใจว่า process ใดได้ใช้ CPU เมื่อใด จัด memory ให้ process อย่างไร จัดการ I/O ที่ต้องรอ และเก็บหรือเรียกคืนข้อมูล persistent ผ่าน API และ System Call ที่ OS ควบคุมได้

1.5 Concurrency: เมื่อหลายสิ่งดำเนินไปพร้อมกัน

Lecture ใช้ประโยค It's all about Concurrency เพื่อชี้ว่ากิจกรรมหลายอย่างเกิดซ้อนกันได้ ทั้ง process, I/O, interrupt, kernel work และ user work หากงานเหล่านี้เข้าถึง state ร่วมโดยไม่ประสานกัน ระบบอาจทำงานผิดพลาด
ใน Lecture 1 เรามอง Concurrency ในภาพรวมก่อน ประเด็นนี้ไม่ได้หมายถึงแค่การเปิดหลายโปรแกรมพร้อมกัน แต่รวมถึงลำดับที่งานสลับกันทำงานและการเข้าถึง shared state ซึ่ง OS ต้องจัดการให้ถูกต้อง

สไลด์หน้า 9 ยังชี้ว่า multithreaded program พบปัญหา concurrency ได้เช่นกัน แต่แนวทางจัดการต่างจาก concurrency ระดับ process เพราะ threads ภายใน process เดียวกันใช้ address space ร่วมกัน เรื่อง Thread API และ Locks/Semaphores เป็นหัวข้อที่แผนที่รายวิชาแยกไปเรียนภายหลัง

1.6 Persistence: ข้อมูลต้องอยู่รอดแม้ memory จะหาย

DRAM เป็น volatile storage เมื่อไฟดับ ข้อมูลใน main memory จะสูญหาย OS จึงจัดการข้อมูลบน non-volatile storage ผ่าน abstraction อย่าง **File System** เพื่อให้โปรแกรมสร้าง อ่าน เขียน และจัดระเบียบข้อมูลที่ต้องเก็บไว้ได้

Lecture ตั้งคำถามเพิ่มเติมว่าเหตุใด OS จึงไม่สร้าง private virtual disk แยกให้ทุก application ประเด็นที่ต้องจำคือ ส่วนนี้เน้นให้ OS จัดการ disk ร่วมกันผ่าน file system ไม่ได้ทำ virtualization แบบเดียวกับ CPU และ memory ในบทนี้

> **เชื่อมกับ Computer Architecture**
>
> มุมมองจาก Computer Architecture ช่วยแยกบทบาทและความเร็วของ memory hierarchy กับ I/O device โดย CPU และ main memory ทำงานเร็วกว่าการเข้าถึง disk มาก OS จึงต้องคำนึงถึง I/O latency เมื่อออกแบบ buffering, blocking, interrupt และ Virtual Memory/Page Fault

1.7 สิ่งที่ต้องคำนึงเมื่อออกแบบ OS

Lecture ระบุเป้าหมายการออกแบบหลายด้าน ซึ่งมักมี trade-off ต่อกัน:

ประเด็น	ความหมายในบริบท OS
Abstraction	สร้าง interface ให้ใช้งาน resource ที่ซับซ้อนได้ง่ายขึ้น
Performance	ลด overhead ที่เกิดจาก abstraction และการควบคุม
Isolation	ป้องกันไม่ให้ process หนึ่งอ่านหรือทำลาย state ของอีก process
Reliability	รักษาความถูกต้องแม้บาง component หรือ program มีปัญหา
Energy	ใช้ทรัพยากรและพลังงานอย่างเหมาะสม
Security	ควบคุมสิทธิ์และป้องกัน action ที่ไม่ได้รับอนุญาต
Mobility	แนวคิด OS ต้องรองรับบริบทอุปกรณ์หลากหลาย

1.8 Historical Motivation

สไลด์ท้าย Lecture สรุปพัฒนาการจาก **Early Mainframes** ไปสู่ **Multiprogramming on Minicomputer** และ OS บน PC สมัยใหม่ เมื่อคอมพิวเตอร์มีต้นทุนสูง การปล่อย CPU ว่างระหว่างรอ I/O ทำให้ใช้ทรัพยากรได้ไม่คุ้ม จึงเกิดแนวคิดให้หลายงานอยู่ในระบบและสลับใช้ทรัพยากร ส่งผลให้ต้องพัฒนา multiprogramming, protection และ OS ให้รองรับงานที่ซับซ้อนขึ้น

Lecture 1 หน้า 13–15 ให้ภาพพัฒนาการเพิ่มเติม: OS รุ่นแรกมีลักษณะเป็นชุด library สำหรับงาน I/O และมีคนควบคุมการรันทีละ program; เมื่อมีหลาย job อยู่ในระบบ การทำ **Multiprogramming** ช่วยใช้ CPU ระหว่างรอ I/O และนำมาสู่โจทย์ด้าน memory protection กับ interrupt. สไลด์เปรียบเทียบ DOS ซึ่งไม่มี memory protection, Mac OS รุ่นแรกที่จัด scheduling ได้ไม่ดี, Windows NT ที่นำแนวคิดจาก minicomputer OS มาใช้ และ Mac OS X ที่มี UNIX เป็นแกน พร้อมชี้การเปลี่ยนจากเครื่องที่แชร์กันทั้ง workgroup มาเป็นเครื่องส่วนบุคคล

สไลด์หน้า 4 เป็นแผนที่หัวข้อทั้งวิชา โดยยังมีเรื่อง Threads, Locks, Semaphores, Deadlock และ Mass Storage ที่เรียนต่อภายหลัง เอกสารนี้ยึดขอบเขตสอบกลางภาค Lecture 1–6 จึงระบุหัวข้อเหล่านั้นไว้เป็นแนวทาง แต่ไม่ขยายเป็นเนื้อหาบทเรียน

ใน course map ยังมี I/O System, File System และ Mini Projects ได้แก่ compile/build OS, ศึกษาและนำเสนอองค์ประกอบของ OS และแก้ไข kernel modules ด้วย หัวข้อเหล่านี้เป็นภาพรวมของรายวิชา ไม่ใช่เนื้อหาที่ Lecture 1–6 อธิบายเป็นบทเต็ม

> **จุดที่มักสับสน**
>
> ประวัติใน Lecture มีไว้ช่วยอธิบายว่าทำไมจึงต้องมี OS มากกว่าท่องปีหรือชื่อระบบ เมื่อเกิด multiprogramming ระบบก็ต้องมี isolation, scheduling และ memory management ที่รัดกุมขึ้น

1.9 Chapter Summary

> **สรุปท้ายบท**
>
> - OS เป็น software layer ที่ทำให้ application ใช้ hardware ผ่าน abstraction และ controlled interfaces
> - สามแกนใหญ่ของคอร์สคือ **Virtualization**, **Concurrency**, และ **Persistence**
> - CPU และ memory ถูก virtualize เพื่อให้หลาย process แชร์ physical resources ได้โดยมี isolation
> - System Call เป็น controlled entry จาก User Mode ไป Kernel Mode ผ่าน trap mechanism
> - OS เป็นทั้ง abstraction provider และ resource manager
> - design ต้องสมดุล convenience, performance, isolation, reliability, energy, security และ mobility

Key Mechanisms

User/Kernel Mode, Trap, Trap Handler, Return path, privileged execution.

Key Policies

Lecture 1 ยังไม่ลงรายละเอียด scheduling algorithm บทนี้วางกรอบไว้ว่า scheduler และ memory manager จะใช้ policy ต่าง ๆ บนกลไกที่ hardware รองรับ

ComArch Connections

Processor control state, privileged operations, I/O devices, interrupt/trap-style control transfer, physical memory.

Common Confusions

- OS ต่างจาก application library อย่างเดียว; kernel มี privilege และควบคุม resources
- System Call ต่างจาก ordinary procedure call
- Virtualization ต่างจาก simulation เสมอไป; ในคอร์สนี้คือการสร้าง abstraction/illusion เหนือ physical resource
- Concurrency ต่างจาก จำเป็นต้องมีหลาย CPU cores; interleaving บน CPU เดียวก็สร้าง concurrency problem ได้

Quick Revision Checklist

- ☐ อธิบาย Hardware และ OS และ Application ได้
- ☐ อธิบาย System Call ตั้งแต่ user code ผ่าน trap เข้าสู่ kernel จน return กลับได้
- ☐ แยก CPU virtualization กับ memory virtualization ได้
- ☐ อธิบายความหมายของ concurrency และ persistence ในภาพรวมได้
- ☐ บอกเหตุผลว่าทำไม User Mode/Kernel Mode ช่วย protection ได้

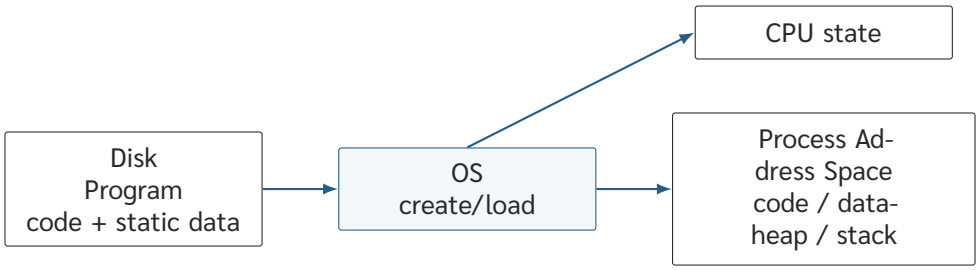
> **จุดเน้นก่อนสอบ**
>
> หลังจบ Lecture 1 ควรอธิบายความเชื่อมโยงได้ว่า **เมื่อโปรแกรมต้องการใช้ resource, OS จะให้บริการผ่าน abstraction และ API ส่วนงานที่ต้องใช้สิทธิ์สูงจะผ่าน kernel โดยอาศัย hardware mode และ control transfer เพื่อรักษา protection** แนวคิดนี้จะกลับมาอีกใน Process, Scheduling และ Virtual Memory

Lecture 2 — Process and Process APIs

อ้างอิง: Lecture 2 หน้า 1–44 | process, สถานะและ PCB, context switch, timer interrupt และ process APIs

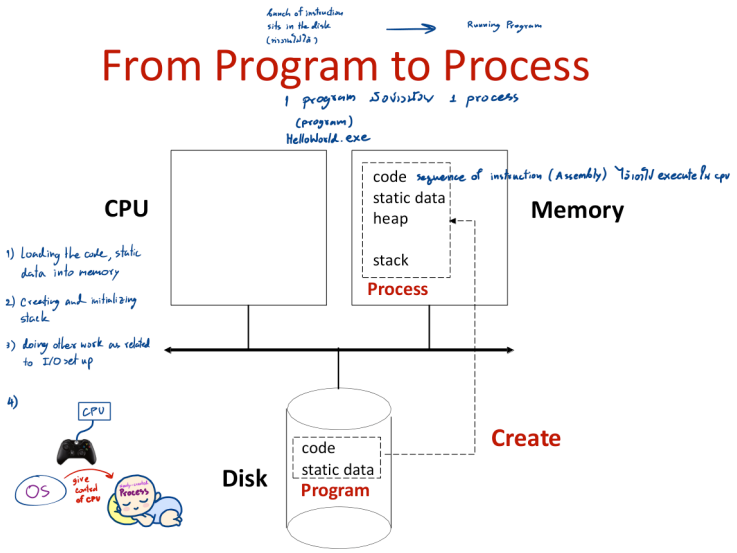
2.1 From Program to Process

> **เนื้อหาจาก Lecture**
>
> **Program** คือชุด instruction/code และ static data ที่เก็บอยู่บน disk ส่วน **Process** คือ program ที่กำลังถูก execute พร้อม execution state ของมัน การเปลี่ยนจาก program ไปเป็น process อย่างน้อยต้องมีการโหลด code/static data เข้า memory, สร้างและ initialize stack, จัดเตรียม state ที่เกี่ยวข้องกับ I/O/ runtime และทำให้ CPU สามารถเริ่ม execute ได้



ภาพประกอบจากสไลด์หน้า 3 ช่วยแยกความหมายของ program กับ process ให้ชัดเจน

ภาพประกอบ 2.1 Lecture 2 หน้า 3 — From Program to Process



จุดสังเกต

- ซ้ายล่าง: program อยู่บน disk เป็น code/static data ที่ยังไม่ execute
- OS สร้าง process โดย load code/data, initialize stack และ setup execution/I/O state
- กรอบ memory แสดง process image ที่มี code, static data, heap, stack

ที่มา: Lecture 2 หน้า 3 พร้อมหมายเหตุเรื่องการโหลดและ process

Program คือชุดคำสั่งที่จัดเก็บอยู่บน storage เมื่อ OS สร้าง process และเตรียม state ที่จำเป็นแล้ว CPU จึงเริ่มนำคำสั่งนั้นมาทำงานได้

2.2 Process = (P, D, C, S)

Lecture กำหนด process เป็น entity ที่มีสี่องค์ประกอบ:

Symbol	Component	ความหมายตาม Lecture
P	Program / code	code หรือชิ้นส่วน program ที่ process ผูกอยู่
D	Processing data	ชุดตัวแปรข้อมูลของ process
C	Control state	control variables ที่บอกว่าจะ execute ที่ไหน/อย่างไรต่อ
S	Process state	สถานะ เช่น Ready, Running, Blocked/Suspended

> **อธิบายให้เห็นภาพ**
> ถ้ามีเพียง code อย่างเดียว เรายัง resume process ไม่ได้ เพราะ CPU ต้องรู้ตำแหน่ง instruction ปัจจุบัน, register values, stack และสถานะการรอ resource ด้วย ดังนั้น process เป็นทั้งโปรแกรม **และ state ที่ทำให้การ execute ต่อเนื่องได้**

2.3 Process Address Space และ Process Descriptor

Lecture แบ่ง memory ที่เกี่ยวข้องกับ process เป็น:

1. **Code/Text Segment** — instructions
2. **Data Segment / Heap** — dynamic data เช่นสิ่งที่ได้จาก `malloc()`
3. **Stack** — parameters, local variables, return address และ state ของ function calls
4. **Process Descriptor / Process Control Block (PCB)** — เก็บใน kernel space และเข้าถึงได้ใน kernel mode

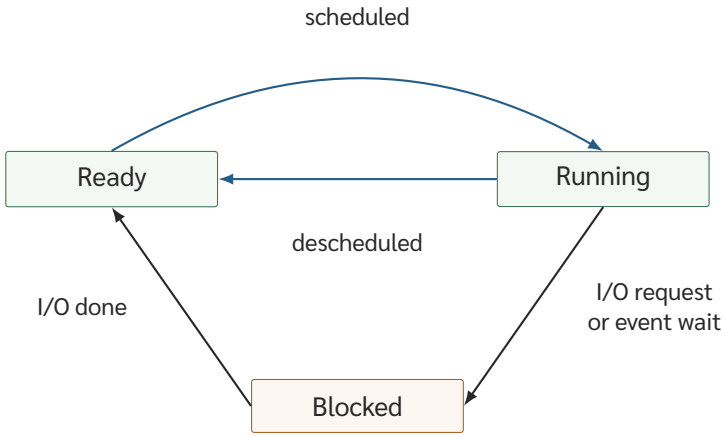


2.3.1 PCB เก็บอะไร

Lecture ระบุว่า PCB เป็น OS data structure สำหรับข้อมูลที่สัมพันธ์กับแต่ละ process เช่น identifier, state, CPU context/register state, scheduling/accounting information และข้อมูล resource ที่จำเป็นต่อการจัดการ process

> **เชื่อมกับ Computer Architecture**
> PCB สำคัญเพราะ CPU มี state ที่เปลี่ยนตลอดเวลา ได้แก่ **Program Counter (PC)**, general-purpose registers, stack pointer และ processor control state เนื้อหา Computer Architecture Lecture 7 แสดงให้เห็นว่า instruction execution อาศัย state เหล่านี้ เมื่อ OS หยุด Process A ไว้ชั่วคราวและจะกลับมาทำงานต่อ จึงต้องบันทึกค่า state ไว้ใน PCB

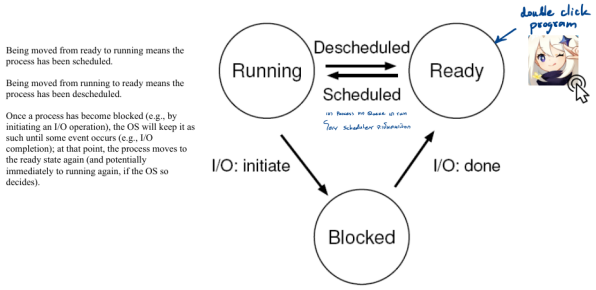
2.4 Process State: Ready, Running, Blocked



- **Ready**: พร้อมรัน แต่ยังไม่ได้ CPU
- **Running**: กำลัง execute บน processor
- **Blocked**: ทำ operation บางอย่างแล้วไม่พร้อมรัน เช่นกำลังรอ I/O completion

หมายเหตุในสไลด์หน้า 6 กำกับ transition ไว้ว่า เมื่อเลือก process จาก Ready ให้ใช้ CPU จะเรียกว่า Scheduled และเมื่อถอด process ออกจาก CPU จะเรียกว่า Descheduled

ภาพประกอบ 2.2 Lecture 2 หน้า 6 — Ready / Running / Blocked



จุดสังเกต

- **Ready**: พร้อมใช้ CPU แต่ยังไม่ถูกเลือก
- **Running**: กำลัง execute instruction
- **Blocked**: ยังรันต่อไม่ได้จน event/I/O เสร็จ
- Scheduled/Descheduled เป็น transition ระหว่าง Ready↔Running; I/O initiate/done เชื่อม Running↔Blocked↔Ready

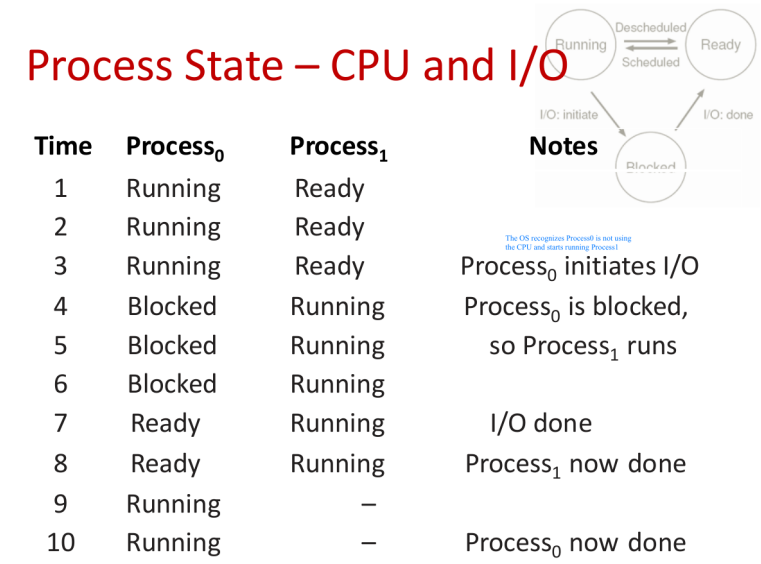
ที่มา: Lecture 2 หน้า 6 พร้อมหมายเหตุเรื่องสถานะ process

2.4.1 CPU-only vs CPU + I/O timeline

สไลด์หน้า 7–8 แสดงว่า process ที่ใช้ CPU อย่างเดียวสามารถอยู่ Running ต่อเนื่องจนถูกสลับ ส่วน process ที่ออก I/O จะเปลี่ยนเป็น Blocked ทำให้ OS มีโอกาสให้ process อื่นใช้ CPU ระหว่างรอ

> **จุดที่มักสับสน**
>
> **Ready ต่างจาก Blocked.** ทั้งคู่ ไม่ได้ใช้ CPU ตอนนี้ เหมือนกัน แต่ Ready สามารถถูก scheduler เลือกได้ทันที ส่วน Blocked ยังมีเงื่อนไขที่ต้องเสร็จก่อน (เช่น I/O done)

ภาพประกอบ 2.3 Lecture 2 หน้า 8 — CPU + I/O Process Timeline



Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	$Process_0$ initiates I/O
4	Blocked	Running	$Process_0$ is blocked,
5	Blocked	Running	so $Process_1$ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	$Process_1$ now done
9	Running	–	
10	Running	–	$Process_0$ now done

จุดสังเกต

- เมื่อ Process 0 initiate I/O จะออกจาก Running ไป Blocked
- CPU จึงไม่จำเป็นต้อง idle: scheduler เลือก Process 1 จาก Ready มารันได้
- เมื่อ I/O done, Process 0 กลับ Ready ไม่ได้แปลว่าต้อง Running ทันที

ที่มา: Lecture 2 หน้า 8 ใช้เชื่อม state transition กับการใช้ CPU

2.5 Typical Process APIs และ OS Data Structures

Lecture ยก operation หลัก ๆ ที่ OS ต้องรองรับ: create, destroy/terminate, wait, miscellaneous control, status query และ communication. API เหล่านี้ทำให้ user process ขอให้ OS จัดการ lifecycle ของ process โดยไม่ต้องแก้ kernel structures เอง

OS ต้องมี process lists/queues และ PCB เพื่อ track ว่า process ใด Ready, Running, Blocked และมี resource/state อะไรบ้าง

2.6 CPU Virtualization Challenge: Performance + Control

จุดเปลี่ยนของ Lecture คือคำถามว่า OS จะให้ process รัน เกือบตรงบน CPU เพื่อ performance แต่ยังรักษา control ได้อย่างไร

> **เนื้อหาจาก Lecture**
>
> Limited Direct Execution (LDE) แก้โจทย์โดยประกอบด้วยสามส่วนตามสไลด์:
> 1. Restricted Operation
> 2. Switch Between Processes / Context Switch
> 3. Timer Interrupt

2.6.1 1) Restricted Operation — User Mode / Kernel Mode

Kernel mode สามารถทำ privileged operations ได้ ส่วน user mode ถูกจำกัด ตัวอย่างใน Lecture: user process ไม่สามารถขอ I/O โดยตรง; หากต้องเข้าถึง disk ต้องผ่าน system call

ขั้นตอนสำคัญ:

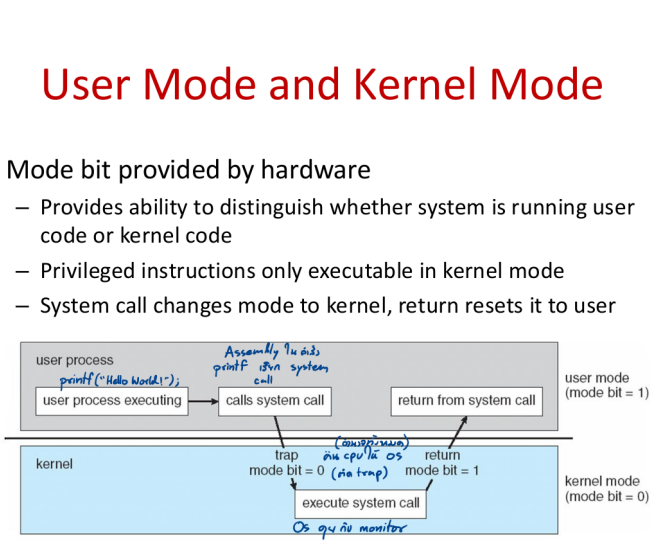
1. user program execute special instruction **trap**
2. privilege ถูกยกไป kernel mode
3. kernel ทำ privileged operation
4. kernel ใช้ **return-from-trap** เพื่อกลับ user code และลด privilege

2.6.2 Trap Table

OS บอก hardware ว่าเมื่อ exceptional event แต่ละชนิดเกิดขึ้นควรวิ่งไป code ใดผ่าน **trap table** ซึ่งถูกตั้งค่าตอน boot. Lecture ยก hard-disk interrupt เป็นตัวอย่างหนึ่งของ event ที่ต้องมี handler address ที่เชื่อถือได้

> **จุดที่มักสับสน**
>
> Trap table ต้องถูกตั้งโดย kernel/privileged code. ถ้า user process เปลี่ยน trap table เองได้ มันจะเปลี่ยน control flow ของ privileged event และทำลาย protection model ได้

ภาพประกอบ 2.4 Lecture 2 หน้า 19 — Hardware Mode Bit



จุดสังเกต

- Hardware mode bit เป็นส่วนของ mechanism ที่แยก privileged execution ออกจาก user execution
- trap ทำให้ control เข้า kernel ผ่าน entry ที่ตั้งไว้; return กลับ user พร้อมลด privilege
- นี่คือฐานของ **Limited Direct Execution**: รัน user code ตรงบน CPU แต่จำกัดสิ่งอันตราย

ที่มา: Lecture 2 หน้า 19.

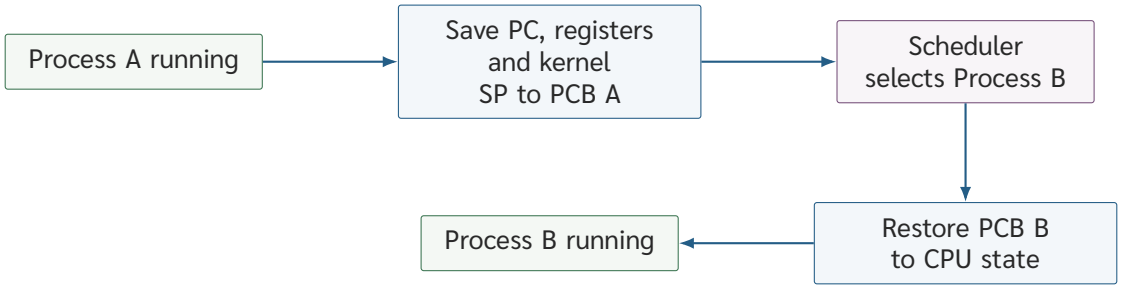
2.7 2) Switch Between Processes — Scheduler vs Context Switch

Lecture แยกสองชั้น:

- **Scheduler**: ตัดสินใจว่า **ควร switch ไป process ไหน**
- **Context Switch**: low-level code ที่ save/restore state เพื่อทำ switch จริง

> **เนื้อหาจาก Lecture**
>
> **Context Switch** คือขั้นตอน storing และ restoring state ของ process เพื่อให้ภายหลังกลับมา execute ต่อจากจุดเดิมได้

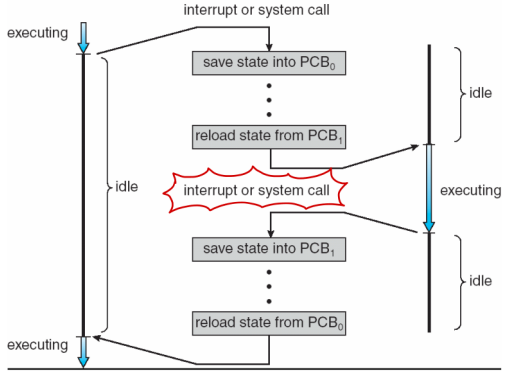
ตาม Lecture OS save general-purpose registers, PC และ kernel stack pointer ลง state/PCB ของ process ปัจจุบันแล้ว restore context ของ process ถัดไปจาก Ready queue; เมื่อ execute return-from-trap, process ใหม่จึงกลับเข้าการ execute ที่ state ของมัน



> **เชื่อมกับ Computer Architecture**
>
> Architecture Lecture 7 แสดงว่า PC บอก instruction sequence, register input/output ถูกควบคุมด้วย control signals และ memory access ผ่าน MAR/MDR. Context switch จึงไม่ใช่แค่ เปลี่ยนชื่อ process แต่เป็นการเปลี่ยน processor-visible state. หาก restore PC/register ผิดแม้ค่าเดียว process จะ resume ผิดจุดหรือใช้ข้อมูลของอีก process

Lecture ให้ intuition เรื่อง cost ว่า context switch อยู่ระดับ sub-microsecond บนระบบ 2–3 GHz ตามค่าที่สไลด์ยกมา; ประเด็นสำคัญคือ switch มี overhead จึงไม่ควรถือว่าฟรี

ภาพประกอบ 2.5 Lecture 2 หน้า 23 — Saving / Restoring Context



จุดสังเกต

- Context ที่ต้องรักษาเกี่ยวกับ PC, registers และ kernel stack/state ที่ทำให้ process resume ถูกจุด
- ฝั่งออกจาก A = save; ฝั่งเข้า B = restore
- Scheduler เลือก process; Context Switch ทำการเปลี่ยน CPU state — สองคำนี้ไม่ใช่สิ่งเดียวกัน

ที่มา: Lecture 2 หน้า 23 เชื่อมกับ CPU register state เพื่ออธิบายเหตุผลที่ต้อง save และ restore

2.8 3) OS จะเอา CPU กลับมาได้อย่างไร

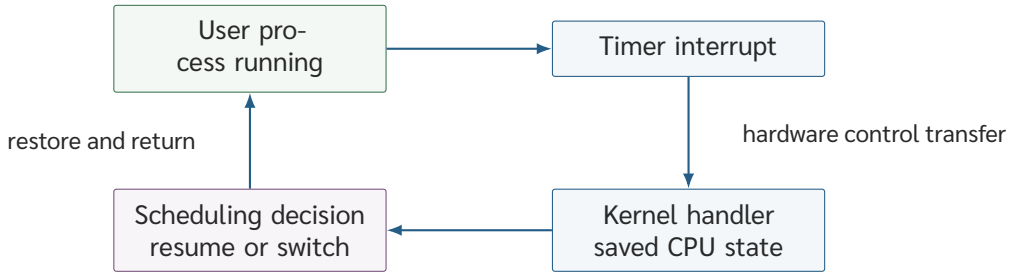
2.8.1 การคืน CPU แบบ Cooperative

วิธีนี้อาศัยให้ process คืน CPU ผ่าน system call, การเรียก `yield()` หรือเหตุการณ์ที่ทำให้ trap เข้า OS ข้อจำกัดคือหาก process ติดอยู่ใน infinite loop และไม่เรียก yield หรือ system call OS อาจดึง CPU กลับมาไม่ได้ Lecture จึงยกกรณีร้ายแรงที่ต้อง reboot ระบบ

2.8.2 การดึง CPU กลับโดยไม่พึ่ง Process: Timer Interrupt

OS ตั้ง timer device ให้ส่ง interrupt เป็นระยะ เมื่อเกิด interrupt จะมีลำดับดังนี้:

1. process ที่กำลังรันถูกหยุดชั่วคราว
2. hardware บันทึก state ที่จำเป็นต่อการกลับมาทำงานต่อ
3. interrupt handler ที่ OS เตรียมไว้ได้รับ control
4. OS ตัดสินใจว่าจะให้ process เดิมทำงานต่อหรือสลับไป process อื่น

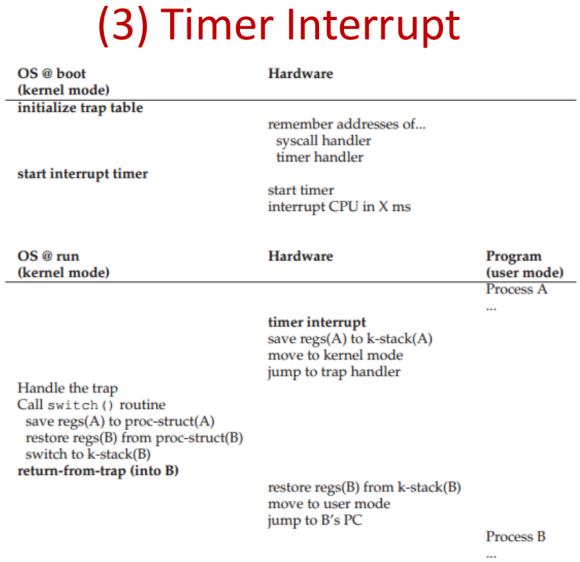


> **เชื่อมกับ Computer Architecture**
>
> Computer Architecture Lecture 5 อธิบาย hardware interrupt, IRQ, ISR, การเปิด/ปิด interrupt และ return-from-interrupt นอกจากนี้ยังชี้ว่า interrupt latency รวมเวลาที่ใช้บันทึกและคืน state กลไก **timer interrupt** จึงทำให้ kernel ดึง control กลับมาได้โดยไม่ต้องรอให้ user process ยินยอม

> **จุดเน้นก่อนสอบ**
>
> หัวใจของ LDE ตามสไลด์หน้า 30 คือ **Restricted Operation, Context Switch และ Timer Interrupt** ควรแยกให้ได้ว่าแต่ละกลไกทำหน้าที่ป้องกันการเข้าถึง สลับ state และดึง control กลับมาอย่างไร

2.9 UNIX Process APIs

ภาพประกอบ 2.6 Lecture 2 หน้า 29 — Timer Interrupt



จุดสังเกต

- Timer interrupt ทำให้ OS ดึง CPU กลับมาได้โดยไม่ต้องรอให้ process yield
- เมื่อเข้า interrupt handler แล้ว kernel จะตัดสินใจว่าจะ resume process เดิมหรือสลับ process
- ภาพนี้เชื่อม Mechanism (timer/context switch) กับ Policy (scheduler)

ที่มา: Lecture 2 หน้า 29 พร้อมลำดับที่เขียนกำกับไว้ในสไลด์

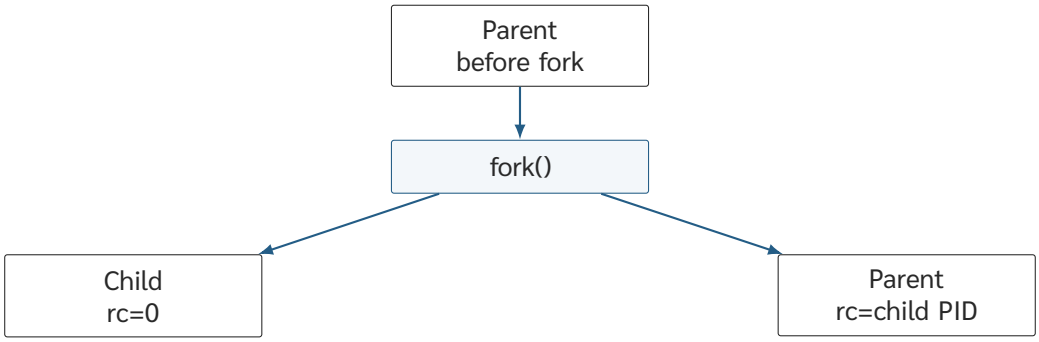
2.9.1 fork(): สร้าง process ใหม่

Lecture ใช้ code ต่อไปนี้เพื่อแสดง parent/child path:

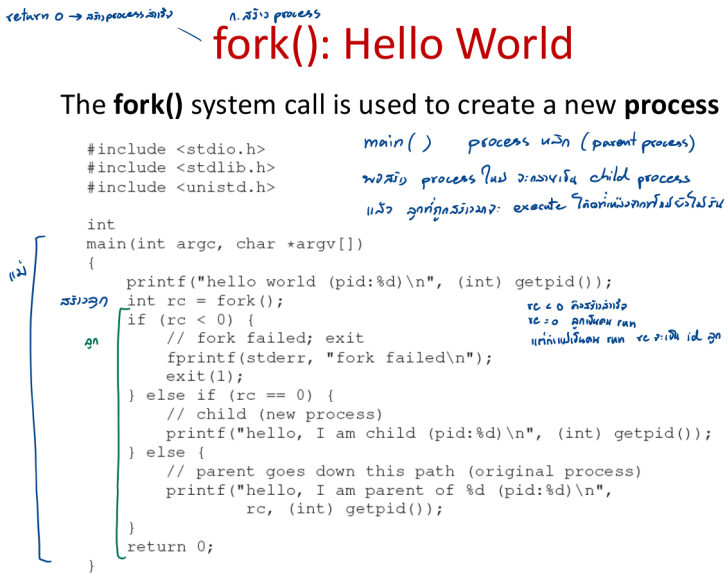
```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {
        // parent goes down this path (original process)
        printf("hello, I am parent of %d (pid:%d)\n",
               rc, (int) getpid());
    }
    return 0;
}
```

- child เป็น process ใหม่และเริ่มต่อจากจุดหลัง `fork()` ไม่ได้เริ่มใหม่ที่ `main()`
- child เป็น almost exact copy แต่มี address space, registers, PC ฯลฯ ของตนเอง
- ใน child: return value ของ `fork()` = 0
- ใน parent: return value คือ PID ของ child; กรณี error < 0
- บน single CPU ไม่มีการรับประกันว่า parent หรือ child จะได้ทำงานก่อน ลำดับ output จึงเปลี่ยนได้



ภาพประกอบ 2.7 Lecture 2 หน้า 34 — fork() Example + Annotation



จุดสังเกต

- fork สำเร็จแล้ว execution แตกเป็น parent และ child จาก instruction หลัง fork
- child เห็น return value = 0; parent เห็น child PID
- output ordering ยังไม่ deterministic ถ้าไม่มี wait/synchronization

ที่มา: Lecture 2 หน้า 34 พร้อมหมายเหตุเรื่อง return value ของ parent และ child

2.9.2 wait() / waitpid(): จัดลำดับ parent–child

`wait()` ทำให้ parent หยุดรอจนกว่า child ตัวหนึ่งจะ terminate ส่วน `waitpid()` ใช้รอ child ที่ระบุ ในตัวอย่างของ Lecture การเพิ่ม wait ทำให้ child แสดงผลก่อน parent อย่างแน่นอน ทั้งสองฟังก์ชันคืน PID และรับ exit status ผ่าน status argument

```
// parent path
int wc = wait(NULL);
printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
       rc, wc, (int) getpid());
```

2.9.3 exec(): เปลี่ยน program image ที่ process กำลังรัน

Lecture ระบุ family ของ `exec()` เช่น `execvp()`. ใช้เมื่อต้องการให้ child รัน program ที่ต่างจาก parent:

```
char *myargs[3];
myargs[0] = strdup("word_count");
myargs[1] = strdup("p.txt");
myargs[2] = NULL;
execvp(myargs[0], myargs);
printf("this shouldn't print out");
```

หาก `exec()` สำเร็จ มันไม่ return มายัง instruction ถัดไป เพราะ process image ถูกแทนด้วย executable ใหม่

> **จุดที่มักสับสน**
>
> **fork() ต่างจาก exec()** fork สร้าง process ใหม่ที่ทำงานต่อจากจุดเดิม ส่วน exec เปลี่ยน program image ของ process ที่มีอยู่ Unix shell แยกสอง operation นี้เพื่อให้ตั้งค่า environment หรือ redirection ได้ระหว่าง fork กับ exec

2.9.4 Termination, signals, command-line tools

Process จบการทำงานได้ด้วย `exit(status)` หรือจาก fatal error/signal จากนั้น OS คืน resources และ parent สามารถอ่าน exit status ผ่าน wait ได้ Lecture ยังกล่าวถึง `kill()` และ signal subsystem สำหรับควบคุม process รวมถึง `ps` และ `top` สำหรับตรวจดูสถานะ

2.9.5 Why fork() + exec() ช่วยสร้าง shell

ลำดับการทำงานตาม Lecture:

1. shell ค้นหา executable ที่ตรงกับ command
2. เรียก `fork()` เพื่อสร้าง child
3. ตั้งค่า environment หรือ redirect I/O ก่อนเรียก exec
4. child เรียกฟังก์ชันในกลุ่ม `exec()`
5. shell หรือ parent เรียก `wait()`
6. เมื่อ child ทำงานเสร็จ shell กลับมาแสดง prompt

ตัวอย่าง `wc p.txt > newfile.txt`: shell ปิด standard output แล้วเปิด `newfile.txt` ก่อนเรียก exec ทำให้ program ที่ถูก exec เขียนผลลงไฟล์ได้ โดยไม่ต้องรู้รายละเอียดการทำงานของ shell

ภาพประกอบ 2.8 Lecture 2 หน้า 42 — Why fork() and exec()?

Why fork() and exec()? (1)

- The separation of **fork()** and **exec()** is essential in building a Unix shell (command-line-interpreter) because it lets the shell run code after the call to **fork()** but before the call to **exec()**
- With the name of an executable program, plus any arguments passed to the Unix shell, it:
  - figures out where the executable is
  - calls **fork()** to create a new child process
  - alters the environment of the about-to-be-run program, if necessary
  - calls some variant of **exec()** to run the command
  - waits for the command to complete by calling **wait()**
  - when the child completes, the shell returns from the **wait()** and prints out a prompt again, ready for the next command

จุดสังเกต

- จุดสำคัญคือ **ช่องว่างระหว่าง fork กับ exec**: shell ยังมีโอกาส setup environment / file descriptors ก่อนเปลี่ยน program image
- ลำดับบนสไลด์เริ่มจากค้นหา executable จากนั้น fork และตั้งค่า environment ก่อน exec ส่วน parent จะ wait แล้วจึงกลับสู่ prompt
- นี่อธิบายว่าทำไม Unix แยก สร้าง process ออกจาก เปลี่ยน program ที่ process รัน

ที่มา: Lecture 2 หน้า 42.

2.9.6 Pipes

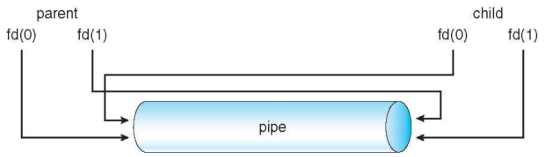
Lecture กำหนด pipe แบบ producer/consumer:

- producer เขียน write-end
- consumer อ่าน read-end
- `int pipe(int fd[2])`: `fd[0]` สำหรับ read, `fd[1]` สำหรับ write
- ถ้า parent รับข้อมูลจาก child: parent ปิด `fd[1]` แล้วอ่าน `fd[0]`; child ปิด `fd[0]` แล้วเขียน `fd[1]`

ภาพประกอบ 2.9 Lecture 2 หน้า 44 — Pipes

Pipes

- Pipes allow communication in a producer/consumer style
  - Producer writes to one end (the write-end of the pipe)
  - Consumer reads from the other end (the read-end of the pipe)
  - If the parent wants to receive data from the child, it should close *fd[1]* and read from *fd[0]*, and the child should close *fd[0]* and write to *fd[1]*



  - **int pipe(int** *fd[2]***);** *fd[0]* is set up for reading, *fd[1]* for writing

จุดสังเกต

- `fd[0]` คือ read end; `fd[1]` คือ write end
- หาก parent รับข้อมูลจาก child ให้แต่ละฝั่งปิด end ที่ไม่ได้ใช้ เพื่อให้ dataflow เป็นไปตามทิศทางที่ต้องการ
- มอง pipe เป็นช่องทาง producer/consumer ที่ kernel จัดการ ไม่ใช่ shared variable ทั่วไป

ที่มา: Lecture 2 หน้า 44 ทิศทางของ endpoint แสดงไว้ในภาพเพื่อให้อ่านตามได้ง่าย

2.10 Chapter Summary

สรุปท้ายบท

- Process คือ program ที่กำลังทำงานพร้อม data และ state ที่จำเป็น PCB เก็บข้อมูลฝั่ง kernel ที่ใช้จัดการและ resume process
- Process มี state หลักคือ Ready, Running และ Blocked เมื่อเริ่ม I/O จะเปลี่ยนจาก Running เป็น Blocked และกลับเป็น Ready เมื่อ I/O เสร็จ
- CPU virtualization ใช้ Limited Direct Execution: Restricted Operation + Context Switch + Timer Interrupt

- Scheduler เป็น policy; Context Switch เป็น mechanism
- Timer Interrupt แก้ปัญหา process ไม่ยอมคืน CPU ด้วยตัวเอง
- UNIX model แยก process creation (fork), program replacement (exec), synchronization (wait), termination/signals และ pipe communication

Key Mechanisms

User/Kernel mode, trap table, trap/return-from-trap, PCB save/restore, timer interrupt, context switch.

Key Policies

การเลือก process ถัดไปเป็นหน้าที่ของ scheduler policy ซึ่งจะเรียนต่อใน Lecture 3

ComArch Connections

PC/register/kernel stack pointer, interrupt handler, privilege, processor state, controlled return path.

Common Confusions

- Program ต่างจาก Process
- Ready ต่างจาก Blocked
- Scheduler decision ต่างจาก Context Switch implementation
- System Call trap ต่างจาก Timer Interrupt แม้ทั้งคู่พา control เข้า kernel
- Context Switch ต่างจาก Process Swapping ไป disk (จะพบใน Lecture 6)
- fork ต่างจาก exec

Quick Revision Checklist

☐ วาด Ready–Running–Blocked transition ได้
☐ อธิบาย PCB ว่าทำไมต้องมีและเกี่ยวกับ CPU state อย่างไร
☐ อธิบาย LDE ตั้งแต่ user execution ผ่าน trap หรือ timer interrupt เข้าสู่ kernel จน context switch และ return ได้
☐ อธิบาย cooperative vs non-cooperative CPU control ได้
☐ trace return values ของ fork และ output ordering ได้
☐ อธิบาย fork + exec + wait flow ของ shell ได้

Lecture 3 — Scheduling

อ้างอิง: Lecture 3 หน้า 1–40 | scheduling metrics, FIFO/SJF/STCF/RR, MLFQ และ lottery scheduling

3.1 Scheduling คือ Policy สำหรับแบ่งปันทรัพยากร

> **เนื้อหาจาก Lecture**
>
> Scheduling คือการตัดสินใจว่าจะจัดสรร CPU ให้ process ใด ใน Lecture 2 เราเรียน context switch ซึ่งเป็นกลไกที่ทำให้สลับ process ได้ ส่วน Lecture นี้เน้น policy ที่ใช้เลือก process ให้เหมาะกับเป้าหมายของระบบ

> **เชื่อมกับ Computer Architecture**
>
> Timer Interrupt + Context Switch ทำให้เกิด preemption ได้ แต่ไม่ได้เป็นตัวเลือกว่าจะให้ใครใช้ CPU หน้าที่นั้นเป็นของ Scheduler ซึ่งใช้กลไกดังกล่าวประกอบการตัดสินใจ จำให้ได้ว่า **Mechanism ต่างจาก Policy**

3.1.1 Workload และสมมติฐานของ Job

Lecture เรียกกลุ่ม process ที่ scheduler ต้องจัดการว่า workload หรือ job โดยเริ่มจากแบบจำลองอย่างง่าย: ทุก job ใช้ CPU อย่างเดียว ไม่มี I/O และรู้ run-time ล่วงหน้า จากนั้นจึงค่อยผ่อนสมมติฐานให้ใกล้ระบบจริง

3.2 Scheduling Metrics

เราใช้ metrics วัด performance และ fairness ของ policy บางครั้งการเพิ่ม performance อาจทำให้การแบ่ง CPU ไม่เป็นธรรม จึงต้องพิจารณา trade-off ทั้งสองด้าน

3.2.1 Turnaround Time

$$T_{turnaround} = T_{completion} - T_{arrival} \tag{1}$$

ถ้าทุก job มาถึงพร้อมกันที่เวลา 0: $T_{turnaround} = T_{completion}$.

3.2.2 Response Time

$$T_{response} = T_{first\text{-}run} - T_{arrival} \tag{2}$$

Response Time วัดว่า user/job ต้องรอนานเท่าใดก่อนเริ่มได้รับ CPU ครั้งแรก ไม่ใช่เวลาจนงานเสร็จ

> **จุดที่มักสับสน**
>
> **Response Time ต่างจาก Turnaround Time** policy ที่ดีต่อ metric หนึ่งอาจไม่ดีต่ออีก metric หนึ่ง เช่น SJF/STCF มุ่งลด turnaround ส่วน RR ช่วยให้แต่ละ process เริ่มตอบสนองได้เร็วขึ้น

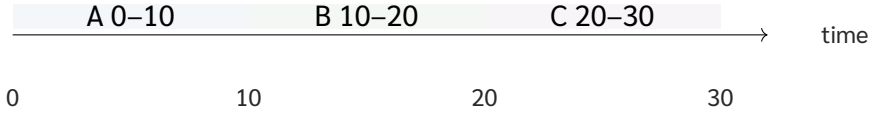
> **รายละเอียดสำคัญจากสไลด์**
>
> **Lecture 3 หน้า 23 — Batch กับ Interactive** งานแบบ batch มักวัดผลจากเวลาที่ทำเสร็จ ส่วนงาน interactive ต้องตอบสนองผู้ใช้ได้เร็ว จึงให้ความสำคัญกับ response time ภาพในสไลด์เชื่อมจาก SJF/STCF/RR ไปสู่ MLFQ ซึ่งพยายามรองรับงานทั้งสองแบบโดยไม่ต้องรู้ job length ล่วงหน้า

3.3 FIFO / FCFS

First In, First Out (FIFO) หรือ First Come, First Served เลือก process ตามลำดับที่มาถึง

ตัวอย่างคำนวณ

ตัวอย่างใน Lecture: A, B และ C มาถึงพร้อมกันที่ $t = 0$ แต่ละ job ใช้ CPU 10 s และเข้าคิวตามลำดับ A, B, C



$T_A = 10$, $T_B = 20$, $T_C = 30$ s.

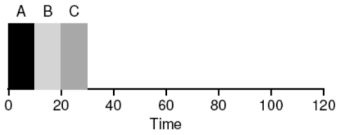
$$\overline{T}_{turnaround} = \frac{10 + 20 + 30}{3} = 20 \text{ s}$$

ภาพประกอบ 3.1 Lecture 3 หน้า 7 — FIFO / FCFS Timeline

First In, First Out (FIFO)

(First Come, First Served)

- Imagine three jobs arrive in the system, A, B, and C, at the same time ($T_{arrival}$ = 0)
- FIFO has to put some job first, let's assume A -> B -> C
- Assume also that each job runs for 10 seconds
- What will the average turnaround time be for these jobs?



(10+20+30)/3 = 20 seconds

จุดสังเกต

- Job ถูกวางตาม arrival order; ไม่มีการเลือกจาก job length
- ใช้ timeline อ่าน completion time ของแต่ละ job แล้วคำนวณ turnaround
- ความง่ายของ FIFO ไม่ได้แปลว่า metric ดีทุก workload

ที่มา: Lecture 3 หน้า 7.

3.3.1 ข้อจำกัดของ FIFO: Convoy Effect

สมมติว่า A ใช้เวลา 100 s ส่วน B และ C ใช้เวลา 10 s และมาถึงพร้อมกัน หาก FIFO เรียงเป็น A, B, C งานสั้นทั้งสองต้องรออยู่หลังงานยาว:

$$\overline{T}_{turnaround} = \frac{100 + 110 + 120}{3} = 110 \text{ s}$$

ปัญหาคือ resource consumer สั้น ๆ ถูก queue อยู่หลัง long job.

3.4 Shortest Job First (SJF)

Shortest Job First (SJF) เลือก job ที่ใช้เวลาสั้นที่สุดก่อน ภายใต้สมมติฐานว่าทุก job มาถึงพร้อมกัน SJF ให้ average turnaround ต่ำที่สุดตาม Lecture

สำหรับ A=100, B=10, C=10 ถ้ารันตามลำดับ B, C, A:

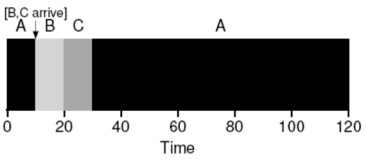
$$\overline{T}_{turnaround} = \frac{10 + 20 + 120}{3} = 50 \text{ s}$$

หมายเหตุในสไลด์หน้า 9 ชี้ว่าในตัวอย่างนี้ SJF ให้ average turnaround ต่ำกว่า FIFO

ภาพประกอบ 3.2 Lecture 3 หน้า 11 — STCF with Different Arrival Times

STCF's example

- A arrives at t = 0 and needs 100 secs, B and C arrive at t = 10, each need 10 secs



- Ave turnaround time (120+10+20)/3 = 50
- In batch scheduling, this make sense
- However, in time-shared machines where users sit at a terminal and demand interactive performance, a new metric was introduced: response time

จุดสังเกต

- A เริ่มทำงานก่อน เมื่อ B และ C มาถึง scheduler จะเปรียบเทียบ **remaining time**
- หากงานใหม่มี remaining time สั้นกว่า A, STCF สามารถ preempt A ได้
- จุดสอบคือแยก total job length ของ SJF กับ remaining time ณ ตอนตัดสินใจของ STCF

ที่มา: Lecture 3 หน้า 11.

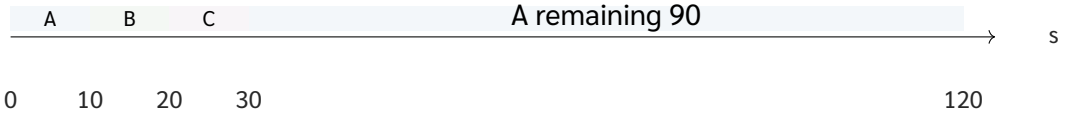
3.5 เมื่อ Arrival Time ต่างกัน: STCF / Preemptive SJF

ในตัวอย่างถัดมา Lecture เปลี่ยนเงื่อนไขให้ A มาถึงที่ $t = 0$ และใช้เวลา 100 s ส่วน B กับ C มาถึงที่ $t = 10$ และใช้เวลา 10 s เท่ากัน หาก scheduler ไม่ preempt A ค่า average turnaround จะเป็น 103.33 s ตามสไลด์

Shortest Time-to-Completion First (STCF) หรือ Preemptive SJF ตรวจ remaining time เมื่อมี job ใหม่เข้ามา และรัน job ที่เหลือเวลาน้อยที่สุด

ตัวอย่างคำนวณ

Timeline STCF:



เวลาที่แต่ละ job เสร็จคือ A=120, B=20 และ C=30 ส่วนเวลาที่มาถึงคือ A=0 และ B=C=10

$$T_A = 120, \quad T_B = 10, \quad T_C = 20$$

$$\overline{T}_{turnaround} = \frac{120 + 10 + 20}{3} = 50 \text{ s}$$

ดังนั้น response time ของ A และ B เท่ากับ 0 เพราะเริ่มทำงานทันทีเมื่อมาถึง ส่วน C เริ่มที่ $t = 20$ หลังมาถึงที่ $t = 10$ จึงมี response time เท่ากับ 10 s

$$\overline{T}_{response} = \frac{0 + 0 + 10}{3} = 3.33 \text{ s}$$

> **เชื่อมกับ Computer Architecture**
>
> STCF ต้องอาศัย preemption Lecture ระบุว่า timer interrupt และ context switch ช่วยให้ scheduler หยุด A แล้วเปลี่ยนไปรัน job ใหม่ได้ นี่คือตัวอย่างที่ hardware mechanism รองรับการทำงานของ OS policy

3.6 Round Robin (RR)

Round Robin (RR) ไม่ปล่อยให้ job หนึ่งรันจนเสร็จ แต่ให้แต่ละ job ใช้ CPU ตาม **time slice / quantum** แล้วสลับไป job ถัดไปใน run queue ทำซ้ำจนเสร็จ วิธีนี้ช่วยแบ่งเวลาใช้ CPU ให้ใกล้เคียงกันในช่วงสั้น
Lecture ระบุว่า time slice ต้องเป็นจำนวนเท่าของ timer-interrupt period ตัวอย่างเช่น หาก timer interrupt เกิดทุก 10 ms ก็เลือก quantum ได้เป็น 10, 20 ms และค่าที่เป็นเท่าต่อไป

> **ตัวอย่างคำนวณ**
>
> ตัวอย่างเปรียบเทียบใน Lecture ให้ A, B และ C มาถึงพร้อมกัน โดยแต่ละ job ต้องการ CPU 5 s
> สำหรับ SJF เวลาเริ่มทำงานครั้งแรกคือ 0, 5 และ 10 s จึงได้ average response $= (0 + 5 + 10)/3 = 5$ s
> สำหรับ RR (ตามผลในสไลด์ ใช้ time slice ที่ทำให้ first-run offsets เป็น 0, 1 และ 2):
>
> $$\overline{T}_{response} = \frac{0 + 1 + 2}{3} = 1 \text{ s}$$
>
> RR จึงตอบสนองได้เร็วกว่า แม้ turnaround โดยทั่วไปจะไม่ใช่จุดแข็งของวิธีนี้

ภาพประกอบ 3.3 Lecture 3 หน้า 14 — Round Robin

Round Robin (RR)



- Instead of running jobs to completion, RR
  - runs a job for a time slice
  - then switches to the next job in the run queue
  - repeatedly does so until all jobs are finished
- RR is considered ‘fair’, as it evenly divides the CPU among active processes on a small time scale
- Note that the length of a time slice must be a multiple of the timer-interrupt period
  - If the timer interrupts every 10 milliseconds, the time slice could be 10, 20, or any other multiple of 10 ms

จุดสังเกต

- RR แบ่ง CPU เป็น time slice แล้วหมุนไปตาม run queue
- quantum สั้นช่วยลด first response time แต่ทำให้เกิด context switch บ่อยขึ้น
- quantum ที่ยาวมากทำให้ RR มีพฤติกรรมใกล้ FIFO

ที่มา: Lecture 3 หน้า 14.

3.6.1 Quantum trade-off และ Amortization

- quantum สั้นช่วยให้ตอบสนองเร็ว แต่ context-switch overhead เกิดบ่อย
- quantum ยาวช่วยเฉลี่ย switch cost ต่อเวลาทำงานได้ดีขึ้น แต่ทำให้ระบบตอบสนองช้าลง

ตัวอย่างในสไลด์กำหนด quantum 10 ms และ context switch 1 ms จึงมี overhead ราว 10% หากเพิ่ม quantum เป็น 100 ms overhead จะลดลงเหลือประมาณ 1% ตามแบบจำลองใน Lecture

> **จุดที่มักสับสน**
>
> อย่าสรุปว่า quantum ยิ่งสั้นยิ่งดีโดยไม่กล่าวถึง overhead การเลือก quantum ต้องสมดุลระหว่าง responsiveness กับต้นทุนของ context switch

3.7 Trade-off ระหว่าง Turnaround กับ Response

Lecture แยกแนวโน้มของแต่ละ policy ไว้ชัดเจน:

- SJF/STCF เลือกงานที่เหลือเวลาสั้น จึงมุ่งลด turnaround แต่อาจตอบสนองต่อ job อื่นช้า
- RR สลับให้แต่ละ job ได้ CPU จึงมุ่งลด response time แต่อาจไม่ได้ลด turnaround

นี่คือตัวอย่างของ system-design trade-off เพราะไม่มี policy เดียวที่ให้ผลดีที่สุดกับทุก metric และทุก workload

3.8 นำ I/O มาพิจารณาในการจัดตาราง

เมื่อ job เริ่ม I/O ก็ไม่ต้องใช้ CPU และจะเปลี่ยนเป็น Blocked ระหว่างนั้น scheduler สามารถให้ process อื่นทำงานได้ เมื่อ I/O เสร็จ hardware interrupt จะแจ้ง OS ให้ย้าย process กลับเป็น Ready

> **เชื่อมกับ Computer Architecture**
>
> Computer Architecture อธิบายว่า hardware ส่ง IRQ เพื่อเรียก ISR หรือ handler ก่อน return ส่วน OS เชื่อมเหตุการณ์นี้กับ process state: เมื่อเริ่ม I/O process จะเป็น **Blocked** และเมื่อ device completion interrupt เกิดขึ้นจะกลับเป็น **Ready**

ตัวอย่างใน Lecture ให้ A และ B ต้องการ CPU คนละ 50 ms โดย A ใช้ CPU ครั้งละ 10 ms สลับกับ I/O ครั้งละ 10 ms ส่วน B ใช้ CPU ต่อเนื่อง หากรอให้ A ทำงานทั้งหมดจบก่อนแล้วค่อยให้ B เริ่ม ระบบจะพลาดโอกาสทำ CPU และ I/O ซ้อนกัน Scheduler จึงสลับไปรัน B ระหว่างที่ A รอ I/O ได้

3.9 เมื่อไม่รู้ Job Length ล่วงหน้า: Multi-Level Feedback Queue (MLFQ)

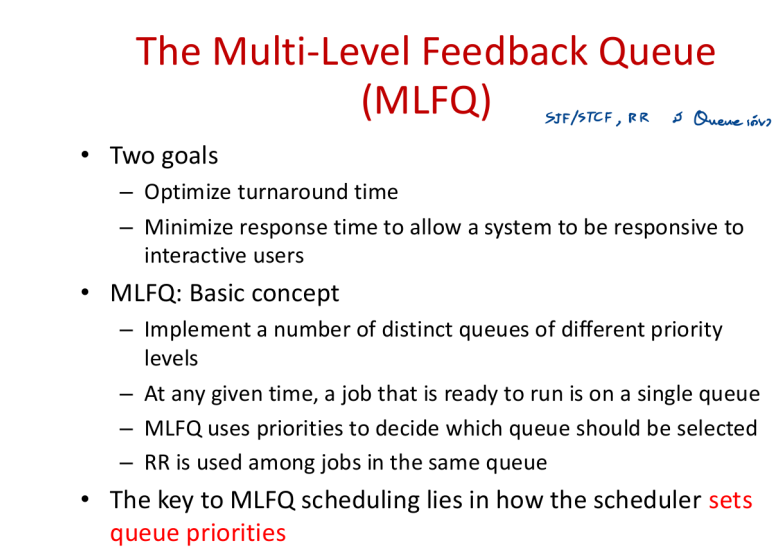
ในระบบจริง OS ไม่รู้ว่า job จะใช้ CPU นานเท่าไร MLFQ จึงพยายามตอบโจทย์สองด้านพร้อมกัน:

- ลด turnaround โดยให้งานสั้นได้ priority สูง
- ลด response time สำหรับผู้ใช้ interactive

MLFQ มีหลาย queue ซึ่งกำหนด priority ต่างกัน process ที่อยู่ในสถานะ Ready จะอยู่ใน queue เดียวในแต่ละขณะ Scheduler เลือกจาก queue ที่มี priority สูงก่อน และใช้ RR เมื่อมีหลาย process อยู่ใน queue ระดับเดียวกัน

> **อ่านภาพและหมายเหตุ**
>
> สไลด์หน้า 22 มีหมายเหตุเขียนมือว่า “สอบแน่นอน” ไว้ใกล้คำถามเรื่อง OS ไม่รู้ job length และคำตอบ MLFQ จึงควรฝึกอธิบายแนวคิดและไล่การทำงานของ MLFQ ให้คล่อง

ภาพประกอบ 3.4 Lecture 3 หน้า 24 — Multi-Level Feedback Queue



จุดสังเกต

- MLFQ มีหลาย priority queues โดยเลือก queue ที่มี priority สูงก่อน
- feedback มาจากพฤติกรรมการใช้ CPU ไม่ใช่ job length ที่รู้ล่วงหน้า
- ภายใน queue ที่มี priority เท่ากัน ใช้ RR ตามกฎใน Lecture

ที่มา: Lecture 3 หน้า 24.

3.9.1 ใช้พฤติกรรมที่สังเกตได้กำหนด Priority

MLFQ ปรับ priority ตามพฤติกรรมที่สังเกตได้:

- job ที่รอ keyboard หรือ I/O บ่อยมักมีพฤติกรรม interactive จึงได้รับ priority สูง
- job ที่ใช้ CPU ต่อเนื่องเป็นเวลานานจะถูกลด priority

3.9.2 Final MLFQ Rules ตาม Lecture

1. ถ้า Priority(A) > Priority(B) จะให้ A ใช้ CPU ก่อน B
2. ถ้า Priority(A) = Priority(B) จะแบ่ง CPU ด้วย RR
3. เมื่อมี job ใหม่เข้าระบบ ให้วางไว้ใน queue ที่มี priority สูงสุด
4. เมื่อ job ใช้ **time allotment** ของ queue นั้นครบแล้ว ให้ลด priority ลงหนึ่งระดับ ไม่ว่า job จะคืน CPU ก่อนหมด quantum กี่ครั้งก็ตาม
5. เมื่อครบช่วงเวลา S ให้ย้ายทุก job กลับไปยัง queue ที่มี priority สูงสุด (priority boost)

ภาพประกอบ 3.5 Lecture 3 หน้า 35 — Final MLFQ Rules

The Final Rules

- ***Rule 1:*** If Priority(A) > Priority(B), A runs (B doesn't)
- ***Rule 2:*** If Priority(A) = Priority(B), A & B run in RR
- ***Rule 3:*** When a job enters the system, it is placed at the highest priority (the topmost queue)
- ***Rule 4:*** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e. it moves down one queue)
- ***Rule 5:*** After some time period S, move all the jobs in the system to the topmost queue

จุดสังเกต

- Rule 1–2 = เลือก priority queue และใช้ RR เมื่อ priority เท่ากัน
- Rule 3 = new job เริ่มบนสุด
- Rule 4 = ใช้ time allotment สะสม ปิดช่องgaming
- Rule 5 = periodic priority boost แก้ starvation / เปิดทางให้ behavior ที่เปลี่ยนถูกประเมินใหม่

ที่มา: Lecture 3 หน้า 35 ใช้ทบทวนกฎ MLFQ ฉบับสุดท้าย

> **อธิบายให้เห็นภาพ**
>
> กฎข้อ 4 ฉบับสุดท้ายแก้ช่องโหว่ของกฎรุ่นก่อน หากนับเฉพาะการใช้ quantum จนหมด process อาจคืน CPU ก่อนหมด quantum ซ้ำ ๆ เพื่อหลีกเลี่ยงการลด priority กฎใหม่จึงสะสมเวลา CPU ที่ใช้ในแต่ละระดับจนครบ time allotment

3.9.3 ปัญหาที่ MLFQ ต้องป้องกัน

Starvation: หากมีงาน interactive จำนวนมาก งาน batch ที่ใช้เวลานานอาจไม่ได้ใช้ CPU กฎ priority boost จึงช่วยให้ทุก job ได้เริ่มต้นใหม่ในระดับสูงสุด

Gaming: process อาจพยายาม yield ก่อน quantum หมดเพื่อหลีกเลี่ยงการลด priority กฎ time allotment ป้องกันวิธีนี้โดยนับเวลา CPU สะสมในแต่ละระดับ

พฤติกรรมที่เปลี่ยนไป: งาน CPU-bound อาจกลายเป็นงาน interactive ได้ การ boost เป็นระยะช่วยให้ scheduler ประเมินพฤติกรรมของงานใหม่

จำนวน queue, quantum ของแต่ละ queue และช่วงเวลา S ต้องปรับให้เหมาะกับ workload Lecture ไม่ได้กำหนดค่าชุดเดียวที่ใช้ได้กับทุกระบบ

จุดเน้นก่อนสอบ

เมื่อทบทวน MLFQ ควรเขียนกฎทั้ง 5 ข้อและไล่ให้ได้ว่า job เลื่อน queue เพราะอะไร ควรอธิบายวิธีแก้ starvation และ gaming ได้ด้วย รวมทั้งชี้ให้เห็นว่า MLFQ ใช้พฤติกรรมที่สังเกตได้มาประมาณ job length ที่ระบบไม่รู้ล่วงหน้า

3.10 Proportional-Share และ Lottery Scheduling

ช่วงท้ายของ Lecture เปลี่ยนเป้าหมายมาเป็นการแบ่ง CPU ตามสัดส่วน Proportional-Share Scheduler จึงพยายามให้แต่ละ job ได้ส่วนแบ่ง CPU ตามจำนวน ticket ที่ถือ

3.10.1 Tickets

Ticket ใช้แทนส่วนแบ่ง CPU หาก A มี 75 tickets และ B มี 25 tickets จากทั้งหมด 100 ใบ เมื่อมองในระยะยาว A ควรได้ CPU ราว 75% และ B ราว 25%

3.10.2 Holding a Lottery

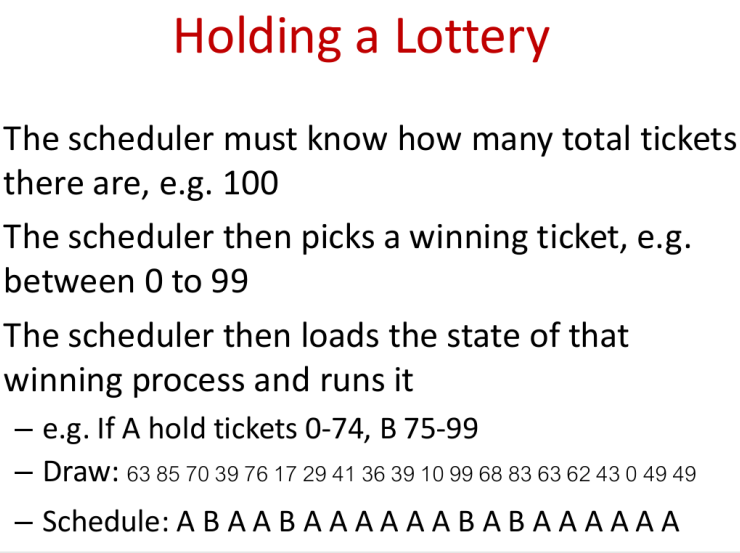
ในแต่ละรอบการจัดตารางหรือ time slice:

1. scheduler รู้จำนวน ticket รวม เช่น 100
2. สุ่มหมายเลข ticket ที่ชนะระหว่าง 0–99
3. ตรวจว่า ticket นั้นเป็นของ process ใด
4. คืน state ของ process ที่ชนะ แล้วให้ใช้ CPU

ตัวอย่างใน Lecture กำหนดให้ A ถือ ticket หมายเลข 0–74 และ B ถือ 75–99 หากสุ่มได้ลำดับ 63, 85, 70, 39, 76, ... ผู้ชนะในแต่ละรอบจะแปรผัน แต่สัดส่วนการได้ CPU จะเข้าใกล้จำนวน ticket เมื่อดูในระยะยาว

ตาม Lecture วิธีสุ่มช่วยหลีกเลี่ยงกรณีขอบบางแบบ ใช้ state ที่ต้องติดตามไม่มาก และทำงานได้รวดเร็ว อย่างไรก็ตาม ผลลัพธ์ขึ้นอยู่กับการจัดสรร ticket มาก Lecture ยกตัวอย่างให้ user ได้รับ ticket แล้วแบ่งให้ job ของตน หรือใช้วิธีจัดสรรที่ซับซ้อนขึ้น

ภาพประกอบ 3.6 Lecture 3 หน้า 39 — Holding a Lottery



จุดสังเกต

- รวม ticket ทั้งหมดแล้วสุ่มหมายเลขที่ชนะ
- process ที่ถือ ticket หมายเลขนั้นได้ CPU ในรอบนี้
- สัดส่วนการใช้ CPU จะเข้าใกล้สัดส่วน ticket ในระยะยาว แต่ไม่จำเป็นต้องเป็น 75/25 ในทุกช่วงสั้น

ที่มา: Lecture 3 หน้า 39.

3.11 Scheduling Comparison

นโยบาย	แย่ง CPU ได้หรือไม่	หลักการ	จุดเด่นและข้อจำกัด
FIFO	ไม่แย่ง	เรียง ตาม เวลามาถึง	เข้าใจง่าย แต่งานสั้นอาจต้องรอหลังงานยาว
SJF	ไม่แย่ง	งานที่ใช้เวลาสั้นที่สุด	ลด average turnaround ได้เมื่อทุกงานมาพร้อมกัน แต่ต้องรู้ความยาวงาน
STCF	แย่งได้	เวลาที่เหลือน้อยที่สุด	ลด turnaround เมื่อเวลา arrival ต่างกัน แต่อาจตอบสนองช้า
RR	แย่งได้	แบ่ง เวลา เป็น quantum	ตอบสนองเร็ว แต่มี context-switch overhead
MLFQ	แย่งได้	feedback และ priority queue	ประมาณพฤติกรรมของงานโดยไม่รู้ job length แต่ต้องป้องกัน starvation และ gaming
Lottery	แย่ง ได้ เมื่อแบ่งเวลา	สุ่มตาม ticket	ให้สัดส่วน CPU ในระยะยาวตามจำนวน ticket

3.12 Chapter Summary

สรุปท้ายบท

- Scheduling เป็น policy ส่วน timer interrupt และ context switch เป็น mechanism metrics หลัก คือ turnaround และ response
- FIFO เรียบง่าย SJF/STCF มุ่งลด turnaround ส่วน RR มุ่งให้ตอบสนองเร็ว โดยต้องคำนึงถึง quantum และ context-switch overhead
- เมื่อเริ่ม I/O process จะเป็น Blocked และกลับ Ready เมื่อ completion interrupt มาถึง ทำให้ CPU กับ I/O ทำงานซ้อนกันได้
- MLFQ ใช้ feedback และ priority เพื่อประมาณพฤติกรรม interactive กับ CPU-bound พร้อมกฎป้องกัน starvation และ gaming
- Lottery scheduling ใช้ ticket และการสุ่มเพื่อจัดสรร CPU ตามสัดส่วน

Formula / Calculation Recap

$$T_{turnaround} = T_{completion} - T_{arrival}, \qquad T_{response} = T_{first\text{-}run} - T_{arrival}$$

ค่าเฉลี่ยของ metric = ผลรวมของทุก job หารด้วยจำนวน job ควรวาด timeline ก่อนคำนวณเสมอ

ComArch Connections

Timer interrupt ทำให้เกิด preemption ได้ การ save/restore CPU state เป็นต้นทุนของ context switch และ I/O completion interrupt ทำให้ process เปลี่ยนจาก Blocked เป็น Ready

Common Confusions

- STCF ต่างจาก SJF และ response time ต่างจาก turnaround time
- RR quantum ต่างจาก timer period โดย quantum ต้องเป็นจำนวนเท่าของ timer period ตาม Lecture
- MLFQ priority boost ย้ายทุก job ไป queue บนสุด ส่วนกฎข้อ 4 นับ time allotment สะสมเพื่อป้องกัน gaming

Quick Revision Checklist

- ☐ วาด timeline ของ FIFO/SJF/STCF/RR และคำนวณค่าเฉลี่ย
- ☐ อธิบาย quantum กับ context-switch overhead
- ☐ ไล่ I/O จาก Blocked ผ่าน interrupt กลับ Ready
- ☐ ทวนกฎ MLFQ และวิธีแก้ starvation/gaming
- ☐ อธิบายสัดส่วน ticket 75/25

Lecture 4 — Memory Concept and Contiguous Memory Allocation

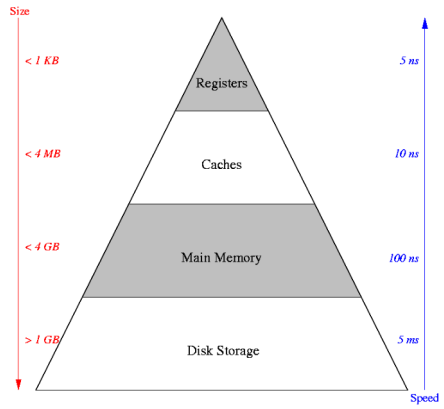
อ้างอิง: Lecture 4 หน้า 1–35 | address translation, Base/Bounds, contiguous allocation และ fragmentation

> **หมายเหตุเรื่องการตรวจสไลด์ Lecture 4**
>
> บทนี้สรุปหัวข้อ Lecture 4 จากภาพสไลด์ที่กู้คืนได้ 7 หน้าและบัญชีหัวข้อเดิม อย่างไรก็ตามไฟล์ PDF ต้นฉบับเสียหาย จึงยังตรวจเทียบรายละเอียดกับสไลด์ครบทั้ง 35 หน้าไม่ได้

4.1 Memory Hierarchy และหน้าที่ของ OS

Lecture เริ่มจากทบทวนลำดับชั้นของหน่วยความจำ เพื่อเปรียบเทียบความจุและความเร็วของแต่ละชนิด

ภาพประกอบ 4.1 Lecture 4 หน้า 3 — Memory Hierarchy Review



จุดสังเกต

- เมื่อพิจารณาจาก register และ cache ไปสู่ main memory และ disk จะพบว่า capacity เพิ่มขึ้น แต่ latency ก็สูงขึ้น
- Lecture 4 เน้น main memory ส่วน Lecture 6 จะกล่าวถึง disk ในฐานะ backing store
- แม้ executable จะเก็บอยู่บน disk แต่ CPU นำคำสั่งมาทำงานจาก disk โดยตรงไม่ได้ ส่วนที่ต้องใช้จึงต้องถูกนำเข้า main memory ก่อน

ที่มา: Lecture 4 หน้า 3.

Register และ cache มีความจุน้อยแต่เข้าถึงได้เร็วกว่า main memory ส่วน disk มีความจุสูงกว่าแต่ใช้เวลาเข้าถึงนานที่สุด โปรแกรมเก็บ executable ไว้บน disk และต้องนำคำสั่งกับข้อมูลที่จำเป็นเข้า main memory ก่อนเริ่มทำงาน

> **เนื้อหาจาก Lecture**
>
> หน้าที่ของ OS ด้าน memory ที่สไลด์หน้า 4 เน้นมีสามแกน:
> 1. ทำ virtualize, manage และ organize memory
> 2. จัด abstraction ให้โปรแกรมใช้งานได้สะดวก โดยคำนึงถึงทั้งเวลาและพื้นที่
> 3. ป้องกันและแยกพื้นที่ของ process ออกจากกัน

4.2 Memory / Address Space Abstraction

Physical Address คือตำแหน่งที่ memory hardware ใช้จริง ส่วน **Logical Address / Virtual Address** คือตำแหน่งที่ program อ้างอิงผ่าน abstraction ของ OS แต่ละ process จึงเห็น address space ของตนเอง แม้ตำแหน่งจริงจะอยู่ใน physical memory ร่วมกัน

> **จุดที่มักสับสน**
>
> **Logical/Virtual Address ต่างจาก Physical Address** address ที่โปรแกรมเห็นไม่จำเป็นต้องตรงกับตำแหน่งจริงใน RAM ระบบจึงต้องแปล address ก่อนเข้าถึง physical memory

4.2.1 Programmable Machine → Time Sharing

ใน programmable machine รุ่นแรก มี program เดียวใช้ memory ส่วนใหญ่ร่วมกับ OS เมื่อระบบพัฒนาเป็น time-sharing และ multiprogramming แต่ละ process จึงต้องมี address space ของตนเอง ทำให้ต้องแก้ปัญหา mapping, protection และ relocation

4.3 Address Translation / Address Binding

Address Translation คือการแปลง logical/virtual address เป็น physical address Lecture แบ่งช่วงเวลาที่ทำ address binding ได้ดังนี้

ช่วงเวลา	แนวคิด	ผลที่ตามมา
Compile time	หากทราบตำแหน่ง memory ล่วงหน้า จะสร้าง absolute code ได้	หากเปลี่ยนตำแหน่งเริ่มต้น ต้อง compile ใหม่
Link/Load time	สร้าง relocatable code แล้วกำหนดตำแหน่งตอน link หรือ load	เปลี่ยนตำแหน่งได้ก่อนเริ่มทำงาน แต่โดยทั่วไปต้อง load ใหม่
Execution time	เลื่อน การ กำหนด ตำแหน่ง ไป จนถึง runtime และย้าย process ได้ระหว่างทำงาน	ต้องมี hardware ช่วยแปล address แบบ dynamic

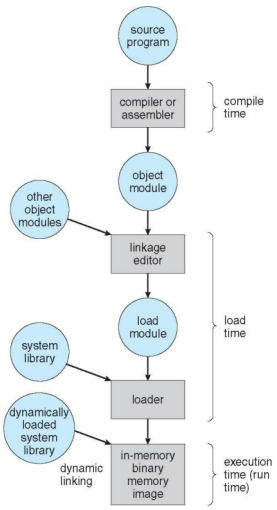
> **เชื่อมกับ Computer Architecture**
>
> Execution-time binding ต้องมี hardware อยู่ในเส้นทางของ memory reference เพราะ CPU สร้าง address ระหว่างทำงาน ระบบจึงต้องแปล address และตรวจสิทธิ์ก่อนเข้าถึง memory กลไกนี้เป็นพื้นฐานของ MMU, Page Table และ TLB ที่จะเรียนใน Lecture 5

ภาพประกอบ 4.2 Lecture 4 หน้า 8 — Address Translation / Binding

แปลง address ขั้นตอนไหน?

Address Translation

Translate logical (virtual) address to physical address (aka. address binding)



8

จุดสังเกต

- Program อ้างอิง logical/virtual address ส่วน memory hardware ใช้ physical address
- Binding ทำได้ตอน compile, load หรือ execution ให้สังเกตว่าตำแหน่งจริงถูกกำหนดเมื่อใด
- หากทำ binding ตอน execution ต้องมี hardware แปล address ในเส้นทางเข้าถึง memory

ที่มา: Lecture 4 หน้า 8.

4.4 Programming API Perspective: code, stack, heap

Lecture แยกมุมมอง storage class ของภาษา C ออกจากชนิดหน่วยความจำที่ OS จัดการ

- **Code**: program instructions
- **Stack**: data ที่ compiler/runtime จัดตำแหน่งให้อัตโนมัติ เช่น local/Auto storage
- **Heap**: dynamic data ที่ programmer ขอผ่าน allocator เช่น `malloc()`

บาง OS อาจแบ่ง segment มากกว่าสามส่วน

4.5 Memory Management and Organization

Lecture แยกคำถามด้านการจัดการ memory ออกเป็นเรื่องการจัดสรรและการเลือกตำแหน่ง

- **Memory Allocation**: single/multiple processes, static/dynamic, contiguous/noncontiguous
- **Placement / Replacement**: ตัดสินว่าข้อมูลใหม่ควรอยู่ตำแหน่งใด และภายหลังใน VM ว่าอะไรควรถูกแทนออก

4.5.1 Contiguous vs Noncontiguous

Contiguous Allocation (การจัดวางต่อเนื่อง): address space ทั้งหมดของ process ต้องอยู่ใน block เดียวของ physical memory หากไม่มี hole ที่ใหญ่พอ ก็ไม่สามารถจัดวาง process ได้

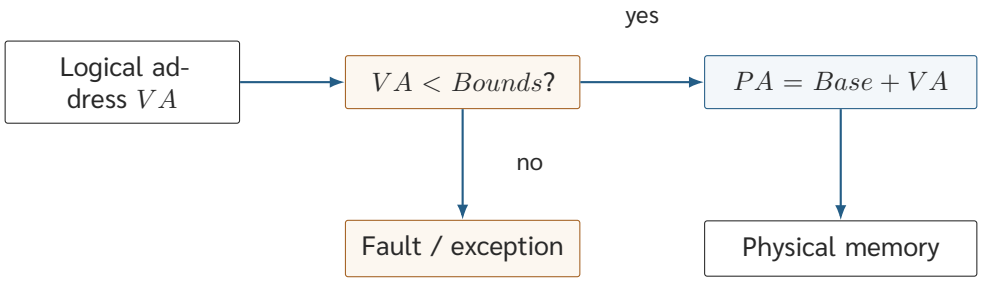
Noncontiguous Allocation (การจัดวางแบบไม่ต่อเนื่อง): แบ่ง process เป็น pages หรือ segments แล้ววางไว้คนละตำแหน่งใน physical memory จึงใช้พื้นที่ว่างขนาดเล็กได้ดีขึ้น Lecture 5 จะอธิบายวิธีนี้ต่อ

4.6 Single Partition Allocation และ Boundary Scheme

สมมติว่ามี process เดียวกำลังทำงาน และ address space เล็กกว่า physical memory **Boundary Scheme** ใช้ register สองตัว:

- **Base / Relocation Register**: physical address เริ่มต้น
- **Bounds / Limit Register**: ขนาด/ช่วง logical address ที่อนุญาต

ตามแบบจำลองใน Lecture CPU มี register คู่นี้หนึ่งชุด



ตัวอย่างคำนวณ

ตัวอย่างใน Lecture กำหนด address space ตั้งแต่ 0 ถึง $4\text{KB} - 1$ โดยมี Base=16 KB=16384 B และ Bounds=4 KB=4096 B
General rule:

$$0 \le VA < Bounds \Rightarrow PA = Base + VA$$

$$\begin{aligned} VA = 0 &\rightarrow PA = 16384 \\ VA = 1024\ \text{B} &\rightarrow 16384 + 1024 = 17408\ \text{B} \\ VA = 3000\ \text{B} &\rightarrow 16384 + 3000 = 19384\ \text{B} \\ VA = 4400\ \text{B} &\rightarrow \text{out of bounds} \rightarrow \text{Fault} \end{aligned}$$

จุดเน้นก่อนสอบ

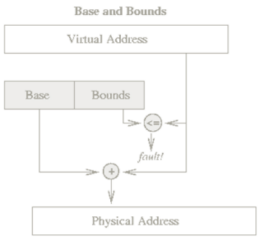
ทำ address problem ให้เรียงสองขั้นเสมอ: **(1) bounds check ก่อน (2) ค่อย add Base.** การบวกก่อนแล้วค่อยดูทีหลังทำให้หลุด protection logic ของ scheme.

ภาพประกอบ 4.3 Lecture 4 หน้า 18 — Base / Bounds Worked Example

Example

- Suppose the address space of the process is 0 to 4KB-1
- Suppose the Base register contains 16KB and the Bounds register contains 4KB
- The virtual address is compared with the Bounds register contents to check within bounds
- The virtual address is added to the Base register contents to give the physical address

Virtual	→	Physical Address
0	→	16 KB (16384)
1 KB	→	17 KB (17408)
3000	→	19384
4400	→	*Fault (out of bounds)*



18

จุดสังเกต

- Logical address ต้องผ่าน **bounds check** ก่อน
- valid แล้วค่อยคำนวณ $PA = Base + VA$
- ตัวอย่าง Base=16KB, Bounds=4KB ทำให้เห็นทั้ง relocation และ protection ใน register pair เดียว

ที่มา: Lecture 4 หน้า 18.

4.7 Multiple Partition Allocation

เมื่อรองรับหลาย process physical memory ถูกแบ่ง partition. Lecture แบ่งสองแบบใหญ่:

- **Fixed-size partitions**: partition ขนาดคงที่ เปลี่ยน runtime ไม่ได้
- **Flexible/Variable/Dynamic partitions**: partition ขนาดตาม process

ทั้งสองยังเป็น contiguous allocation: address space ของ process หนึ่งยังต้องอยู่ติดกันใน physical memory

4.8 Static vs Dynamic Relocation

4.8.1 Static Relocation

Loader rewrite address ใน executable ด้วย relocation value ก่อน run. Lecture example instruction อ้าง logical address 1000 และ relocation value 3000 จึง loader เปลี่ยนเป็น address 4000.
ข้อจำกัดตาม Lecture:

- protection ไม่แข็งแรง — process อาจสร้าง bad address
- relocation value เปลี่ยนไม่สะดวก; ต้อง reload

4.8.2 Dynamic Relocation

Base/Bounds ถูกใช้ระหว่าง runtime; แต่ละ process มี relocation value ของตัวเองและ OS เปลี่ยน Base/Bounds เมื่อ context switch

> **เชื่อมกับ Computer Architecture**
>
> Lecture ระบุ hardware requirements ชัด: ต้องมี Base/Bounds registers, circuitry สำหรับ translate + bounds check, privileged instructions เพื่อ update registers, ability to raise exception เมื่อ illegal access, และ privileged mechanism สำหรับ register exception handlers. เหตุผลที่ instruction แก้ Base/Bounds ต้อง privileged: ถ้า user process ตั้ง Base เองได้ มันสามารถ map ไป physical memory ของ process/kernel อื่นได้

ภาพประกอบ 4.4 Lecture 4 หน้า 32 — Hardware Requirements for Base/Bounds

Hardware Requirements

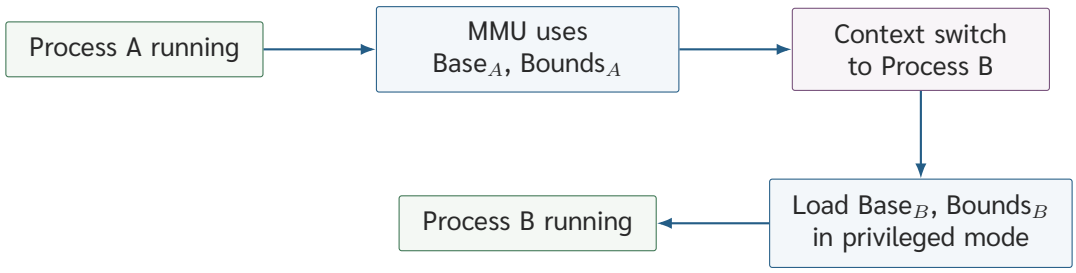
- Base/Bounds pair of registers
  - *Need pair per CPU to support address translation and bounds checks*
- Ability to translate virtual addresses and check if within limits
  - *Circuitry to do translations and check bounds limits*
- Privileged instruction(s) to update Base/Bounds registers
  - *Must be able to set these values before letting a program run*
- Ability to raise exceptions
  - *When processes try to access privileged instructions or out-of-bounds memory*
- Privileged instruction(s) to register exception handlers
  - *OS must be able to tell hardware what code to run if exception occurs*

32

จุดสังเกต

- ต้องมี register pair + circuitry แปล address และ bounds check ใน memory-reference path
- การตั้ง Base/Bounds และ exception handler ต้องเป็น **privileged operation**
- illegal/out-of-bounds access ต้องทำให้ hardware raise exception เข้าสู่ OS

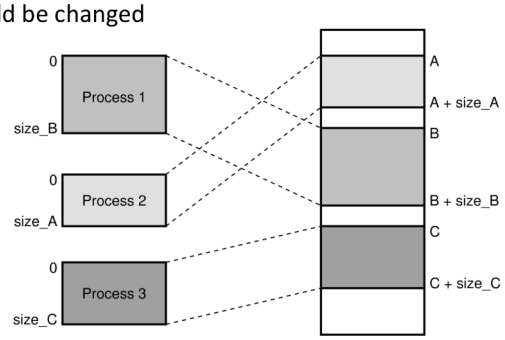
ที่มา: Lecture 4 หน้า 32 ภาพนี้เชื่อม memory virtualization กับ hardware support



ภาพประกอบ 4.5 Lecture 4 หน้า 26 — Dynamic Relocation + Context Switch

Example of Dynamic Relocation

- After context switching, the Base and Bounds register pair should be changed



26

จุดสังเกต

- แต่ละ process มี Base/Bounds ของตัวเอง
- ตอน context switch kernel ต้องโหลด pair ของ process ใหม่ให้ CPU/MMU ใช้
- นี่คือ connection โดยตรงระหว่าง Process context (Lecture 2) กับ Memory protection (Lecture 4)

ที่มา: Lecture 4 หน้า 26.

4.9 Fixed-Size Partitions: Internal Fragmentation

ถ้า executable/process เล็กกว่า partition พื้นที่ที่เหลือใน partition ถูกจองไว้แต่ใช้ไม่ได้ จึง **Internal Fragmentation**. ถ้า process ใหญ่กว่า partition ก็ใส่ไม่ได้เลย

อธิบายให้เห็นภาพ

คำว่า internal ช่วยจำตำแหน่งของ waste: พื้นที่สูญเปล่าอยู่ **ภายใน** block/partition ที่ถูก allocate ให้ process แล้ว

ภาพประกอบ 4.6 Lecture 4 หน้า 27 — Fixed Partitions → Internal Fragmentation

Problem of Fixed-sized Partitions

- An executable program is too small for a partition
  - That mean wasting memory space as some of the partition will be unused – this is called *internal fragmentation* ของเกินแต่ไม่ใช้
- An executable program is too big to fit a partition

27

จุดสังเกต

- process เล็กกว่า partition จึง พื้นที่ **ภายใน partition ที่จัดให้แล้ว** ถูกทิ้ง = internal fragmentation
- อีกกรณีคือ program ใหญ่กว่า partition จนใส่ไม่ได้ แม้ total memory อาจยังเหลือ
- จำ keyword จากตำแหน่งของ waste: **inside allocated region** = internal

ที่มา: Lecture 4 หน้า 27 หมายเหตุในสไลด์ชี้ว่าพื้นที่ที่จองไว้ไม่ได้ถูกใช้

4.10 Flexible-Size Partitions: External Fragmentation

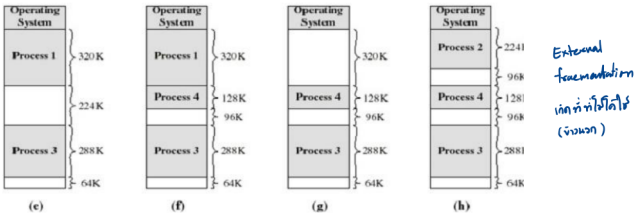
Flexible partitions จัดพื้นที่ตามขนาด process จึงลด internal fragmentation แต่เมื่อ process terminate จะทิ้ง holes หลายก้อน ถ้าพื้นที่ว่างรวมพอแต่ไม่มี hole เดียวใหญ่พอสำหรับ process ใหม่ จะเกิด **External Fragmentation**.

Lecture ใช้ตัวอย่าง physical memory 1024KB, OS 128KB และ job sizes 320,224,288,128,224,192KB เพื่อแสดงการวางและเกิด holes เมื่อ process เข้า/ออก

ภาพประกอบ 4.7 Lecture 4 หน้า 29 — External Fragmentation

External Fragmentation

- External Fragmentation occurs when processes are terminated, and no hole spaces are big enough for a new incoming process (suppose we want to execute a process which is 192KB)
- (1)320KB, (2)224KB, (3)288KB, (4)128KB, (5)224KB, (6)192KB



29

จุดสังเกต

- Free memory รวมอาจพอ แต่แตกเป็น holes ที่ไม่มี block เดียวใหญ่พอ
- นี่คือ **external** fragmentation: waste อยู่ระหว่าง allocations
- Coalescing รวม hole ที่ติดกัน; Compaction ย้าย allocations เพื่อรวม free space แต่มี cost สูงกว่า

ที่มา: Lecture 4 หน้า 29.

4.10.1 Coalescing vs Compaction

วิธีการ	ทำอะไร	ข้อสังเกต
Coalescing	รวม free space ที่อยู่ติดกันให้เป็น hole ที่ใหญ่ขึ้น	ไม่ต้องย้าย block ที่ยังถูกใช้งาน
Compaction	ย้าย process เพื่อรวม free space ให้เป็นก้อนใหญ่	ต้อง อาศัย dynamic relocation เพราะ ต้องเปลี่ยน Base หลังย้าย

> **จุดที่มักสับสน**
>
> **Internal ต่างจาก External Fragmentation** แบบ fixed partition ทำให้พื้นที่ว่างเหลืออยู่ภายใน partition ที่จัดให้ process แล้ว ส่วนแบบ flexible partition ทำให้เกิด holes คั่นระหว่างพื้นที่ที่ถูกจัดสรร วิธีแก้จึงต่างกัน

4.11 Free List และ lifecycle ของ memory

เมื่อสร้าง process OS จะค้น data structure เช่น **free list** เพื่อหาพื้นที่สำหรับ address space แล้วทำเครื่องหมายว่าถูกใช้งาน เมื่อ process จบ OS คืน memory เข้า free list และล้าง structures ที่เกี่ยวข้อง นอกจากนี้ Base/Bounds ต้องถูกบันทึกและคืนค่าระหว่าง context switch ส่วน dynamic relocation ทำให้ย้าย process ที่หยุดอยู่ไปยังตำแหน่งอื่นใน memory ได้

4.12 Trailing Lecture Material: C Storage Classes

Lecture หน้า 34–35 กลับมาทบทวนมุมมอง programming API:

- **Auto**: default local variables; lifetime ตาม function/block
- **Static**: local variable คงอยู่ตลอด program lifetime
- **Extern**: reference global variable ที่เห็นข้าม program files
- **Register**: ขอให้ local variable อยู่ register แทน RAM ตาม framing ของ Lecture

ตัวอย่าง Auto: variable จะเริ่มมีอายุเมื่อเข้า function หรือ block และสิ้นสุดเมื่อออกจากบริเวณนั้น ดังนั้น block-local variable เช่น k จึงใช้นอก block ไม่ได้

> **จุดที่มักสับสน**
>
> ส่วน C Storage Classes อยู่ใน Lecture จริง แต่เป็น programming-API perspective และไม่ควรเอาไปปนกับ OS-level Base/Bounds หรือ segmentation scheme. ใช้เพื่อเชื่อมว่าภาษา/compile-time view กับ OS memory view เป็นคนละ abstraction level.

4.13 Chapter Summary

> **สรุปท้ายบท**
>
> - OS virtualizes main memory เพื่อ transparency, efficiency, protection/isolation
> - Logical address ต้องถูก bind/translate เป็น physical address; execution-time binding ต้อง hardware support
> - Base/Bounds ทำ translation แบบง่าย: check Bounds แล้ว add Base
> - Dynamic relocation เปลี่ยน Base/Bounds ตาม process และรองรับ moving process; update ต้อง privileged
> - Fixed partitions จึง internal fragmentation; Flexible partitions จึง external fragmentation
> - Coalescing รวม holes ที่ติดกัน; Compaction ย้าย blocks เพื่อรวม free space และต้อง dynamic relocation
> - Contiguous allocation มีข้อจำกัดที่นำไปสู่ Noncontiguous Allocation ใน Lecture 5

Key Mechanisms

Address translation, Base/Bounds check, exception on invalid access, save/restore Base/Bounds on context switch.

Key Policies

Contiguous placement choice, fixed vs flexible partitioning, free-list management, coalescing/compaction.

Formula / Calculation Recap

$$VA < Bounds \Rightarrow PA = Base + VA; \qquad VA \geq Bounds \Rightarrow Fault$$

หน่วย Base, Bounds, VA ต้องอยู่ unit เดียวกันก่อนบวก/เปรียบเทียบ

ComArch Connections

Physical main memory, registers, privileged register update, compare/add circuitry, exception handler path.

Common Confusions

- logical address ต่างจาก physical address
- Base = starting physical address; Bounds = legal range/size ไม่ใช่ ending address โดยตรงใน lecture model
- Static relocation rewrite address ก่อน run; Dynamic relocation ทำ translation runtime
- internal fragmentation ต่างจาก external fragmentation
- Context switch เปลี่ยน Base/Bounds ของ current process แต่ไม่เท่ากับ moving process ใน memory เสมอไป

Quick Revision Checklist

☐ แยก compile/load/execution-time binding ได้
☐ ทำ Base/Bounds translation และ detect fault ได้
☐ อธิบายเหตุผลที่ Base/Bounds update ต้อง privileged ได้
☐ วาด fixed vs flexible partitions และชี้ internal/external fragmentation ได้
☐ อธิบาย coalescing vs compaction ได้

Lecture 5 — Noncontiguous Memory Allocation

อ้างอิง: Lecture 5 หน้า 1–30 | Segmentation, Paging, page table, TLB และ hybrid translation

5.1 ทำไมต้อง Noncontiguous Allocation

Contiguous allocation ใน Lecture 4 กำหนดให้ address space ของ process อยู่ติดกันทั้งก้อนใน physical memory วิธีนี้ทำให้เกิด fragmentation และต้องกันพื้นที่ให้ช่องว่างใน address space เช่น บริเวณระหว่าง heap กับ stack ด้วย Lecture 5 เสนอวิธีจัดสรรหลักสองแบบ:

- **Segmentation**: แบ่ง logical address space เป็นส่วนที่มีความหมายและขนาดไม่เท่ากัน เช่น code, heap, stack
- **Paging**: แบ่ง logical address space เป็น pages ขนาดเท่ากัน และ physical memory เป็น frames ขนาดเท่ากับ page

5.2 Segmentation Scheme

> **เนื้อหาจาก Lecture**
>
> แบ่ง logical address space ออกเป็น **segments** ที่มีขนาดไม่เท่ากัน แต่ละ segment ยังเป็นช่วงต่อเนื่องของ address space จากนั้นขยาย Boundary Scheme ให้มี **Base + Bounds** แยกสำหรับแต่ละ segment ตัวอย่างที่แบ่งเป็น code, heap และ stack จึงต้องมี Base/Bounds สามคู่ใน MMU

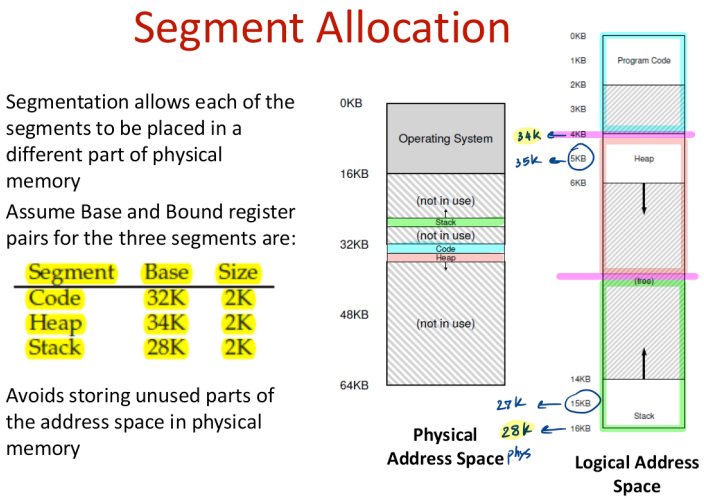
5.2.1 Segment Allocation

ข้อดีคือ code, heap และ stack วางคนละตำแหน่งใน physical memory ได้ จึงไม่ต้องสำรองพื้นที่ของช่องว่างระหว่าง heap กับ stack ไว้ทั้งหมด

ตัวอย่างใน Lecture ใช้ค่าดังนี้:

Segment	Base	Size/Bounds
Code	32 KB	2 KB
Heap	34 KB	2 KB
Stack	28 KB	2 KB

ภาพประกอบ 5.1 Lecture 5 หน้า 5 — Segment Allocation



จุดสังเกต

- code/heap/stack วางคนละตำแหน่งใน physical memory ได้
- แต่ละ segment มี Base + Size/Bounds ของตัวเอง
- benefit หลักคือไม่ต้องเก็บ unused gap ระหว่าง heap กับ stack ทั้งก้อนใน RAM

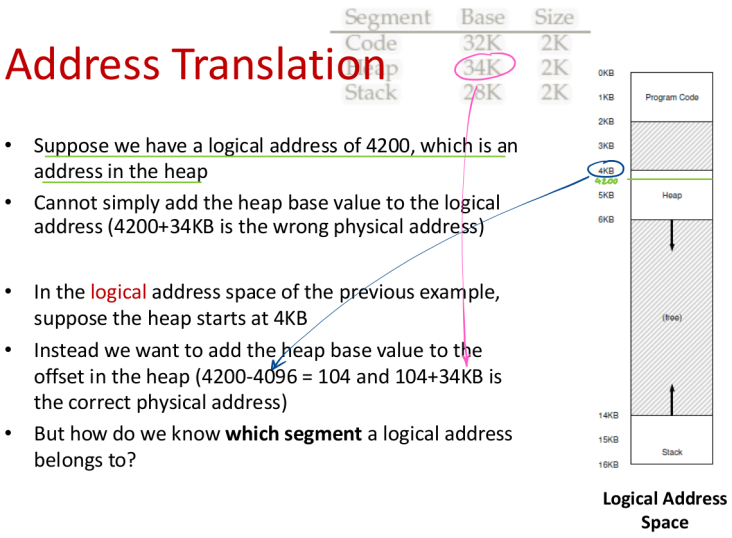
ที่มา: Lecture 5 หน้า 5 พร้อมหมายเหตุเรื่อง base/offset จากสไลด์

5.2.2 Segmentation Address Translation

Logical address ต้องระบุว่าอยู่ segment ใดและ offset ภายใน segment เท่าใด ไม่สามารถเอา logical address ทั้งก้อนไปบวก Base ของ segment ตรง ๆ

> **ตัวอย่างคำนวณ**
>
> ตัวอย่างใน Lecture ให้ logical address เท่ากับ 4200 ซึ่งอยู่ใน heap โดย heap เริ่มที่ logical address 4KB=4096 และมี physical Base เท่ากับ 34KB
> 1. หา offset ภายใน heap:
>
> $$offset = 4200 - 4096 = 104 \text{ B}$$
>
> 2. ตรวจ bounds: $104 < 2$KB ดังนั้น address นี้อยู่ในขอบเขตที่อนุญาต 3. แปลงเป็น physical:
>
> $$PA = 34\text{KB} + 104 = 34816 + 104 = 34920 \text{ B}$$
>
> อย่านำ $4200 + 34$KB มาบวกกันโดยตรง เพราะ 4200 เป็น logical address ของทั้ง process ไม่ใช่ offset ภายใน heap

ภาพประกอบ 5.2 Lecture 5 หน้า 6 — Segmentation Address Translation Example



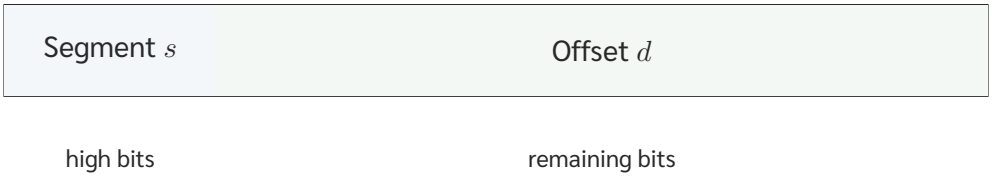
จุดสังเกต

- logical address 4200 อยู่ใน heap ซึ่งเริ่มที่ 4KB จึงได้ offset เท่ากับ $4200 - 4096 = 104$
- อย่าบวก $4200 + Base_{heap}$ โดยตรง เพราะ 4200 ยังเป็น address ของทั้ง logical space
- คำนวณ physical address จาก $Base_{heap} + 104$

ที่มา: Lecture 5 หน้า 6 คงลูกศรและการเน้นในสไลด์ไว้เพื่อชี้ตำแหน่งที่ใช้คำนวณ offset

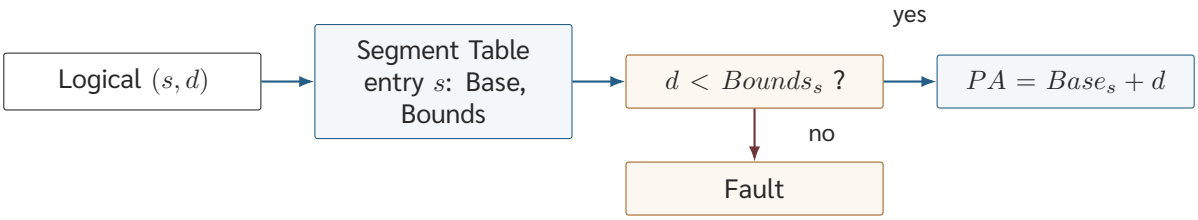
5.2.3 Segment Number + Offset

Lecture เสนอให้ high bits ของ logical address encode segment number และ remaining bits เป็น offset. ถ้ามีสาม segments ใช้สอง bits แยก segment ได้ Stack มีทิศทาง grow backward จึงต้องมีข้อมูลเพิ่มเพื่อบอก growth direction ตาม slide model



5.2.4 Segment Table

เมื่อมีหลาย segments, Base/Bounds ถูกจัดเป็น **Segment Table**; แต่ละ entry เป็น Base+Bounds pair และแต่ละ process มี segment table ของตัวเอง Logical address (s, d) ใช้ s เลือก entry แล้วตรวจ d กับ Bounds ก่อนคำนวณ $Base_s + d$.



หมายเหตุในสไลด์หน้า 8 ระบุว่า Segment Table อยู่ใน RAM การค้นตารางจึงเพิ่มการเข้าถึง memory อีกครั้ง ประเด็นเรื่อง access overhead นี้จะเห็นชัดขึ้นเมื่อเรียน Paging

5.2.5 Protection, growth direction, fine/coarse grain

Lecture ระบุ fields/issue เพิ่ม:

- growth direction ของ segment (สำคัญกับ stack)
- permission เช่น read/write/execute เพื่อ support sharing/protection
- coarse-grained segmentation = segments น้อยและใหญ่; fine-grained = segments มากและเล็ก

Fine-grained segmentation ยังมี external fragmentation เพราะ segment sizes ไม่เท่ากัน

5.2.6 การจัดสรร Memory ให้ Segments

OS เก็บ **free list** ของ holes แล้วเลือกพื้นที่ด้วยวิธีต่อไปนี้:

First Fit
เลือก hole แรกที่มีขนาดพอ เหมาะกับการค้นหาให้เร็ว

Best Fit
เลือก hole ที่เหลือพื้นที่ว่างน้อยที่สุด เพื่อลดพื้นที่เหลือ แต่ก็อาจสร้าง holes ขนาดเล็กหลายจุด

Worst Fit
เลือก hole ที่เหลือพื้นที่ว่างมากที่สุด หวังว่าจะไม่เหลือ holes เล็ก ๆ จำนวนมากเหมือน Best Fit

เพราะ external fragmentation ยังเกิดได้ จึงอาจต้อง coalescing/compaction เช่น Lecture 4

5.3 Paging

> **เนื้อหาจาก Lecture**
>
> Paging แบ่ง logical address space เป็น **pages** ขนาดเท่ากัน และแบ่ง physical memory เป็น **frames** ที่มีขนาดเท่ากับ page ดังนั้นหนึ่ง frame จึงบรรจุได้หนึ่ง page Lecture ระบุขนาด page ที่พบได้ทั่วไปไว้ที่ 512–16K bytes แต่ละ process มี **page table** สำหรับจับคู่ page กับ frame

ข้อดีของ Paging คือจัดการ free space ได้ง่าย เพราะ frames มีขนาดเท่ากัน และไม่เกิด external fragmentation แบบ segmentation อย่างไรก็ตาม page สุดท้ายอาจเหลือพื้นที่ว่างภายใน frame ทำให้เกิด internal fragmentation ได้

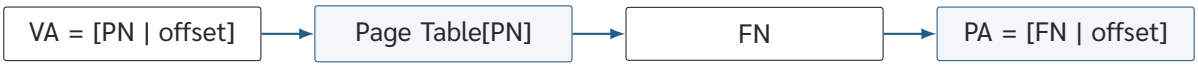
> **จุดที่มักสับสน**
>
> **Page ต่างจาก Frame** Page เป็นหน่วยใน logical/virtual address space ส่วน Frame เป็นช่องขนาดเท่ากันใน physical memory Page Table ใช้บอกว่าแต่ละ page อยู่ใน frame ใด

5.4 Page Table และการแปล Address

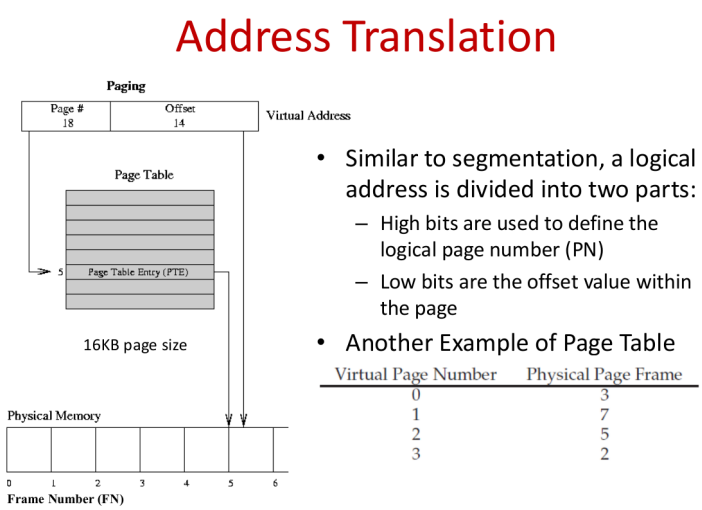
แบ่ง logical address ออกเป็นส่วนต่าง ๆ ดังนี้:

- high bits แทน **Page Number (PN/VPN)**
- low bits แทน **Page Offset**

ใช้ PN ค้นหา Frame Number (FN/PFN) ใน Page Table แล้วประกอบ physical address จาก frame number และ offset เดิม ค่า offset ไม่เปลี่ยน เพราะ page และ frame มีขนาดเท่ากัน



ภาพประกอบ 5.3 Lecture 5 หน้า 15 — Paging Address Translation



จุดสังเกต

- แยก logical address เป็น **Page Number** + **Offset**
- ใช้ Page Number เป็น index ค้น PFN ใน Page Table ส่วน offset คงเดิม
- ประกอบ Physical Address จาก PFN และ offset เดิม

ที่มา: Lecture 5 หน้า 15.

5.4.1 คำนวณจำนวน Bits ของ Paging

Lecture หน้า 16 ใช้ logical address ขนาด 32 bits โดย 20 bits ด้านสูงเป็น page number และ 12 bits ด้านต่ำเป็น offset ดังนั้น page size คือ $2^{12} = 4096$ B หรือ 4KB

ตัวอย่างคำนวณ

ตัวอย่างฝึกทำเพิ่มเติม ตัวอย่างนี้สร้างขึ้นเพื่อฝึกวิธีคิด ไม่ใช่ค่าจาก Lecture กำหนด VA ขนาด 32 bits, page size 4KB และ $VA = 0\text{x}12345$ โดยสมมติว่า Page Table จับคู่ $PN = 0\text{x}12$ กับ frame 7

1. จำนวน offset bits คือ $\log_2 4096 = 12$ bits
2. คำนวณ $PN = \lfloor VA/4096 \rfloor = 0x12 = 18$
3. คำนวณ $offset = VA \bmod 4096 = 0x345 = 837$
4. ค้น Page Table แล้วได้ PFN=7
5. ประกอบเป็น $PA = 7 \times 4096 + 837 = 29509 = 0\text{x}7345$

ให้ทำตามลำดับนี้: **แยก address field ค้น mapping คง offset เดิม แล้วประกอบ physical address**

5.5 Page Table Size Problem

Lecture หน้า 16 สมมติว่าแต่ละ PTE ใช้ 32 bits เก็บ frame number และอีก 1 valid bit รวมเป็น 33 bits ต่อ entry เมื่อมี 2^{20} PTEs:

$$2^{20} \times 33 = 34{,}603{,}008 \text{ bits}$$

แปลงเป็น byte โดยหาร 8:

$$\frac{34{,}603{,}008}{8} = 4{,}325{,}376 \text{ bytes} \approx 4.125 \text{ MiB} \approx 4.33 \text{ MB}$$

สไลด์ย้ำว่าขนาดนี้เป็นของ process เดียว จึงต้องหาโครงสร้างที่ลดพื้นที่ของ page table โดยเก็บ page table ไว้ใน kernel memory และใช้ Page Table Base Register กับ Page Table Size Register ระบุตำแหน่งและขนาด

ภาพประกอบ 5.4 Lecture 5 หน้า 16 — How Large is the Page Table?

How Large is the Page Table?

- Consider a 32 bit logical address where the 20 MSBs define page number and the 12 LSBs define page offset (4KB page)
- Assume 32 bits to keep all the FNs (frame number) in physical memory
- Use 1 bit for a valid bit (space used)
- So, 2^{20} is the number of PTEs and the size of the page table is 2^{20} * 33 (~34 MByte)
- This is just one process!
- As the tables are so large, they are held in kernel memory – a Page Table Base Register contains the physical address of the start of the page table and a Page Table Size Register its size

	V	Physical Page
0		32 bits
1		
2		
	⋮	⋮
$2^{20}-1$		

100 process → 3GB
1000 process → 34 GB

16

จุดสังเกต

- เมื่อ VA มี 32 bits และ page size 4KB จะใช้ 12 bits เป็น offset และ 20 bits เป็น page number จึงมี 2^{20} PTEs ต่อ process
- $2^{20} \times 33$ ได้หน่วยเป็น bit; เมื่อหาร 8 จะได้ประมาณ 4.125 MiB ต่อ process
- ประเด็นของตัวอย่างคือ **linear page table ใช้พื้นที่มาก** เมื่อ address space มีขนาดใหญ่

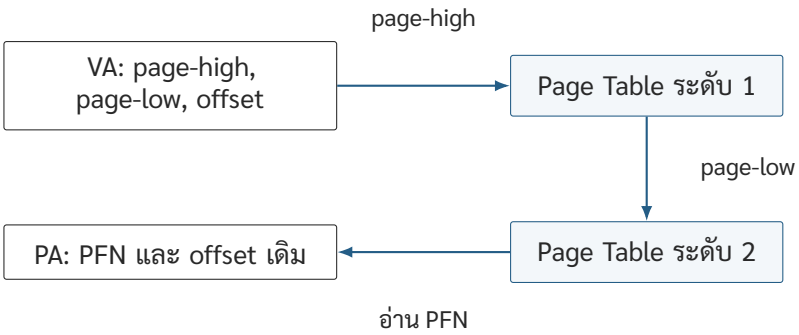
ที่มา: Lecture 5 หน้า 16; หนังสือคำนวณหน่วย bit-to-byte เพิ่มจากนิพจน์บนสไลด์

จุดที่มักสับสน

ตรวจหน่วยกับตัวเลขที่สไลด์พิมพ์ สไลด์ระบุ ~ 34 MByte แต่ค่าที่คูณได้คือ 34,603,008 **bits**; เมื่อหาร 8 จะได้ 4,325,376 bytes หรือประมาณ 4.125 MiB. ตัวเลข 34 MByte บนสไลด์จึงไม่สอดคล้องกับหน่วยของนิพจน์ที่พิมพ์ไว้

5.6 Multi-Level Page Table

Two-Level Paging แบ่ง page table ออกเป็นหลายระดับ โดยใช้ส่วน high ของ Page Number ค้น first-level table และส่วน low ค้น second-level table ไม่จำเป็นต้องสร้าง second-level table ทุกส่วนไว้พร้อมกัน จึงประหยัด memory ในบริเวณของ address space ที่ยังไม่ได้ใช้ โดยทั่วไปออกแบบให้ second-level page table หนึ่งชุดมีขนาดพอดีกับหนึ่ง page



ข้อแลกเปลี่ยนคือใช้ memory เก็บตารางน้อยลง แต่การแปล address อาจต้องอ่าน memory หลายครั้งหากไม่พบข้อมูลใน TLB

5.7 Inverted Page Table

Lecture ตั้งข้อสังเกตว่า frames มักมีจำนวนน้อยกว่า virtual pages **Inverted Page Table** จึงมีหนึ่ง entry ต่อ physical frame โดยเก็บ logical page number และ PID ของ process เจ้าของ page เมื่อต้องการแปล address ระบบค้น entry ที่ตรงกับ (PID, logical page) แล้วใช้ index ของ entry นั้นเป็น physical frame number

วิธีนี้ช่วยลดขนาด table แต่ใช้เวลาแปล address มากขึ้นเพราะต้องค้นหา จึงเป็นตัวอย่างของ time-space trade-off

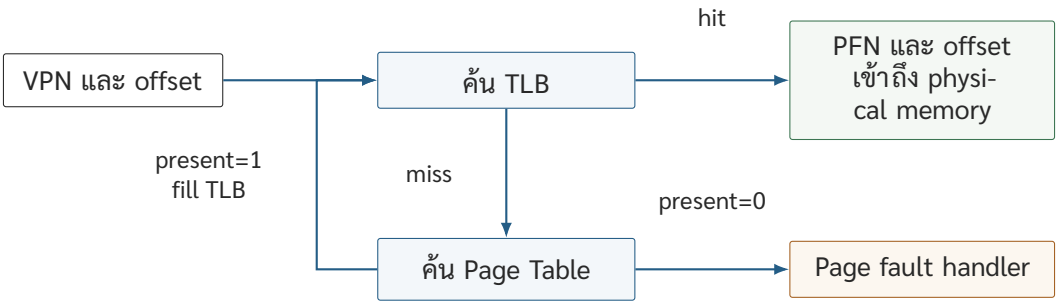
5.8 Hashed Page Table

ระบบ hash logical page number ไปยัง bucket หนึ่ง แต่ละ bucket เชื่อมรายการ entry ซึ่งเก็บ logical page number, mapped frame และ pointer ไปยัง entry ถัดไป ระบบจึงค้นเฉพาะ chain ที่เกี่ยวข้อง และไม่ต้องมี entry สำหรับทุก logical page Lecture ระบุว่าวิธีนี้พบได้ใน address space ที่ใหญ่กว่า 32 bits

5.9 Translation Lookaside Buffer (TLB)

เนื่องจาก Page Table อยู่ใน main memory การใช้ linear table จึงเพิ่ม memory lookup ให้กับ logical address ทุกครั้ง แม้ multi-level table จะลดขนาดตาราง แต่ก็อาจเพิ่มจำนวน lookup Lecture จึงแนะนำ **Translation Lookaside Buffer (TLB)** ซึ่งเป็น hardware cache ภายใน MMU

> **เนื้อหาจาก Lecture**
>
> TLB เป็น hardware cache ที่เก็บ translation ระหว่าง logical page กับ physical frame แบบ **fully associative** และอาศัย locality of reference Hardware จะค้น TLB ก่อนเข้า Page Table:
> - หาก TLB hit ระบบจะได้ translation ทันที
> - หาก TLB miss ระบบจะไปค้น Page Table แล้วนำ translation ที่พบมาเก็บใน TLB
>
> ตามแบบจำลองใน Lecture TLB เก็บ translation ของ process ปัจจุบัน เมื่อเกิด context switch ระบบจึงล้าง entries หรือทำเครื่องหมายว่าใช้ไม่ได้



เมื่อ TLB เต็ม Lecture ระบุว่าสามารถแทน entry ที่ไม่ได้ใช้นานที่สุดด้วย Least-Recently-Used (LRU) ได้

> **เชื่อมกับ Computer Architecture**
>
> TLB ใช้แนวคิด cache/locality และ associative lookup จาก Computer Architecture แต่ **TLB ไม่ใช่ data cache**: มัน cache address translations. TLB hit/miss บอกว่ามี translation หรือไม่ ไม่ได้บอกว่า page อยู่ RAM หรือ disk — ประเด็น present/page fault จะมาใน Lecture 6.

> **จุดที่มักสับสน**
>
> **TLB miss ต่างจาก Page Fault.** TLB miss สามารถไป Page Table แล้วพบ valid/present mapping ใน memory ได้ตามปกติ Page Fault เป็นคนละ condition ที่ Virtual Memory ตรวจว่า page ที่ต้องใช้ไม่ได้ resident ตาม state ของ PTE.

ภาพประกอบ 5.5 Lecture 5 หน้า 22 — Translation Lookaside Buffer (TLB)

Translation Lookaside Buffer

- The TLB is simply a hardware cache of popular logical page number to physical page frame translations
  - Fully associative – hardware will search entire TLB in parallel
- Based on idea of locality of reference
- The hardware looks in TLB first
  - If it has the required page, the translation is very quick
  - Otherwise, the page table is accessed in the normal way and the translation stored in the TLB
- The TLB contains address translations for the current process
  - The TLB is flushed (entries marked invalid) on a context switch

22

จุดสังเกต

- TLB cache **address translations** ไม่ใช่ cache ของ data bytes
- TLB hit ลดการไปอ่าน Page Table ใน main memory
- TLB miss ยังอาจมี page resident อยู่ใน RAM; ดังนั้น **TLB miss ต่างจาก Page Fault**

ที่มา: Lecture 5 หน้า 22.

5.10 Page Size Trade-offs

ตาม Lecture:

	Page ขนาดเล็ก	Page ขนาดใหญ่
Fragmentation	ลด internal fragmentation	internal fragmentation อาจมากขึ้น
Working set / memory	อาจใช้ memory เก็บ working set น้อยลง เหลือให้ process อื่นมากขึ้น	ต้องโหลดและจัดการข้อมูลเป็นก้อนใหญ่ขึ้น
Page table	มี pages มากขึ้น จึงต้องมี entries มากขึ้น	จำนวน entries และ table overhead ลดลง
TLB reach	แต่ละ entry ครอบคลุมพื้นที่เล็ก	แต่ละ TLB entry ครอบคลุมพื้นที่มากขึ้น
I/O	อาจต้องทำ I/O หลายครั้ง	อาจลดจำนวน I/O ที่ใช้โหลด working set

ภาพประกอบ 5.6 Lecture 5 หน้า 25 — Page Size Trade-off

Page Size

- Small Page Size
  - Reduces internal fragmentation
  - Can reduce the amount of memory required to contain a process's working set
  - More memory is available to other processes
- Large Page Size
  - Reduces wasted memory from table fragmentation
  - Enables each TLB entry to map a larger region of memory, improving performance
  - Reduces the number of I/O operations the system performs to load a process's working set into memory

25

จุดสังเกต

- page ขนาดเล็กช่วยลด internal fragmentation และจัด working set ได้ละเอียดขึ้น แต่ทำให้ Page Table ใหญ่ขึ้น
- page ขนาดใหญ่ทำให้ TLB entry ครอบคลุม memory ได้มากขึ้น และอาจลดจำนวน I/O ที่ต้องใช้โหลด working set
- ไม่มี page size ที่ชนะทุก metric; นี่คือ design trade-off ไม่ใช่คำตอบ ยิ่งใหญ่ยิ่งดี

ที่มา: Lecture 5 หน้า 25 หมายเหตุเปรียบเทียบ internal fragmentation กับขนาด page table

5.11 เปรียบเทียบ Segmentation กับ Paging

ประเด็น	Segmentation	Paging
ขนาดหน่วย	segment มีขนาดไม่เท่ากัน	page/frame มีขนาดเท่ากัน
ความหมาย	แบ่งตามส่วนของงาน เช่น code/heap/stack	แบ่งเป็นช่วงขนาดคงที่
External fragmentation	เกิดได้	ไม่เกิดตามแบบจำลองใน Lecture
Internal fragmentation	ไม่เกิดตามการเปรียบเทียบใน Lecture	เกิดได้ โดยเฉพาะเมื่อ page ใหญ่
Metadata	ใช้ segment table ขนาดเล็กกว่า	page table อาจมีขนาดใหญ่
การจัดการ	จัดสรรหน่วยต่างขนาดได้ยากกว่า	จัดการ free frame ได้ง่ายและยืดหยุ่นกว่า

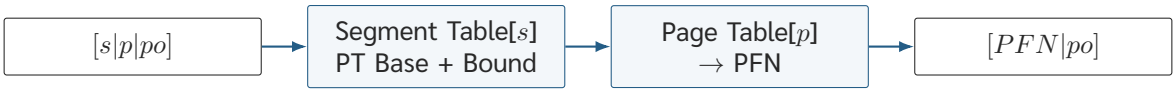
5.12 Hybrid Segmentation + Paging

Hybrid scheme แบ่ง address space เป็น logical segments แล้วแบ่งแต่ละ segment เป็น pages อีกชั้นหนึ่ง แทนที่จะมี page table ใหญ่เพียงชุดเดียว ระบบมี page table แยกสำหรับแต่ละ segment โดย Base เก็บ physical address ของ page table และ Bounds ระบุขนาดของตารางนั้น

Logical address แบ่งสามส่วน:

$$[\ Segment\ s\ |\ Page\ p\ |\ Page\ Offset\ po\ ]$$

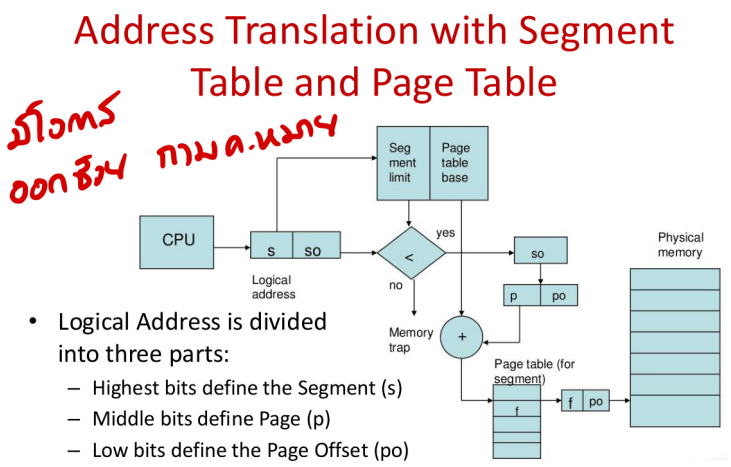
แต่ละ process มี segment table หนึ่งชุด และมี page tables หลายชุดตามจำนวน segments



ข้อดีของวิธีนี้คือใช้ memory เก็บตารางน้อยกว่า linear page table เพราะช่องว่างที่ยังไม่จัดสรร เช่น ระหว่าง heap กับ stack ไม่ต้องมี page-table entry และยังใช้ TLB ได้ ข้อจำกัดคือ page tables ของแต่ละ segment มีขนาดไม่เท่ากัน จึงยังเกิด external fragmentation ได้ นอกจากนี้ Segmentation ยังตั้งอยู่บนสมมติฐานบางอย่างเกี่ยวกับรูปแบบการใช้ address space

> **อ่านภาพและหมายเหตุ**
>
> ข้อความเขียนมือสีแดงบางส่วนในสไลด์หน้า 27 และ 29 อ่านได้ไม่ชัด จึงไม่ถอดความส่วนที่ไม่แน่ใจ เนื้อหาในบทนี้อ้างเฉพาะข้อความที่ตรวจสอบได้จากต้นฉบับ

ภาพประกอบ 5.7 Lecture 5 หน้า 29 — Hybrid Segmentation + Paging Translation



จุดสังเกต

- แบ่ง address เป็นสามส่วน: segment, page ภายใน segment และ page offset
- Segment Table ชี้ไปยัง Page Table ของ segment นั้น แล้ว Page Table แปลง page เป็น frame
- วิธีนี้รวมการแบ่งตามความหมายของ Segmentation กับ page ขนาดคงที่ แต่มี lookup และ metadata ซับซ้อนขึ้น

ที่มา: Lecture 5 หน้า 29 ข้อความเขียนมือสีแดงบางส่วนอ่านไม่ชัด จึงถอดเฉพาะส่วนที่ตรวจสอบได้

5.13 Chapter Summary

สรุปท้ายบท

- Segmentation แบ่ง logical regions ขนาดไม่เท่ากันและใช้ Base/Bounds ต่อ segment จึงไม่ต้องจัดสรรช่องว่างใน address space แต่ยังเกิด external fragmentation
- Paging แบ่ง pages และ frames ให้มีขนาดเท่ากัน แต่ละ process มี Page Table สำหรับจับคู่ page กับ frame จัดการ free space ได้ง่ายและไม่เกิด external fragmentation แต่อาจเกิด internal fragmentation และมี page-table overhead
- Address translation ของ paging คง offset เดิมและแปลง Page Number เป็น Frame Number
- Multi-level, Inverted และ Hashed Page Table ลดพื้นที่ metadata ด้วยข้อแลกเปลี่ยนต่างกัน
- TLB เป็น fully-associative hardware cache สำหรับเก็บ translations และอาศัย locality ในแบบจำลองของ Lecture จะ flush เมื่อ context switch
- การเลือก page size มีผลต่อ fragmentation, page-table size, TLB reach และ I/O overhead
- Hybrid scheme ใช้ segment table ร่วมกับ page table ของแต่ละ segment เพื่อประหยัดพื้นที่ตาราง แต่ยังมีข้อจำกัดของ Segmentation

กลไกสำคัญ

MMU translation, การค้น segment/page table, Base/Bounds check, TLB associative lookup และผลของ context switch

นโยบายสำคัญ

First/Best/Worst Fit สำหรับ segments, TLB replacement (แสดง LRU ใน Lecture) และการเลือก page size

Formula / Calculation Recap

Segmentation: $PA = Base_s + offset$ after bounds check.
Paging: $PN = \lfloor VA/PageSize \rfloor$, $offset = VA$ mod $PageSize$, $PA = PFN \times PageSize + offset$.

ComArch Connections

MMU, physical address, associative lookup, locality, hardware cache concept, register/state changes on context switch.

Common Confusions

- Segment ต่างจาก Page; unequal/meaningful vs fixed-size
- Page ต่างจาก Frame
- Page Table ต่างจาก TLB
- TLB miss ต่างจาก Page Fault
- Paging removes external fragmentation but can create internal fragmentation
- Inverted Page Table มี entry ต่อ physical frame ไม่ใช่ต่อ virtual page

Quick Revision Checklist

☐ ทำ segment+offset translation และ bounds check ได้
☐ ทำ page-number/offset bit split และ physical reconstruction ได้
☐ อธิบาย linear vs 2-level vs inverted vs hashed page table ได้
☐ trace TLB hit และ TLB miss path ได้
☐ เปรียบเทียบ Segmentation vs Paging แบบ fragmentation/metadata/management ได้
☐ trace Hybrid $[s|p|po]$ ผ่าน segment table และ page table ได้

Lecture 6 — Virtual Memory

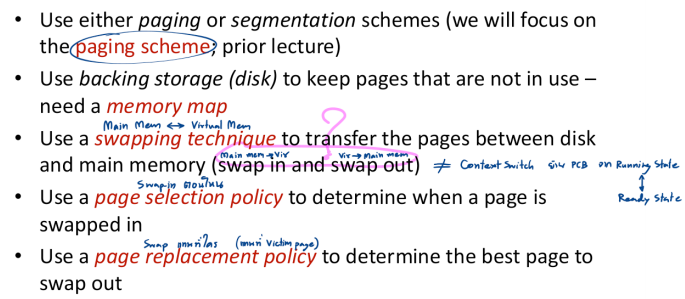
อ้างอิง: Lecture 6 หน้า 1–34 | backing store, page fault, replacement, EAT, thrashing และ working set

6.1 เหตุผลที่ใช้ Virtual Memory: Address Space ใหญ่กว่า Physical Memory

ก่อนหน้านี้สมมติว่า address space ของ process ที่กำลังทำงานต้องอยู่ใน physical memory ทั้งหมด Lecture 6 ผ่อนสมมติฐานนี้ โดยอนุญาตให้ application มีขนาดใหญ่กว่า RAM ได้ **Virtual Memory** ย้าย page ที่ยังไม่ต้องใช้ไปเก็บใน backing storage และนำกลับมาเมื่อจำเป็น

> **เนื้อหาจาก Lecture**
>
> แนวคิด Virtual Memory ใน Lecture ใช้ paging เป็นหลัก และมีองค์ประกอบดังนี้:
>
> - backing storage เช่น disk ใช้เก็บ pages ที่ไม่ได้อยู่ใน main memory
> - swapping ย้าย page ระหว่าง disk กับ memory
> - **page selection policy** กำหนดว่าเมื่อใดควรนำ page เข้ามา
> - **page replacement policy** เลือกว่า page ใดควรถูกนำออกเมื่อ memory เต็ม

ภาพประกอบ 6.1 Lecture 6 หน้า 4 — Virtual Memory Concept

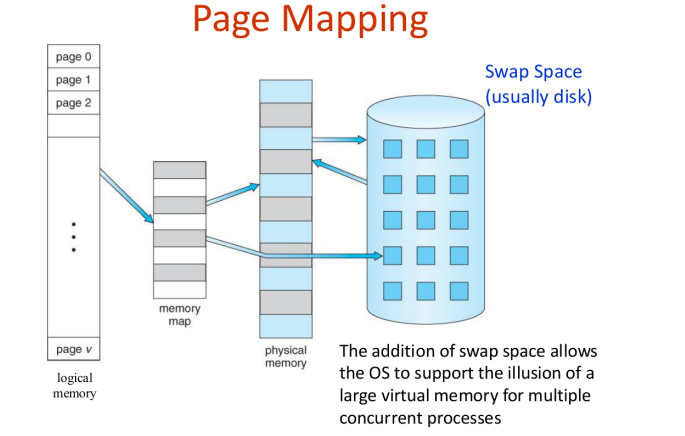


จุดสังเกต

- ไม่จำเป็นต้องเก็บทุก page ไว้ใน RAM ตลอดเวลา page ที่ยังไม่ใช้เก็บไว้ใน backing storage ได้
- **Page selection** ตัดสินใจว่าเมื่อใดควรนำ page เข้ามา
- **Page replacement** เลือกว่า page ใดควรนำออกเมื่อ memory เต็ม

ที่มา: Lecture 6 หน้า 4 พร้อมหมายเหตุเรื่อง swap และ context จากสไลด์

ภาพประกอบ 6.2 Lecture 6 หน้า 5 — Page Mapping + Swap Space



จุดสังเกต

- logical pages ไม่จำเป็นต้องอยู่ RAM ทุกหน้าในเวลาเดียวกัน
- memory map/page metadata บอกว่า page resident ใน physical memory หรือเก็บอยู่ backing store
- swap space ทำให้ OS สร้าง illusion ของ virtual memory ที่ใหญ่กว่า physical RAM ให้หลาย process พร้อมกัน

ที่มา: Lecture 6 หน้า 5.

6.2 Process Switching ต่างจาก Process Swapping

	การสลับ process / Context Switch	การ swap process
สิ่งที่ย้าย	CPU context และ register state ระหว่าง process	เนื้อหา process/page ระหว่าง main memory กับ backing storage
กลไกหลัก	scheduler และ CPU state	memory manager และ disk I/O
เวลา/ต้นทุน	บันทึกและคืนค่า state	ช้ากว่ามากเพราะต้องทำ disk I/O
ผลที่เกิดขึ้น	process อื่นได้ใช้ CPU	เปลี่ยน ว่า address-space content อยู่ ใน memory หรือไม่

Lecture หน้า 8 ย้ำว่า process เริ่มทำงานได้แม้โหลดมาเพียงบาง pages แต่ถ้า page ที่ต้องใช้ถูก swap ออกไปหมด process จะทำงานต่อไม่ได้จนกว่าจะนำ page กลับเข้า memory

> **จุดที่มักสับสน**
>
> **Context Switch ต่างจาก Swap** การสลับ process เปลี่ยนว่าใครได้ใช้ CPU ส่วนการ swap ย้ายข้อมูลระหว่าง memory กับ disk ทั้งสองอย่างอาจเกิดใน flow เดียวกันระหว่าง page fault แต่เป็นคนละกลไก

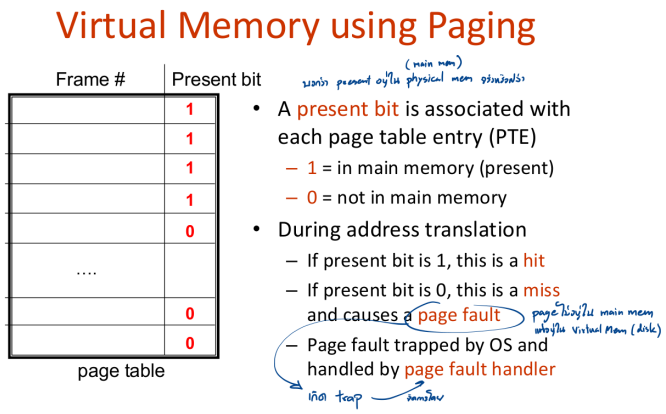
6.3 Present Bit และ Page Fault

PTE มี **present bit**:

- 1 หมายถึง page อยู่ใน main memory จึงเข้าถึงได้ตามปกติ
- 0 หมายถึง page ไม่อยู่ใน main memory การอ้างอิง page นี้จึงทำให้เกิด **Page Fault**

Page Fault ถูก trap เข้า OS และ Page Fault Handler จัดการ

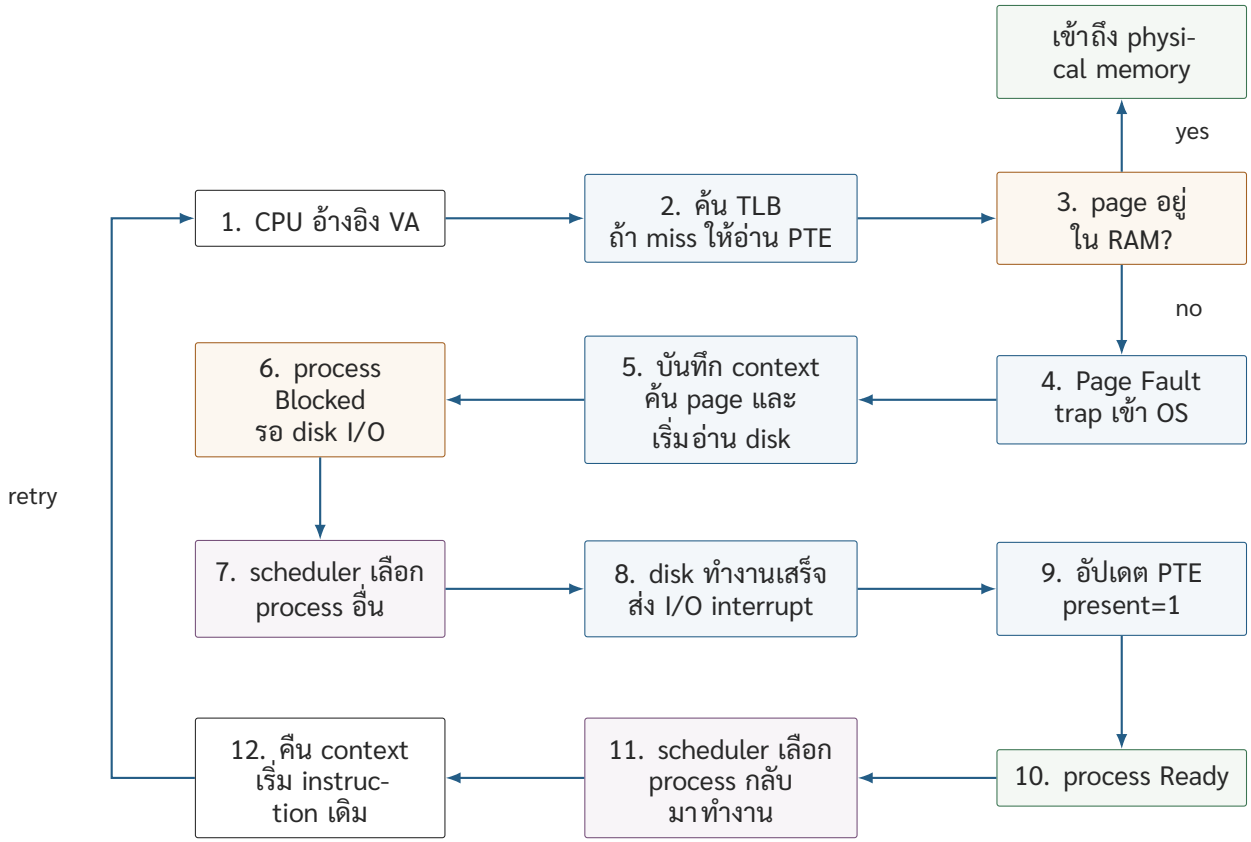
ภาพประกอบ 6.3 Lecture 6 หน้า 9 — Present Bit: Hit vs Page Fault



> **จุดสังเกต**
>
> - present=1 หมายถึง page อยู่ใน main memory และ PTE มี frame number ที่ใช้ได้
> - present=0 หมายถึง page ไม่อยู่ใน RAM การอ้างอิงครั้งนี้จึงทำให้เกิด **Page Fault** และ trap เข้า OS handler
> - TLB miss ต่างจาก Page Fault เพราะระบบอาจพบ PTE ที่ present=1 หลังค้น Page Table

ที่มา: Lecture 6 หน้า 9 พร้อมหมายเหตุเรื่อง present bit และ page fault

6.3.1 End-to-End Page Fault Flow



เส้นทางด้านบนคือ page อยู่ใน RAM และเข้าถึงต่อได้ ส่วนเส้นทางด้านล่างแสดงลำดับเมื่อเกิด page fault จน OS นำ page กลับมาและเริ่ม instruction เดิมอีกครั้ง

Lecture หน้า 10–11 ให้ sequence โดยละเอียด:

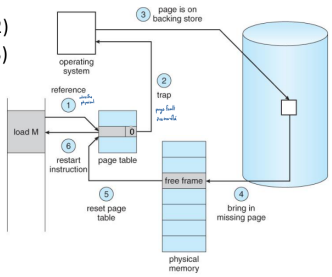
1. memory reference พบ present bit=0 จึงเกิด page fault trap
2. บันทึก process context
3. ตรวจตำแหน่ง page บน disk โดย PTE อาจใช้เก็บ disk address แทน PFN ตามแบบจำลองในสไลด์
4. เริ่มอ่าน page จาก disk ไปยัง free frame
5. นำ process เข้าคิวรออุปกรณ์ แล้วให้ CPU ทำงานอื่นระหว่างรอ
6. เมื่อ disk ทำงานเสร็จ จะส่ง I/O completion interrupt
7. อัปเดต Page Table/PTE ให้ชี้ PFN และกำหนด present=1
8. เมื่อ process ได้ CPU อีกครั้ง ให้คืน context และ page-table registers แล้วเริ่ม instruction ที่หยุดไว้ใหม่

> **เชื่อมกับ Computer Architecture**
>
> ขั้นตอนนี้เชื่อมเนื้อหาทั้งวิชาเข้าด้วยกัน เมื่อ CPU อ้างอิง address กลไกแปลที่อยู่จะตรวจว่า page อยู่ใน RAM หรือไม่ หากไม่อยู่จะเกิด trap เข้า kernel เพื่อเริ่ม disk I/O และให้ process รอในสถานะ Blocked ระหว่างนั้น scheduler เลือก process อื่นมารัน เมื่อ I/O เสร็จ hardware ส่ง interrupt ให้ OS อัปเดต PTE และเปลี่ยน process กลับเป็น Ready หลังได้รับ CPU อีกครั้งจึงคืน context และเริ่ม instruction เดิมใหม่
> Lecture ด้าน I/O ช่วยอธิบาย IRQ และ ISR ส่วน Lecture ด้าน Processing Unit ช่วยอธิบายว่าทำไมต้องบันทึก PC และ register state ให้ถูกต้องก่อนกลับมาทำงานต่อ

ภาพประกอบ 6.4 Lecture 6 หน้า 10 — Page Fault

Page Fault

- On a memory reference (1), if the present bit in the PTE for the logical page is 0, there is a page fault that is trapped by the OS (2)
- The OS finds the page needed (3)
  - The bits in the PTE normally used for the PFN can instead hold the disk address
- The OS brings the page into physical memory (4)
- The PTE is reset (5) to the PFN that now contains the page and the present bit set to 1
- The instruction is restarted (6)



10

จุดสังเกต

- PTE present bit = 0 ทำให้ memory reference เดิมเดินต่อไม่ได้
- hardware trap เข้า OS; handler ต้องหา page บน disk และเริ่ม I/O
- หลัง page กลับมาและ PTE ถูกอัปเดต instruction เดิมจึง restart — ไม่ได้ข้าม instruction ที่ fault

ที่มา: Lecture 6 หน้า 10 หมายเลขในภาพเรียงตามลำดับของ Lecture

ภาพประกอบ 6.5 Lecture 6 หน้า 11 — Page Fault Handler

Page Fault Handler

- Save the process context (registers, process state, etc.)
- Determine the location of the page on the disk
- Issue a read request from the disk to a free frame
- Put the process on a wait queue for this device until the read request is serviced
  - While the process is waiting, allocate the CPU to some other user
- Receive an interrupt from the disk I/O subsystem (I/O completed)
- Reset the page table to show the page is now in memory
- When the CPU is allocated to this process again
  - Save the context for the current user
  - Restore the context and page table registers for the process that caused the page fault, and then restart the interrupted instruction

11

จุดสังเกต

- บันทึก process context ค้นตำแหน่ง page บน disk แล้วเริ่มอ่านข้อมูล โดยนำ process เข้าคิวรอ
- ระหว่างรอ I/O scheduler ให้ CPU แก่ process อื่น เมื่อ disk ทำงานเสร็จจะเกิด I/O completion interrupt
- อัปเดต Page Table แล้วคืน context เมื่อ process ได้ CPU อีกครั้ง จากนั้น **เริ่ม instruction ที่ทำให้เกิด fault ใหม่**

ที่มา: Lecture 6 หน้า 11 เชื่อม Process State, Scheduling, Disk I/O, Interrupt และ Memory Translation

6.4 Page Selection Policies — เมื่อใดควรเอา page เข้า

Demand Paging

ไม่ swap page เข้า จน page นั้นถูกต้องการจริง; swap in เมื่อ page fault

Request Paging

user process ร้องขอ page ให้ swap in; Lecture เตือนว่า user process ไม่ได้รู้ดีที่สุดเสมอ

Pre-paging / Pre-fetching

swap next page ล่วงหน้าก่อน reference โดยใช้ spatial locality; เสี่ยง wasted work หาก prediction ผิด

6.5 Page Replacement — เมื่อ memory เต็มควรนำ page ใดออก

เมื่อ physical memory เต็มหรือใกล้เต็ม OS ต้องเลือก victim page ที่จะนำออก Lecture ระบุว่า **modify/dirty bit** ช่วยลด overhead ได้ เพราะ page ที่ไม่มีการแก้ไขไม่จำเป็นต้องเขียนกลับ disk

Reference strings:

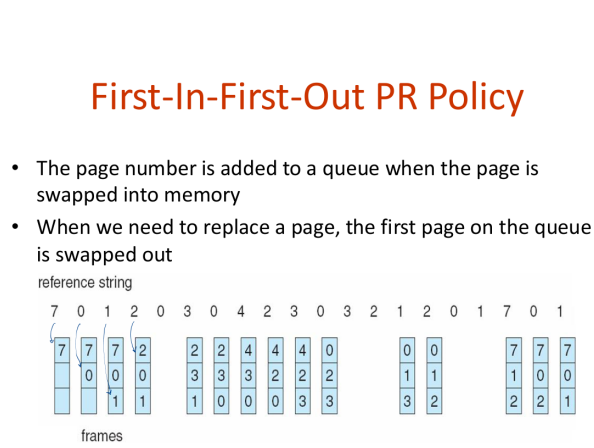
- Example 1: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1
- Example 2: 1,2,3,4,1,2,5,1,2,3,4,5

เริ่มต้นจาก frames ว่าง การอ้างอิงช่วงแรกที่ทำให้ต้องนำ page เข้ามาเติม frame จึงเป็น cold-start หรือ compulsory misses

6.5.1 FIFO

เมื่อ swap page เข้ามา ให้เพิ่ม page นั้นไว้ท้าย queue เมื่อจำเป็นต้อง replace ให้เลือก page ที่อยู่หัว queue ซึ่งเป็น page ที่เข้ามานานที่สุด ในตัวอย่างที่ 1 ของ Lecture มี 3 frames และเกิด **15 page faults** โดยมี hit rate $5/20 = 25\%$

ภาพประกอบ 6.6 Lecture 6 หน้า 15 - FIFO Example 1 แบบแนวนอน



จุดสังเกต

- อ่าน reference string จากซ้ายไปขวา แล้วดู frame stack ใต้แต่ละ reference
- ภาพย่อ state เฉพาะบางจุดเพื่อให้เห็นจังหวะนำ page เข้าและเอา page เก่าสุดออก
- สไลด์สรุป 15 faults และ 5 hits จาก 20 references; ตารางถัดไปแสดงทุก step รวมจังหวะ hit

ที่มา: Lecture 6 หน้า 15.

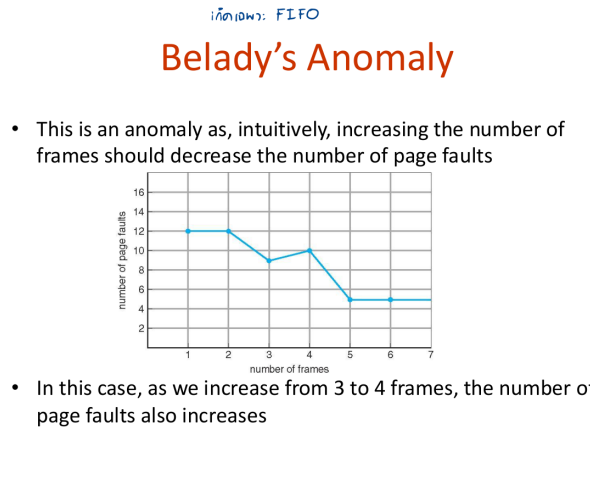
6.5.2 Belady's Anomaly

Lecture Example 2 แสดง FIFO:

- เมื่อมี 3 frames จะเกิด 9 faults
- เมื่อเพิ่มเป็น 4 frames กลับเกิด 10 faults

จึงเป็น anomaly: เพิ่ม frames แต่ faults กลับเพิ่ม

ภาพประกอบ 6.7 Lecture 6 หน้า 16 - กราฟ Belady's Anomaly

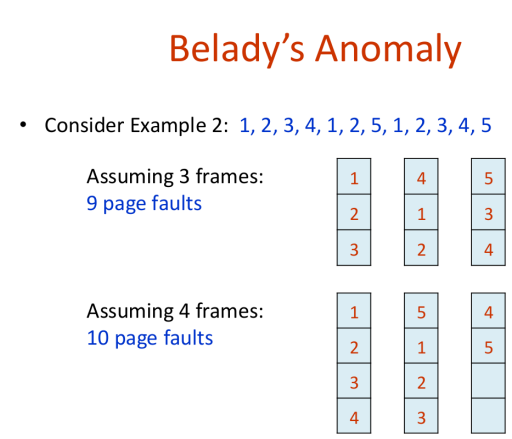


จุดสังเกต

- แกนนอนคือจำนวน frames; แกนตั้งคือจำนวน page faults ของ FIFO
- จุดสำคัญคือจาก 3 frames มี 9 faults ไปเป็น 4 frames มี 10 faults
- กราฟไม่ได้หมายความว่า faults เพิ่มทุกครั้งที่เพิ่ม frame แต่ชี้ว่ามีบางช่วงที่เพิ่มขึ้นได้

ที่มา: Lecture 6 หน้า 16.

ภาพประกอบ 6.8 Lecture 6 หน้า 17 - ตัวอย่าง FIFO ที่เกิด anomaly



จุดสังเกต

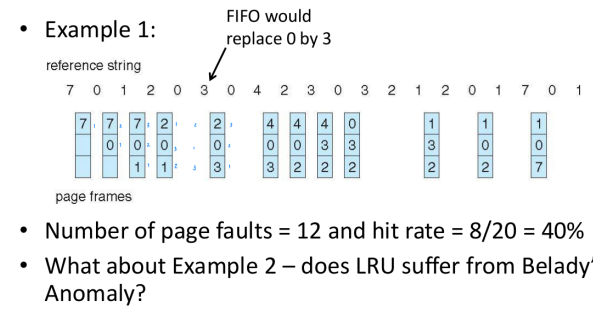
- ใช้ Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- ภาพเทียบ state ของ 3 frames กับ 4 frames และได้ 9 กับ 10 faults ตามลำดับ
- ตัวอย่างนี้แสดงว่าการเพิ่ม frame อาจเปลี่ยนลำดับการแทน page จนเกิด fault มากขึ้น

ที่มา: Lecture 6 หน้า 17.

6.5.3 LRU

Least-Recently-Used (LRU) เลือกแทน page ที่ไม่ได้ถูกใช้มานานที่สุด โดยใช้ประวัติที่ผ่านมาเป็นตัวประมาณว่า page ที่เพิ่งถูกใช้จะมีโอกาสถูกใช้อีก ในตัวอย่างที่ 1 ของ Lecture เมื่อมี 3 frames จะเกิด **12 faults** และมี hit rate $8/20 = 40\%$ ส่วนตัวอย่างที่ 2 เกิด 10 faults เมื่อมี 3 frames และ 8 faults เมื่อมี 4 frames Lecture ระบุว่า LRU มี stack property จึงไม่เกิด Belady's anomaly

ภาพประกอบ 6.9 Lecture 6 หน้า 19 - LRU Example 1

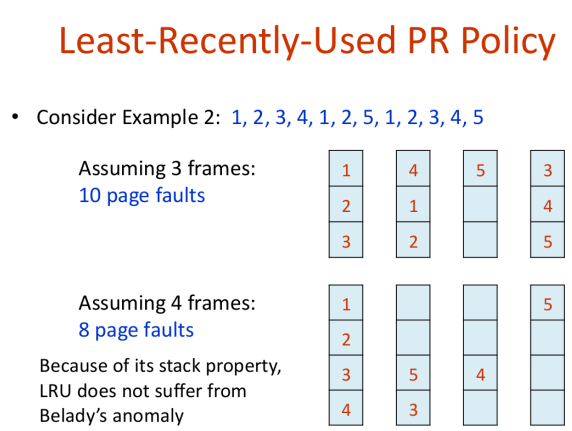


จุดสังเกต

- เมื่ออ้างอิง page 3 ครั้งที่ 6, LRU นำ page 1 ออกเพราะใช้นานที่สุดแล้ว
- หากใช้ FIFO ในจังหวะเดียวกัน จะนำ page 0 ออกตามลำดับเข้า queue
- ภาพจึงเทียบให้เห็นว่า FIFO ดูเวลาที่เข้า ส่วน LRU ดูเวลาที่ใช้ล่าสุด

ที่มา: Lecture 6 หน้า 19; ลูกศรและวงที่เขียนกำกับช่วยชี้จุดตัดสินใจ

ภาพประกอบ 6.10 Lecture 6 หน้า 20 - LRU กับ stack property



จุดสังเกต

- Example 2 ให้ 10 faults เมื่อมี 3 frames และ 8 faults เมื่อมี 4 frames
- จำนวน frames เพิ่มแล้ว faults ไม่เพิ่ม จึงไม่เกิด Belady's anomaly ใน LRU
- stack property อธิบายว่าชุด pages ในระบบที่มี frame มากกว่าจะครอบคลุมชุดที่มี frame น้อยกว่า

ที่มา: Lecture 6 หน้า 20.

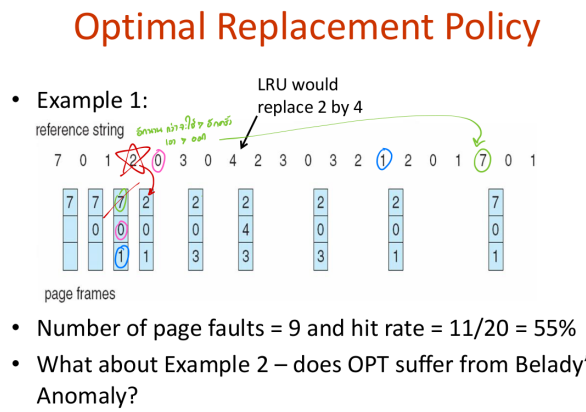
6.5.4 LFU

Least-Frequently-Used (LFU) เก็บ reference count และเลือกแทน page ที่มี count ต่ำที่สุด หาก count เท่ากัน Lecture อนุญาตให้ใช้อีก algorithm เช่น FIFO เป็นกฎตัดสิน ตัวอย่าง 1 และ 2 ในสไลด์ให้ฝึกคำนวณ แต่ไม่ได้พิมพ์ผลลัพธ์สุดท้ายไว้ ตาราง LFU ในหนังสือนี้จึงเป็นตัวอย่างคำนวณเพิ่ม โดยใช้ FIFO ตัดสินเมื่อ count เท่ากัน และระบุผลว่าเป็นค่าที่คำนวณเพิ่ม ไม่ใช่เฉลยที่พิมพ์ใน Lecture

6.5.5 OPT

Optimal Replacement (OPT) เลือกแทน page ที่จะถูกอ้างอิงอีกครั้งช้าที่สุดในอนาคต จึงให้จำนวน faults ต่ำที่สุด แต่ระบบทั่วไปไม่รู้ล่วงหน้าว่าจะอ้างอิงอะไร จึงใช้ OPT เป็น baseline สำหรับเปรียบเทียบ ในตัวอย่างที่ 1 ของ Lecture มี 3 frames และเกิด **9 faults** โดยมี hit rate $11/20 = 55\%$ ส่วนตัวอย่างที่ 2 เกิด 7 faults เมื่อมี 3 frames และ 6 faults เมื่อมี 4 frames OPT ไม่เกิด Belady's anomaly

ภาพประกอบ 6.11 Lecture 6 หน้า 23 - OPT Example 1 มองไปข้างหน้า



จุดสังเกต

- ก่อนอ้างอิง 4 ครั้งที่ 8, frames มี page 2, 0 และ 3
- มอง reference string ต่อไป: 2 จะถูกใช้ครั้งที่ 9, 3 ครั้งที่ 10 และ 0 ครั้งที่ 11
- OPT จึงนำ 0 ออกเพราะจะถูกใช้อีกครั้งช้าที่สุด; LRU จะนำ 2 ออก

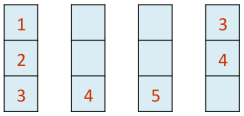
ที่มา: Lecture 6 หน้า 23; วงและลูกศรเขียนกำกับการดู reference ในอนาคต

ภาพประกอบ 6.12 Lecture 6 หน้า 24 - OPT กับจำนวน frames

Optimal Replacement Policy

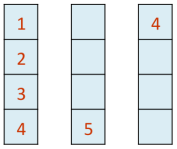
- Consider Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Assuming 3 frames:
7 page faults



Assuming 4 frames:
6 page faults

Because it is optimal, OPT does not suffer from Belady's anomaly



24

จุดสังเกต

- Example 2 ได้ 7 faults เมื่อมี 3 frames และ 6 faults เมื่อมี 4 frames
- OPT ใช้ผลในอนาคตเป็น benchmark จึงไม่เกิด Belady's anomaly
- เทียบตารางสรุปด้านล่างเพื่อดูผลของ FIFO, LRU, LFU และ OPT ในสองตัวอย่าง

ที่มา: Lecture 6 หน้า 24.

6.5.6 Worked trace tables - Example 1, 3 frames

> **เนื้อหาจาก Lecture**
>
> อ่านทีละคอลัมน์จากซ้ายไปขวาตามลำดับ reference: Frame 1-3 แสดง page ที่อยู่ใน memory หลังอ้างอิงแต่ละครั้ง, F หมายถึง page fault, hit หมายถึงพบ page ใน memory และ Victim ระบุ page ที่ถูกนำออก

FIFO - Example 1, 3 frames

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
Frame 2	–	0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
Frame 3	–	–	1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
Fault?	F	F	F	F	hit	F	F	F	F	F	F	hit	hit	F	F	hit	hit	F	F	F
Victim	–	–	–	7	–	0	1	2	3	0	4	–	–	2	3	–	–	0	1	2

Faults = 15; hits = 5/20 = 25%

LRU - Example 1, 3 frames

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
Frame 2	–	0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
Frame 3	–	–	1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
Fault?	F	F	F	F	hit	F	hit	F	F	F	F	hit	hit	F	hit	F	hit	F	hit	hit
Victim	–	–	–	7	–	1	–	2	3	0	4	–	–	0	–	3	–	2	–	–

Faults = 12; hits = 8/20 = 40%

LFU - Example 1, 3 frames

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	3	3	3	3	3	3	3	3	3	3	3
Frame 2	–	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Frame 3	–	–	1	1	1	3	3	3	2	2	2	2	2	1	2	2	1	7	7	1
Fault?	F	F	F	F	hit	F	hit	F	F	F	hit	hit	hit	F	F	hit	F	F	hit	F
Victim	–	–	–	7	–	1	–	2	3	4	–	–	–	2	1	–	2	1	–	7

Faults = 13; hits = 7/20 = 35%

OPT - Example 1, 3 frames

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
Frame 2	–	0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
Frame 3	–	–	1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
Fault?	F	F	F	F	hit	F	hit	F	hit	hit	F	hit	hit	F	hit	hit	hit	F	hit	hit
Victim	–	–	–	7	–	1	–	0	–	–	4	–	–	3	–	–	–	2	–	–

Faults = 9; hits = 11/20 = 55%

6.5.7 เปรียบเทียบผลลัพธ์

นโยบาย	ตัวอย่าง 1 3 frames	ตัวอย่าง 1 4 frames	ตัวอย่าง 2 3 frames	ตัวอย่าง 2 4 frames
FIFO	15	10	9	10
LRU	12	8	10	8
LFU (FIFO tie, derived)	13	9	10	8
OPT	9	8	7	6

ตัวเลข FIFO, LRU และ OPT ในตารางอ้างอิงจากผลที่พิมพ์ไว้ใน Lecture ส่วนแถว LFU และผลบางกรณีที่ไม่ได้ระบุในสไลด์คำนวณเพิ่มเพื่อฝึกใช้ algorithm โดยทำเครื่องหมายไว้ชัดเจน

6.6 LRU Approximation และ Second Chance

Exact LRU มีต้นทุนในการติดตามลำดับ Lecture ใช้ภาพของ double-linked list อธิบายว่าเมื่อต้องย้าย page ที่ถูกอ้างอิงไปไว้ต้นรายการ ระบบอาจต้องปรับ pointer หลายตัว วิธีประมาณค่าแบบหนึ่งใช้ **reference bit**:

- เริ่ม 0
- เมื่อมีการอ้างอิง page ให้กำหนด bit=1
- clear เป็น 0 เป็นระยะ
- หา victim จาก page ที่ bit=0

Reference bit บอกเพียงว่า page ถูกอ้างอิงในช่วงหลังหรือไม่ ไม่ได้เก็บลำดับการใช้งานที่แน่นอน และอาจใช้ hardware ช่วยให้ทำงานเร็วขึ้น

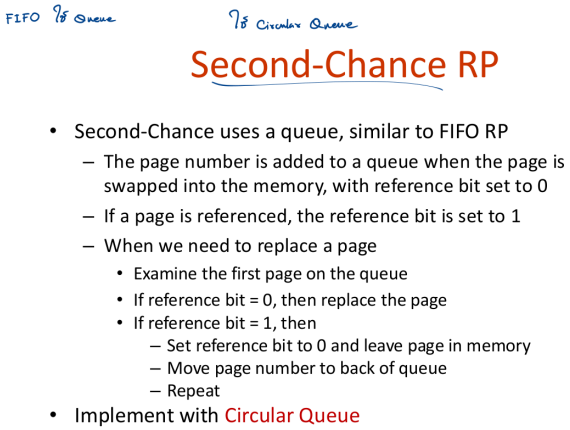
6.6.1 Second-Chance

ใช้ queue คล้าย FIFO แต่ให้โอกาส page ที่ reference bit=1:

1. เมื่อ page เข้า memory ให้เพิ่มไว้ใน queue และกำหนด ref=0
2. เมื่อมีการใช้ page ให้กำหนด ref=1
3. เมื่อต้อง replace ให้ตรวจ page ที่หัว queue
4. หาก ref=0 ให้เลือก page นั้นเป็น victim
5. หาก ref=1 ให้ reset เป็น 0 ย้าย page ไปท้าย queue แล้วตรวจ page ถัดไป

Lecture แสดง **Circular Queue / clock hand** เป็นวิธีเลื่อน pointer วนรอบ queue แทนการเปลี่ยน links ของรายการ

ภาพประกอบ 6.13 Lecture 6 หน้า 26 — Second-Chance Replacement



จุดสังเกต

- เริ่มจาก FIFO/circular queue order
- ถ้า reference bit = 1 ให้ reset แล้ว skip = ให้โอกาสอีกครั้ง
- เจอ ref=0 จึงเลือก victim; เป็น approximation ของ recency ไม่ใช่ exact LRU

ที่มา: Lecture 6 หน้า 26 พร้อมหมายเหตุเรื่อง circular queue จากสไลด์

6.7 Other Replacement Policies

Lecture ระบุชื่อ MFU, NRU, Far Page, Random และอื่น ๆ แต่ไม่ได้ลง algorithm ละเอียดในชุดสไลด์นี้ ดังนั้นหนังสือนี้ไม่เติม behavior เพิ่มเกิน source

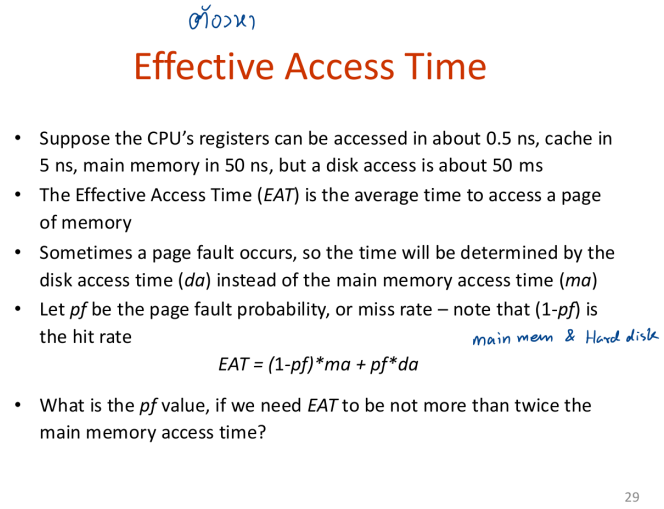
6.8 Effective Access Time (EAT)

Lecture ยกตัวอย่างเวลาเข้าถึงโดยประมาณ: register 0.5 ns, cache 5 ns, main memory 50 ns และ disk 50 ms หาก p_f คือความน่าจะเป็นที่จะเกิด page fault จะคำนวณ EAT ได้ดังนี้:

$$EAT = (1 - p_f) \cdot ma + p_f \cdot da \tag{3}$$

ma คือเวลาเข้าถึง main memory และ da คือเวลาเข้าถึง disk ตามแบบจำลองอย่างง่ายของ Lecture

ภาพประกอบ 6.14 Lecture 6 หน้า 29 — Effective Access Time



จุดสังเกต

- ค่าบนสไลด์คือ register 0.5 ns, cache 5 ns, RAM 50 ns และ disk 50 ms จึงเห็นได้ว่า disk ช้ากว่า RAM มาก
- สูตรที่ Lecture ใช้คือ $EAT = (1 - p_f)ma + p_f da$
- หากกำหนดให้ EAT ไม่เกิน 2 เท่าของเวลาเข้าถึง main memory ให้แก้อสมการเพื่อหาค่าสูงสุดของ p_f

ที่มา: Lecture 6 หน้า 29 พร้อมหมายเหตุเปรียบเทียบ main memory กับ hard disk

ตัวอย่างคำนวณ

โจทย์ใน Lecture ต้องการให้ $EAT \leq 2 \times ma$ เมื่อ $ma = 50$ ns และ $da = 50$ ms
แปลง 50 ms $= 50{,}000{,}000$ ns.

$$(1 - p_f)50 + p_f(50{,}000{,}000) \leq 100$$

$$50 + p_f(49{,}999{,}950) \leq 100$$

$$p_f \leq \frac{50}{49{,}999{,}950} \approx 1.0 \times 10^{-6}$$

หรือประมาณ 0.0001%.
แม้ page-fault probability จะต่ำมาก แต่ average time ก็เพิ่มขึ้นได้มาก เพราะ disk ใช้เวลาเข้าถึงนานกว่า memory หลายลำดับขั้น

จุดที่มักสับสน

สูตร EAT นี้เป็นแบบจำลองอย่างง่ายตาม Lecture อย่าเพิ่มเวลา TLB lookup, การทำงานของ fault handler หรือการเขียน dirty page กลับ disk หากโจทย์ไม่ได้กำหนด เพราะจะเป็นคนละแบบจำลอง

6.9 Thrashing

เมื่อ process ได้รับ frames ไม่พอ การอ้างอิง page จะเกิด fault บ่อย OS ต้องแทนที่ page เดิม แต่ไม่นาน process อาจต้องใช้ page ที่เพิ่งถูกนำออกจึงเกิด fault ซ้ำ CPU utilization จึงต่ำ หาก OS เพิ่ม degree of multiprogramming เพราะเห็น CPU ว่าง การแย่ง frames จะยิ่งรุนแรงและทำให้สถานการณ์แย่ลง

เนื้อหาจาก Lecture

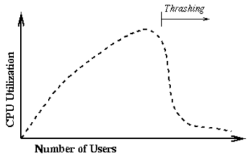
Thrashing คือภาวะที่ process ใช้เวลาส่วนใหญ่ย้าย pages เข้าออก memory แทนที่จะทำ computation ที่เป็นประโยชน์

ภาพประกอบ 6.15 Lecture 6 หน้า 30 — Thrashing Intuition

OS add Process เพิ่ม: แล้วเกิด Page fault ตลอดเวลา
OS จึงตัดว่าเราอ่านงาน : น้อยแต่ swap in/out

Thrashing Problem

- If a process does not have "enough" pages, the page fault (miss) rate is very high
  - Page reference causes page fault
  - Replace an existing page
  - But quickly need this page again
  - This leads to:
    - Low CPU utilization
    - OS thinking that it needs to increase the degree of multiprogramming – another process is added to the system
- Thrashing ≡ a process is busy swapping pages in and out



30

จุดสังเกต

- page fault ถี่จน CPU ทำ useful work น้อยและรอ disk มาก
- การเพิ่ม degree of multiprogramming ตอนนี้อาจยิ่งแย่ เพราะ frames ต่อ process ลดลง
- การจัด frame และ Working Set จึงต้องพิจารณา locality ของงาน ไม่ใช่ดูเพียงว่ามี RAM ว่างเท่าไร

ที่มา: Lecture 6 หน้า 30 พร้อมหมายเหตุเรื่อง page faults และการ swap ที่เกิดซ้ำ

6.10 การจัดสรร Frame: Global กับ Local

	Global Replacement	Local Replacement
Victim pool	ใช้ได้ทุก frame ใน physical memory process หนึ่งอาจแย่ง frame จากอีก process	จำกัดอยู่ในชุด frame ที่จัดให้ process นั้น
Frame ที่ว่าง	process ต่าง ๆ ใช้ร่วมกันได้	จัดสรรตาม policy ที่กำหนด
การจัดสรร	แบ่งใช้ร่วมกันทั้งระบบ	แบ่งเท่ากันหรือเป็นสัดส่วนตามขนาด/priority
ผลต่อกัน	process หนึ่งอาจกระทบ performance ของอีก process	การแทน page ของ process อื่นไม่กระทบโดยตรง

6.11 Working Set

Working Set WS_i ของ process P_i คือจำนวน pages ที่ process อ้างอิงในช่วงเวลาล่าสุด Δ ใช้ประมาณชุด pages ที่จำเป็นต่อการทำงานใน locality ปัจจุบัน

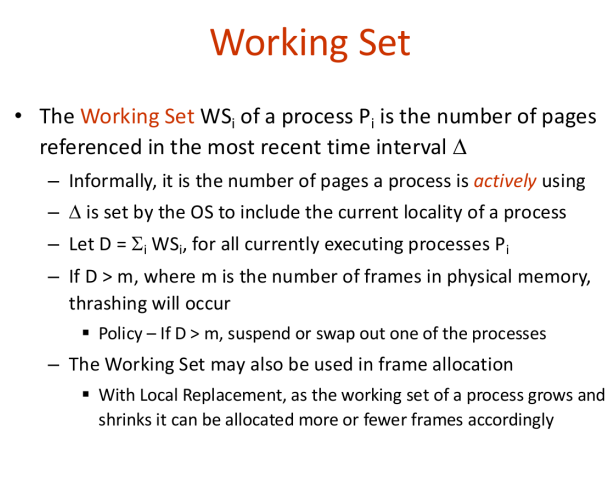
$$D = \sum_i WS_i \tag{4}$$

ให้ m เป็นจำนวน physical frames ที่มีอยู่ หาก $D > m$ แสดงว่า frames ไม่พอต่อ working sets ของทุก process Lecture จึงเสนอให้ suspend หรือ swap out บาง process Working Set ยังช่วยปรับจำนวน frames ที่จัดให้แต่ละ process ใน Local Replacement ตาม locality ของงาน

ตัวอย่างคำนวณ

ตัวอย่างเสริม: สมมติมีสาม processes โดย $WS_1 = 5$, $WS_2 = 7$ และ $WS_3 = 4$ จะได้ $D = 16$ หาก physical memory มี $m = 14$ frames จะเห็นว่า $D > m$ จึงมีความต้องการใช้ frame มากกว่าที่มีอยู่ OS ควรลดจำนวน process ที่ทำงานพร้อมกัน เช่น suspend หรือ swap out บาง process

ภาพประกอบ 6.16 Lecture 6 หน้า 33 — Working Set



จุดสังเกต

- WS_i นับ pages ที่ process P_i reference ใน recent interval Δ
- $D = \sum_i WS_i$ เป็น total demand estimate
- ถ้า demand มากกว่า available frames ($D > m$ ตาม Lecture) ระบบเสี่ยง/เกิด thrashing และควรลด active demand

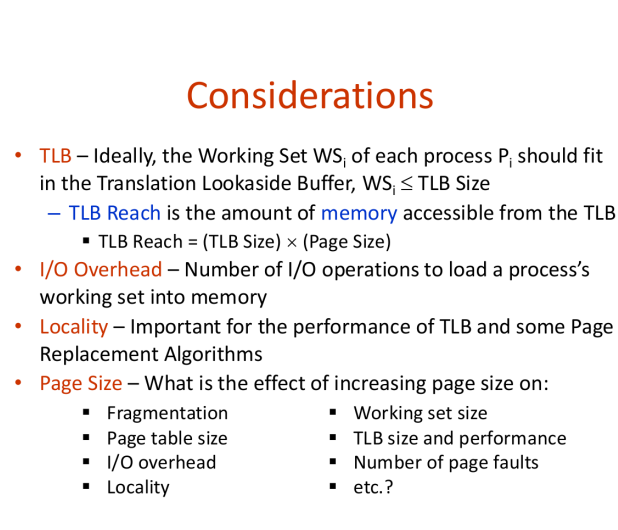
ที่มา: Lecture 6 หน้า 33.

6.12 ประเด็นเชื่อมโยงท้ายบท: TLB Reach, Locality และ Page Size

Lecture ปิดท้ายด้วยความสัมพันธ์ของประเด็นเหล่านี้:

- ในอุดมคติ working set ของ process ควรเก็บได้ใน TLB: $WS_i \leq TLB\ Size$ ตามแบบจำลองในสไลด์
- **TLB Reach** = (TLB Size)×(Page Size)
- I/O overhead = จำนวน I/O operations เพื่อโหลด working set
- locality สำคัญต่อ TLB และ replacement algorithms
- page size กระทบ fragmentation, page-table size, I/O overhead, locality, working-set size, TLB performance และ page faults

ภาพประกอบ 6.17 Lecture 6 หน้า 34 — Final VM Considerations



จุดสังเกต

- TLB Reach = TLB entries × Page Size
- locality กระทบทั้ง TLB และ Page Replacement
- page size เชื่อม fragmentation, page-table size, I/O overhead, working set, TLB performance และ page faults พร้อมกัน

ที่มา: Lecture 6 หน้า 34 เชื่อม Paging ใน Lecture 5 กับ Virtual Memory ใน Lecture 6

> **เชื่อมกับ Computer Architecture**
>
> TLB reach แสดง trade-off ระหว่าง hardware กับการจัดการ memory ได้ชัดเจน จำนวน TLB entries มีจำกัด ขณะที่ page size เป็นตัวกำหนดว่าแต่ละ entry ครอบคลุม memory ได้เท่าใด อย่างไรก็ตาม page ขนาดใหญ่ก็อาจเพิ่ม internal fragmentation ได้ จึงไม่มีค่าหนึ่งที่ดีที่สุดในทุกด้าน

6.13 Chapter Summary

สรุปท้ายบท

- Virtual Memory ใช้ backing storage เพื่อทำให้ logical address space ไม่จำเป็นต้อง resident ใน RAM ทั้งหมด
- Process Switching เปลี่ยน CPU context; Process/Page Swapping ย้าย memory content กับ disk
- เมื่อ present bit=0 จะเกิด Page Fault และเข้า handler เพื่ออ่าน page จาก disk ระหว่างนั้น process จะ Blocked เมื่อ I/O interrupt แจ้งว่างานเสร็จ OS อัปเดต PTE และเปลี่ยน process เป็น Ready ก่อนกลับมารัน instruction เดิม
- Selection policy ตัดสินเมื่อใดเอา page เข้า; Replacement policy ตัดสิน victim เมื่อเอา page ออก
- FIFO อาจมี Belady's anomaly; LRU ใช้ past recency; LFU ใช้ frequency; OPT ใช้อนาคตและเป็น benchmark
- Second Chance ใช้ reference bit + circular queue เพื่อ approximate recency แบบเบากว่า exact LRU
- EAT ชี้ว่าความต่าง disk vs RAM ทำให้ page-fault rate ต้องต่ำมาก
- Thrashing เกิดเมื่อ resident pages ไม่พอกับ locality; Working Set และ frame allocation ช่วยควบคุม demand

Key Mechanisms

Present bit, page-fault trap, context save/restore, disk I/O, I/O completion interrupt, PTE update, instruction restart, reference/modify bits.

Key Policies

Demand/Request/Pre-paging; FIFO/LRU/LFU/OPT/Second Chance; Global/Local frame allocation; Working-Set-based suspension/allocation.

Formula / Calculation Recap

$$EAT = (1 - p_f)ma + p_f da, \qquad D = \sum_i WS_i, \qquad TLB\ Reach = (TLB\ Size)(Page\ Size)$$

Common Confusions

- TLB miss ต่างจาก Page Fault
- Page Fault ต่างจาก segmentation/protection fault
- Context Switch ต่างจาก Process Swapping
- Page selection ต่างจาก Page replacement
- FIFO queue order ต่างจาก LRU recency order
- OPT ใช้จริงทั่วไปไม่ได้เพราะไม่รู้อ้างอิงอนาคต; ใช้ benchmark

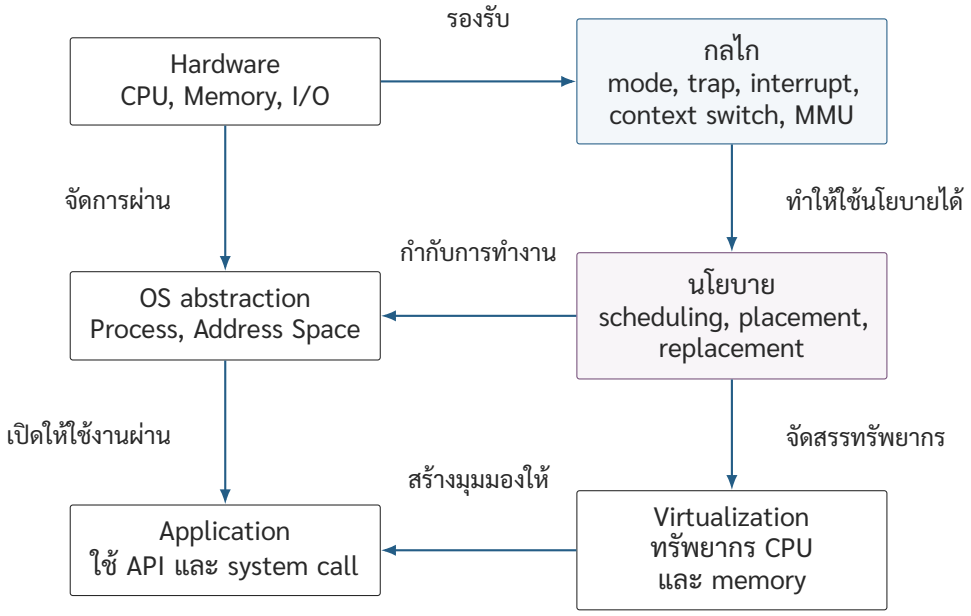
Quick Revision Checklist

☐ วาด end-to-end page-fault flow พร้อม process states และ I/O interrupt ได้
☐ แยก Demand/Request/Pre-paging ได้
☐ trace FIFO/LRU/LFU/OPT/Second Chance จาก reference string ได้
☐ อธิบาย Belady's anomaly ด้วย Example 2 ได้
☐ แก้ EAT inequality และแปลง ms↔ns ถูก
☐ อธิบาย thrashing และ Working Set condition $D > m$ ได้
☐ แยก Global vs Local replacement ได้

CPE333 FINAL REVIEW

ทบทวนความเชื่อมโยงระหว่าง CPU Virtualization, Process, Scheduling, Address Translation และ Virtual Memory

R.1 แผนภาพรวมแนวคิดของ OS



ภาพนี้ช่วยให้เห็นว่าเนื้อหา 6 Lecture ต่อเนื่องกัน ไม่ได้แยกเป็นหัวข้อที่ไม่เกี่ยวข้อง:

1. Lecture 1 วางแนวคิด OS abstraction และ control boundary
2. Lecture 2 อธิบาย process และ LDE ซึ่งทำให้เกิด CPU virtualization
3. Lecture 3 ศึกษา scheduler policy สำหรับเลือกผู้ใช้ CPU
4. Lecture 4 เริ่ม memory virtualization ด้วยการแปล logical address เป็น physical address ผ่าน Base/Bounds
5. Lecture 5 ขยายไปสู่ noncontiguous translation ด้วย Segmentation, Paging และ TLB
6. Lecture 6 ใช้ disk เป็น backing store ทำให้ memory abstraction ใหญ่กว่า RAM และเชื่อม CPU, MMU, I/O, process state กับ scheduling เข้าด้วยกัน

R.2 เปรียบเทียบ Mechanism กับ Policy

หัวข้อ	Mechanism: ระบบทำงานอย่างไร	Policy: ควรเลือกอะไรหรือเมื่อใด
การควบคุม CPU	User/Kernel mode, trap, timer interrupt และ context switch	เลือก process ด้วย FIFO/ SJF/ STCF/ RR/ MLFQ/Lottery
Process state	บันทึก/ คืน ค่า PCB และ จัด ready/ block queue	เลือก process ถัดไป
Contiguous memory	แปล address ด้วย Base/Bounds และแจ้ง exception	เลือก partition และตำแหน่งวาง
Segmentation	ใช้ segment table และตรวจ bounds	เลือกพื้นที่ด้วย First/Best/Worst Fit
Paging	แปล address ด้วย Page Table, MMU และ TLB	เลือก page size และจัดการ free frame
Virtual Memory	ตรวจ present bit, page-fault trap, disk I/O และอัปเดต PTE	เลือก page เข้า/ออกและจัดสรร frame

จุดทบทวน

เมื่อถามว่า OS ทำงานอย่างไร ให้อธิบาย **mechanism** ที่ทำให้ทำงานนั้นได้ก่อน หากถามว่าควรเลือกวิธีใดหรือจัดสรรทรัพยากรให้ใคร ให้อธิบาย **policy** พร้อม metric และ trade-off ที่เกี่ยวข้อง

R.3 ลำดับการเปลี่ยน State และ Flow สำคัญ

R.3.1 System Call

User code เรียก library หรือ API ที่นำไปสู่ trap จากนั้น kernel handler ทำงานด้วยสิทธิ์ที่เหมาะสม เมื่อเสร็จแล้ว return-from-trap คืน control ให้ user code

R.3.2 Preemptive Context Switch

เมื่อ Process A กำลังรันแล้วเกิด timer interrupt OS จะบันทึก context ของ A ลง PCB จากนั้น scheduler เลือก Process B และคืน context ของ B ก่อนให้ B ทำงานต่อ

R.3.3 I/O state transition

Process ที่กำลัง Running จะเปลี่ยนเป็น **Blocked** เมื่อร้องขอ I/O และต้องรอ เมื่อ I/O เสร็จ interrupt จะแจ้ง OS ให้เปลี่ยนเป็น **Ready** จากนั้น process จะกลับเป็น Running เมื่อ scheduler เลือกให้ใช้ CPU

R.3.4 Page Fault

เมื่ออ้างอิง VA แล้วพบว่า page ไม่อยู่ใน RAM จะเกิด page-fault trap OS เริ่มอ่าน page จาก disk และเปลี่ยน process เป็น Blocked ระหว่างนั้น process อื่นใช้ CPU ได้ เมื่อ I/O เสร็จ OS อัปเดต PTE และเปลี่ยน process กลับเป็น Ready เมื่อได้รับ CPU อีกครั้งจึงคืน context และ **เริ่ม instruction เดิมใหม่**

R.4 Formula Sheet

R.4.1 Scheduling

$$T_{turnaround} = T_{completion} - T_{arrival}$$

$$T_{response} = T_{first\text{-}run} - T_{arrival}$$

R.4.2 Base / Bounds

$$VA < Bounds \Rightarrow PA = Base + VA; \qquad VA \geq Bounds \Rightarrow Fault$$

R.4.3 Segmentation

$$LA = (s, d), \quad d < Bounds_s \Rightarrow PA = Base_s + d$$

R.4.4 Paging

$$PN = \left\lfloor \frac{VA}{PageSize} \right\rfloor, \qquad offset = VA \bmod PageSize$$

$$PA = PFN \times PageSize + offset$$

Offset bits $= \log_2(PageSize)$ เมื่อ page size เป็น power of two

R.4.5 Virtual Memory

$$EAT = (1 - p_f)ma + p_f da$$

$$D = \sum_i WS_i, \qquad D > m \Rightarrow \text{thrashing condition ตาม Lecture}$$

$$TLB\ Reach = (TLB\ Size)(Page\ Size)$$

R.5 ตารางเปรียบเทียบ Scheduling

นโยบาย	metric หลัก	กฎเลือก	ข้อควรพิจารณา
FIFO	baseline เรียบง่าย	เรียงตามเวลามาถึง	งานสั้นอาจต้องรอหลังงานยาว
SJF	turnaround	เลือกงานที่สั้นที่สุด	ไม่แย่ง CPU และต้องรู้ความยาวงาน
STCF	turnaround	เลือกเวลาที่เหลือน้อยที่สุด	แย่ง CPU ได้ แต่อาจตอบสนองช้า
RR	response/fairness	หมุนตาม quantum	คำนึงถึง quantum และ switch overhead
MLFQ	turnaround และ response	ปรับ priority ตามพฤติกรรม	ใช้ boost และ accounting ป้องกัน starvation/gaming
Lottery	สัดส่วน CPU	สุ่มจาก ticket	ผลระยะสั้นแปรผันตามการสุ่ม

R.6 ตารางเปรียบเทียบการจัดการ Memory

	Contiguous: Base/Bounds	Segmentation	Paging
ตำแหน่งใน memory	หนึ่ง block ต่อ process	วางแต่ละ segment แยกกัน	วาง page ใน frame ใดก็ได้
หน่วยที่จัดสรร	ทั้ง address space	segment ขนาดไม่เท่ากัน	page/frame ขนาดเท่ากัน
การแปลงที่อยู่	Base+VA	segment Base+offset	Page Table แปลง PN เป็น PFN
การป้องกัน	ตรวจ bounds	ตรวจ bounds/ permissions ต่อ segment	ใช้ PTE/MMU
Fragmentation	ขึ้นกับ scheme	external	internal
Metadata	น้อยมาก	segment table ขนาดเล็ก	page table อาจมีขนาดใหญ่

R.7 เปรียบเทียบโครงสร้าง Page Table

โครงสร้าง	ใช้เมื่อใดหรือเพื่ออะไร	ข้อแลกเปลี่ยนหลัก
Linear	ใช้ PN เป็น index โดยตรง	ใช้ memory มากและต้องอ่าน Page Table
Two-Level	สร้างตารางระดับล่างเฉพาะส่วนที่ใช้	เพิ่มจำนวน lookup ที่ต้องทำ
Inverted	มีหนึ่ง entry ต่อ physical frame	ต้องค้น mapping ของ (PID, VPN)
Hashed	เก็บเฉพาะ mapping ที่ valid	ต้องค้น hash bucket และ chain
TLB	cache translation ที่ถูกใช้บ่อยใน hardware	มี reach จำกัด และอาจต้อง flush ตอน context switch ตามแบบจำลองใน Lecture

R.8 เปรียบเทียบ Page Replacement

นโยบาย	วิธีเลือก victim	ประเด็นที่ควรจำ
FIFO	เลือก page เก่าที่สุดใน queue	ใช้ queue และอาจเกิด Belady's anomaly
LRU	เลือก page ที่ไม่ได้ใช้นานที่สุด	ใช้ประวัติการอ้างอิงและไม่เกิด Belady's anomaly เมื่อมี stack property
LFU	เลือก page ที่มีตัวนับใช้งานต่ำสุด	หาก count เท่ากันต้องกำหนดกฎตัดสิน เช่น FIFO
OPT	เลือก page ที่จะถูกเรียกใช้อีกช้าที่สุด	ให้ faults ต่ำสุด ใช้เป็น benchmark แต่ต้องรู้อนาคต
Second Chance	FIFO ร่วมกับ reference bit	หาก ref=1 ให้ reset bit แล้วตรวจ page ถัดไป

R.9 จุดที่มักสับสน

1. **Program ต่างจาก Process**: program คือ code/data ที่จัดเก็บไว้ ส่วน process รวม state ที่ใช้ระหว่างทำงาน
2. **Ready ต่างจาก Blocked**: Ready พร้อมใช้ CPU ส่วน Blocked ต้องรอ event
3. **Procedure Call ต่างจาก System Call**: มี privilege และ control boundary ต่างกัน
4. **Trap ต่างจาก Hardware Interrupt**: trap เกิดจาก instruction หรือ exception ส่วน interrupt เป็นเหตุการณ์ภายนอกที่เกิดไม่พร้อมกับ instruction
5. **Policy ต่างจาก Mechanism**: RR/MLFQ เป็น policy ส่วน timer/context switch เป็น mechanism
6. **Context Switch ต่างจาก Process Swapping**: อย่างแรกเปลี่ยน CPU state อย่างหลังย้ายข้อมูลระหว่าง memory กับ disk
7. **Logical ต่างจาก Physical Address**
8. **Internal ต่างจาก External Fragmentation**
9. **Page ต่างจาก Frame**
10. **Page Table ต่างจาก TLB**
11. **TLB miss ต่างจาก Page Fault**
12. **Page Fault ต่างจาก protection/segmentation fault**
13. **Page Selection ต่างจาก Page Replacement**
14. **SJF ต่างจาก STCF**: SJF ไม่แย่ง CPU ส่วน STCF แย่ง CPU ได้และพิจารณา remaining time
15. **RR quantum too short is not free**: context-switch overhead can dominate

R.10 คำถามทบทวน 30 ข้อ

1. OS อยู่ระหว่าง application กับ hardware เพื่ออะไร?
2. User Mode และ Kernel Mode ต่างกันอย่างไร?
3. System Call ใช้ trap ทำอะไร?
4. ทำไม trap table ต้องถูกตั้งโดย kernel?
5. Process ต่างจาก Program อย่างไร?
6. P,D,C,S ของ Process คืออะไร?
7. PCB ต้องเก็บอะไรเพื่อ resume process?
8. Ready, Running, Blocked เปลี่ยนกันอย่างไร?
9. Limited Direct Execution มี 3 ส่วนอะไร?
10. Cooperative control พังอย่างไรเมื่อ process infinite loop?
11. Timer Interrupt ช่วย OS regain CPU อย่างไร?
12. Scheduler กับ Context Switch ต่างกันอย่างไร?
13. fork(), wait(), exec() ทำคนละอะไร?
14. Turnaround กับ Response formula ต่างกันอย่างไร?
15. FIFO drawback คืออะไร?
16. ทำไม SJF optimal เฉพาะภายใต้ assumptions บางชุด?
17. STCF ใช้ preemption อย่างไร?
18. Quantum สั้น/ยาว trade-off อย่างไร?
19. MLFQ final 5 rules คืออะไร?
20. Priority Boost แก้อะไร? Better Accounting แก้อะไร?
21. Lottery ticket 75/25 แปลว่าอะไร?
22. Logical address ต่าง physical address อย่างไร?
23. Base/Bounds translation ทำสองขั้นอะไร?
24. Static vs Dynamic relocation ต่างกันอย่างไร?
25. Internal vs External fragmentation อยู่ ตรงไหน ของ allocation?
26. Segmentation กับ Paging ต่างกันอย่างไร?
27. PN/offset แยกจาก address bit อย่างไร?
28. TLB hit/miss flow เป็น อย่างไร และ ทำไม miss ไม่เท่ากับ page fault?
29. Page Fault end-to-end ตั้งแต่ CPU ถึง disk แล้วกลับมา restart instruction อย่างไร?
30. Thrashing, Working Set, $D > m$ เชื่อมกันอย่างไร?

R.11 Last-Minute Trace Drills

แบบฝึก A — CPU Virtualization

ลองเขียนลำดับภายใน 60 วินาที เริ่มจาก user process ที่กำลังรัน เมื่อเกิด timer interrupt OS เข้า handler และบันทึก context ลง PCB จากนั้น scheduler เลือก process ถัดไป ก่อนคืน context เพื่อให้ทำงานต่อ

แบบฝึก B — Scheduling

กำหนด arrival time และ burst time ของ jobs แล้ววาด Gantt chart สำหรับ FIFO/SJF/STCF/RR ก่อนคำนวณ metric การวาด timeline ช่วยป้องกันการสลับ completion time กับ first-run time

แบบฝึก C — Address Translation

ก่อนทำโจทย์ให้เขียน field ของ address ก่อน: Base/Bounds = VA, Segmentation = $s|d$, Paging = $PN|offset$ และ Hybrid = $s|p|po$

แบบฝึก D — Page Replacement

สร้างตารางที่มีคอลัมน์ Reference, Frames, Fault? และ Victim ในทุก step ติดตาม queue สำหรับ FIFO, ลำดับการใช้งานสำหรับ LRU, count และกฎตัดสินสำหรับ LFU ส่วน OPT ต้องดู reference ในอนาคต

แบบฝึก E — Page Fault

คำตอบไม่ควรหยุดแค่ OS โหลด page จาก disk ให้กล่าวถึง Blocked state, การทำงานของ process อื่น, I/O completion interrupt, การอัปเดต PTE และการเริ่ม instruction เดิมใหม่ให้ครบ

> **สรุปท้ายบท**
>
> ภาพรวมที่ต้องเข้าใจก่อนสอบคือ hardware ให้กลไกที่ OS ใช้สร้าง abstraction และควบคุมทรัพยากร Process ใช้ CPU ตาม policy ของ scheduler ส่วน MMU แปลที่อยู่หน่วยความจำ และ Virtual Memory ใช้ disk เป็น backing storage เมื่อ page ที่ต้องใช้ไม่อยู่ใน RAM จะเกิด Page Fault ซึ่งเชื่อม trap, I/O, process state และ scheduling เข้าด้วยกัน
> ใช้ภาพรวมนี้ควบคู่กับการฝึกคำนวณ scheduling timeline, address translation และ page replacement เพื่อเชื่อมแต่ละ Lecture ให้เป็นระบบเดียว